

DAFTAR ISI

DAFTAR ISI	1
BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Rumusan masalah.....	6
1.3 Tujuan Penelitian.....	7
1.4 Batasan Masalah.....	7
BAB II TINJAUAN PUSTAKA	11
2.1 State of the Art	11
2.2 Tinjauan Teoritis.....	18
2.2.1 Notulensi Rapat	18
2.2.2 Automatic Speech Recognition (ASR).....	19
2.2.3 Disfluensi dan Filler Words dalam Ucapan Spontan.....	20
2.2.4 Word Filler dan Noise Filter	21
2.2.5 Model OpenAI Whisper	22
2.2.6 Information Overload	24
2.2.7 Pemrosesan Bahasa Alami (Natural Language Processing).....	25
2.2.8 Peringkasan Teks Otomatis (Automatic Text Summarization)	26

2.2.9	Algoritma LexRank	27
2.2.10	Large Language Model (LLM)	29
2.2.11	Model Llama 3.2 dan Platform Ollama	29
2.2.12	Pendekatan Hybrid Extractive-Abstractive Summarization	30
2.2.13	TF-IDF dengan Sublinear TF dan Smooth IDF	31
2.2.14	Jaccard Similarity untuk Deduplikasi Kalimat	33
2.2.15	Metrik Evaluasi Akurasi Sistem	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Rapat adalah segala bentuk media komunikasi kelompok yang berlangsung secara langsung dan sering diadakan oleh berbagai organisasi[1]. Melalui rapat, para pemangku kepentingan dapat bertukar informasi, menyelaraskan visi, serta merumuskan keputusan-keputusan strategis yang berdampak langsung terhadap arah kebijakan organisasi. Dalam konteks tata kelola pemerintahan digital (*e-government*), kualitas dokumentasi rapat menjadi semakin krusial karena keputusan yang dihasilkan harus dapat dipertanggungjawabkan secara transparan kepada publik. Notulensi rapat berperan sebagai *audit trail* resmi—rekam jejak legal yang memuat siapa yang hadir, apa yang didiskusikan, dan keputusan apa yang ditetapkan. Tanpa dokumentasi yang akurat dan terstruktur, keputusan organisasi rentan terhadap miskomunikasi, sengketa, bahkan potensi pelanggaran hukum administrasi[2]. Kajian mutakhir menegaskan bahwa rekam rapat pemerintahan lokal—dalam bentuk notulensi maupun transkrip—adalah dokumen resmi yang mendukung akuntabilitas publik dan keterlibatan warga, namun kerap bersifat padat, birokratis, dan sangat heterogen antar institusi sehingga menyulitkan interpretasi manusia maupun analisis komputasional secara sistematis [2].

Secara konvensional, proses pencatatan notulensi masih sangat bergantung pada kemampuan individual seorang notulis. Notulis dituntut untuk mendengarkan, memahami konteks, menyaring informasi penting, sekaligus menulis secara bersamaan—sebuah kondisi yang menimbulkan beban kognitif yang sangat tinggi. Ketika peserta rapat berbicara dengan cepat atau terjadi percakapan tumpang tindih (*overlapping speech*), notulis kerap melewatkan poin-poin kritis yang seharusnya terdokumentasi. Sebagai solusi sementara, banyak organisasi mulai merekam jalannya rapat dalam format audio atau video. Namun demikian, rekaman tersebut tetap memerlukan proses transkripsi manual yang memakan waktu berjam-jam, bahkan untuk durasi rapat yang relatif singkat. Proses ini tidak hanya menguras

tenaga, tetapi juga menunda ketersediaan notulensi sehingga berimbas pada keterlambatan pengambilan keputusan tindak lanjut. Konsolidasi catatan dan pembuatan notulensi rapat tanpa bantuan otomasi masih dianggap sebagai tugas yang melelahkan dan memakan waktu bagi peserta rapat, bahkan di era ketika layanan transkripsi berbasis komputasi telah banyak tersedia[3] .

Persoalan tidak berhenti pada proses transkripsi. Bahkan ketika transkripsi berhasil diselesaikan, hasil konversi 100% atau *verbatim* dari rekaman rapat menghasilkan dokumen teks yang sangat panjang, penuh dengan kata-kata pengisi (*filler words*) seperti "anu", "ya", "jadi", pengulangan kalimat, serta percakapan basa-basi di luar topik substantif. Seorang pimpinan yang harus meninjau kembali isi rapat akan dihadapkan pada dokumen yang mungkin berpuluh-puluh halaman panjangnya hanya untuk menemukan dua atau tiga keputusan kunci. Fenomena ini dikenal sebagai *information overload*—kondisi di mana volume informasi yang diterima melebihi kapasitas pemrosesan kognitif manusia sehingga justru menghambat produktivitas dan kualitas keputusan. Sebuah kajian sistematis mengonfirmasi bahwa *information overload* merupakan permasalahan yang semakin diperparah oleh digitalisasi dunia kerja dan meningkatnya penggunaan teknologi informasi dan komunikasi; kondisi ini secara konsisten dikaitkan dengan menurunnya kinerja, meningkatnya stres kerja, dan terganggunya proses pengambilan keputusan yang efektif dalam organisasi [4].

Perkembangan teknologi kecerdasan buatan, khususnya di bidang pemrosesan ucapan, menawarkan solusi transformatif atas permasalahan transkripsi manual tersebut. *Automatic Speech Recognition* (ASR) adalah teknologi yang memungkinkan komputer mengonversi tuturan manusia menjadi teks secara otomatis dengan kecepatan dan konsistensi yang jauh melampaui kemampuan manusia. Di antara berbagai model ASR yang tersedia saat ini, OpenAI Whisper telah muncul sebagai model *state-of-the-art* yang paling menonjol. Whisper dilatih menggunakan metode *weak supervision* pada 680.000 jam data audio berlabel dari berbagai bahasa di seluruh dunia, termasuk Bahasa Indonesia, menjadikannya model yang sangat tangguh terhadap kondisi audio yang bervariasi, mulai dari

aksen yang beragam hingga lingkungan bersuara bising (*noise-robust*). Dengan *Word Error Rate* (WER) yang sangat rendah untuk Bahasa Indonesia, Whisper menjadi fondasi teknologi yang ideal untuk sistem transkripsi rapat otomatis [5].

Meskipun Whisper mampu menghasilkan transkripsi yang akurat, dokumen transkripsi penuh tetap menyisakan masalah *information overload* sebagaimana diuraikan sebelumnya. Oleh karena itu, diperlukan lapisan teknologi tambahan berupa *Automatic Text Summarization* (ATS) yang mampu mendistilasi inti sari rapat dari keseluruhan transkripsi. Penelitian ini mengadopsi algoritma LexRank sebagai pondasi pendekatan *extractive summarization*. LexRank bekerja dengan membangun representasi graf dari keseluruhan dokumen, di mana setiap kalimat adalah sebuah simpul (*node*) dan bobot antar simpul ditentukan oleh nilai kemiripan kosinus (*cosine similarity*) berbasis TF-IDF. Algoritma kemudian menerapkan prinsip sentralitas leksikal berbasis graf—serupa dengan PageRank—untuk mengidentifikasi kalimat-kalimat yang paling representatif dan sentral dalam keseluruhan wacana[6]. Dalam konteks teks berbahasa Indonesia, LexRank telah terbukti mampu menghasilkan ringkasan ekstraktif yang koheren dan relevan [7]. Namun, kelemahan inheren dari pendekatan ekstraktif murni adalah kalimat-kalimat yang terpilih kadang terasa kaku, terpotong di tengah, atau kehilangan konteks transisional sehingga ringkasan sulit dibaca sebagai narasi yang utuh. Untuk mengatasi keterbatasan ini, penelitian ini memperkenalkan nilai kebaruan (*novelty*) utama: sebuah pendekatan hibrida *extractive-abstractive* yang memadukan LexRank dengan *Large Language Model* (LLM) lokal, yaitu Llama 3.2 yang dijalankan melalui platform Ollama. Kalimat-kalimat hasil ekstraksi LexRank digunakan sebagai konteks masukan bagi Llama 3.2 untuk dilakukan *abstractive rewriting*—proses penulisan ulang secara cerdas menjadi narasi yang alami, kohesif, dan mudah dipahami tanpa harus terhubung ke layanan *cloud* berbayar. Pendekatan hibrida semacam ini terbukti secara signifikan mengungguli pendekatan ekstraktif atau abstraktif tunggal dalam hal kualitas keterbacaan dan kepadatan informasi [8].

Berdasarkan seluruh uraian permasalahan dan solusi teknologi di atas, penelitian ini bertujuan untuk merancang dan membangun sebuah Sistem Informasi Manajemen Notulensi Rapat Otomatis Berbasis Web yang mengintegrasikan seluruh komponen teknologi tersebut dalam satu platform terpadu. Secara arsitektural, sistem ini dibangun dengan antarmuka web berbasis PHP dan basis data MySQL sebagai lapisan presentasi dan manajemen data, yang berkomunikasi secara *real-time* dengan *backend microservice* berbasis Python menggunakan kerangka kerja Flask dan protokol Socket.io. Sistem ini mendukung dua moda transkripsi utama: (1) *Live Transcription*, yaitu transkripsi langsung dari mikrofon secara *real-time* selama rapat berlangsung; dan (2) transkripsi asinkronus melalui unggahan berkas rekaman audio atau video yang diproses di *background* tanpa menginterupsi aktivitas pengguna. Setelah transkripsi selesai, sistem secara otomatis menjalankan *pipeline* summarisasi hibrida LexRank + Llama 3.2 untuk menghasilkan notulensi ringkas yang siap baca, yang kemudian dapat langsung diakses, diedit, dan diunduh oleh pengguna melalui satu *dashboard* terintegrasi. Dengan demikian, sistem ini diharapkan dapat menjadi solusi komprehensif yang menjawab tantangan dokumentasi rapat di era digital.

1.2 Rumusan masalah

Berdasarkan latar belakang yang telah dipaparkan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun sistem notulensi rapat otomatis berbasis web yang mampu melakukan transkripsi audio rapat menjadi teks menggunakan model OpenAI Whisper secara *real-time* maupun melalui unggahan berkas rekaman?
2. Bagaimana tingkat akurasinya berdasarkan metrik *Word Error Rate* (WER), *Word Accuracy* (WAcc), dan *Character Error Rate* (CER) pada data rapat berbahasa Indonesia?
3. Bagaimana mengimplementasikan metode *hybrid extractive-abstractive summarization* menggunakan algoritma LexRank dan model Llama 3.2

melalui Ollama untuk menghasilkan ringkasan rapat yang ringkas, kohesif, dan mudah dipahami, serta bagaimana kualitas ringkasan yang dihasilkan berdasarkan metrik ROUGE-L (*Precision*, *Recall*, dan *F1-Score*)?

1.3 Tujuan Penelitian

Sesuai dengan rumusan masalah di atas, penelitian ini bertujuan untuk:

1. Merancang dan membangun sistem notulensi rapat otomatis berbasis web yang mampu melakukan transkripsi audio rapat menjadi teks menggunakan model OpenAI Whisper secara *real-time* maupun melalui unggahan berkas rekaman
2. Mengukur tingkat akurasi transkripsi sistem berbasis OpenAI Whisper menggunakan metrik *Word Error Rate* (WER), *Word Accuracy* (WAcc), dan *Character Error Rate* (CER) pada data rapat berbahasa Indonesia.
3. Mengimplementasikan metode *hybrid extractive-abstractive summarization* menggunakan algoritma LexRank dan model Llama 3.2 melalui Ollama untuk menghasilkan ringkasan rapat yang ringkas, kohesif, dan mudah dipahami, serta mengevaluasi kualitas ringkasan yang dihasilkan menggunakan metrik ROUGE-L (*Precision*, *Recall*, dan *F1-Score*).

1.4 Batasan Masalah

Untuk menjaga fokus penelitian dan memastikan pengembangan sistem sesuai dengan tujuan utama, maka batasan masalah dalam penelitian ini ditetapkan sebagai berikut:

1. Sistem yang dibangun difokuskan pada proses notulensi rapat otomatis berbasis web yang mencakup transkripsi audio dan pembuatan ringkasan rapat secara otomatis.

2. Sistem hanya memproses data masukan berupa audio rapat secara langsung (*live transcription*) melalui mikrofon dan unggahan berkas rekaman dalam format audio (seperti .mp3, .wav, dan .m4a) atau video (seperti .mp4) yang mengandung percakapan rapat.
3. Model *Automatic Speech Recognition* (ASR) yang digunakan pada penelitian ini adalah OpenAI Whisper tanpa melakukan pelatihan ulang (*retraining*) model.
4. Bahasa utama yang digunakan adalah Bahasa Indonesia dengan dukungan pengenalan istilah atau frasa berbahasa Inggris yang umum digunakan dalam konteks rapat (*multilingual context recognition*). Dukungan ini tidak ditujukan untuk menerjemahkan percakapan secara penuh, melainkan untuk mempertahankan istilah asing sesuai konteks pengucapan aslinya—misalnya istilah *machine learning* tetap ditulis dalam Bahasa Inggris dan tidak diterjemahkan menjadi "pembelajaran mesin", sehingga konteks materi dan makna kalimat tetap utuh serta mudah dipahami.
5. Sistem tidak mendukung penerjemahan otomatis antarbahasa (*machine translation*) maupun pemrosesan bahasa selain Bahasa Indonesia sebagai bahasa utama percakapan.
6. Metode *Automatic Text Summarization* yang digunakan merupakan pendekatan *hybrid extractive-abstractive summarization* dengan mengombinasikan algoritma LexRank dan model Meta Llama 3.2 yang dijalankan secara lokal melalui Ollama.
7. Algoritma LexRank digunakan untuk menyeleksi kalimat-kalimat penting hasil transkripsi, sedangkan Llama 3.2 digunakan untuk melakukan *abstractive rewriting* agar ringkasan menjadi lebih kohesif dan mudah dibaca.
8. Sistem tidak membahas proses pelatihan (*training*) maupun optimasi arsitektur model *Large Language Model* (LLM), melainkan hanya implementasi dan integrasi model yang telah tersedia.

9. Pengujian akurasi transkripsi dilakukan menggunakan metrik *Word Error Rate* (WER), sedangkan evaluasi kualitas ringkasan dilakukan menggunakan metrik ROUGE.
10. Dataset pengujian terbatas pada rekaman rapat berbahasa Indonesia yang dikumpulkan secara mandiri atau berasal dari sumber terbuka, dengan kemungkinan adanya istilah teknis berbahasa Inggris yang umum digunakan dalam diskusi akademik, teknologi, maupun organisasi. Jumlah dan durasi data pengujian disesuaikan dengan ketersediaan sumber daya komputasi yang digunakan.
11. Sistem dikembangkan menggunakan arsitektur berbasis web dengan PHP sebagai antarmuka aplikasi, MySQL sebagai basis data, serta *backend* Python Flask dan Socket.io untuk komunikasi dan pemrosesan *real-time*.
12. Hasil ringkasan yang dihasilkan sistem difokuskan pada penyajian poin penting, keputusan rapat, dan inti pembahasan—bukan menghasilkan notulensi formal yang sepenuhnya mengikuti standar administrasi institusi tertentu. Ringkasan yang dihasilkan dapat diunduh dalam bentuk dokumen sederhana sesuai kebutuhan pengguna.
13. Performa sistem, khususnya kecepatan transkripsi dan pembuatan ringkasan, bergantung pada spesifikasi perangkat keras yang digunakan dan tidak dioptimalkan untuk lingkungan dengan sumber daya komputasi terbatas.

1.5 Manfaat Penelitian

Berdasarkan tujuan penelitian tersebut, penelitian ini bermanfaat antara lain:

1. Bagi organisasi dan instansi pemerintah, sistem ini dapat menjadi solusi dalam mengatasi permasalahan dokumentasi rapat yang selama ini masih bergantung pada kemampuan individual notulis, sehingga setiap rapat dapat terdokumentasi secara akurat, cepat, dan dapat dipertanggungjawabkan secara transparan.

2. Bagi notulis dan peserta rapat, sistem ini dapat mengurangi beban kognitif notulis sekaligus memungkinkan seluruh peserta untuk lebih fokus pada substansi diskusi karena proses pencatatan telah dilakukan secara otomatis oleh sistem.
3. Bagi pimpinan dan pengambil keputusan, ringkasan rapat yang dihasilkan secara otomatis dapat mengurangi *information overload* sehingga hasil rapat dapat ditinjau secara efisien dan proses pengambilan keputusan tindak lanjut dapat berlangsung lebih cepat.

BAB II

TINJAUAN PUSTAKA

2.1 State of the Art

Penyusunan penelitian ini merujuk pada berbagai jurnal yang relevan dengan topik penelitian sebagai acuan. Referensi tersebut disajikan dalam format tabel yaitu penelitian terdahulu atau State of the Art. Dalam bagian ini, dijelaskan alasan pemilihan jurnal yang digunakan serta perbedaan utama antara penelitian sebelumnya dengan penelitian yang sedang dilakukan.

No.	Deskripsi
1	Penulis/Tahun : Jia Jin Koay, Alexander Roustai, Xiaojin Dai, Fei Liu / 2021 Judul Artikel : : A Sliding-Window Approach to Automatic Creation of Meeting Minutes Metode yang digunakan: : Neural Abstractive Summarizer dengan pendekatan Sliding-Window pada transkrip rapat panjang. Hasil yang diperoleh: Pendekatan sliding-window mampu menavigasi transkrip rapat yang panjang dan tidak terstruktur untuk mengidentifikasi konten penting. Evaluasi dilakukan pada tiga jenis transkrip (manusia, ASR v1, ASR v2) menggunakan metrik ROUGE, menunjukkan bahwa summarizer berhasil menangkap konten esensial rapat meskipun kualitas transkrip ASR memengaruhi performa. Penelitian ini menegaskan bahwa konsolidasi catatan tanpa otomasi masih merupakan tugas yang melelahkan dan memakan waktu Persamaan :Sama-sama membangun sistem otomatis untuk menghasilkan notulensi/ringkasan rapat dari transkrip audio menggunakan teknik NLP, serta mengevaluasi kualitas ringkasan menggunakan metrik ROUGE. Perbedaan: : Penelitian tersebut menggunakan pendekatan abstractive murni berbasis neural summarizer dengan sliding-window pada dataset AMI/ICSI berbahasa Inggris, sedangkan penelitian ini menggunakan pendekatan hybrid extractive-abstractive (LexRank + Llama 3.2) khusus untuk rapat berbahasa Indonesia. Selain itu, penelitian ini mengintegrasikan ASR Whisper sebagai komponen transkripsi dalam satu sistem terpadu berbasis web.

2	Penulis/Tahun: Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever / 2022 Judul Artikel : Robust Speech Recognition via Large-Scale Weak Supervision Metode yang digunakan: Transformer-based Automatic Speech Recognition (ASR) dilatih dengan metode weak supervision pada 680.000 jam data audio multilingual. Hasil yang diperoleh: Model Whisper menunjukkan performa state-of-the-art pada berbagai benchmark pengenalan ucapan multilingual, termasuk Bahasa Indonesia, dengan Word Error Rate (WER) yang sangat kompetitif. Whisper terbukti robust terhadap variasi aksen, noise lingkungan, dan kondisi audio yang beragam, menjadikannya model yang sangat andal untuk transkripsi otomatis tanpa fine-tuning tambahan. Persamaan : Penelitian ini menjadi landasan teknologi ASR (Automatic Speech Recognition) yang digunakan dalam sistem notulensi rapat otomatis, khususnya komponen transkripsi audio-ke-teks menggunakan model Whisper. Evaluasi performa menggunakan metrik WER juga diadopsi dalam penelitian ini. Perbedaan: Penelitian Radford et al. berfokus pada pengembangan dan validasi model ASR Whisper secara umum tanpa konteks spesifik notulensi rapat, sedangkan penelitian ini mengimplementasikan Whisper sebagai komponen dalam pipeline sistem notulensi lengkap yang terintegrasi dengan LexRank dan Llama 3.2 untuk menghasilkan ringkasan rapat berbahasa Indonesia
3	Penulis/Tahun : Halimah, Surya Agustian, Siti Ramadhani / 2022 Judul Artikel : Peringkasan Teks Otomatis (Automated Text Summarization) pada Artikel Berbahasa Indonesia Menggunakan Algoritma LexRank Metode yang digunakan: LexRank — algoritma peringkasan teks ekstraktif berbasis graf (graph-based extractive summarization) dengan representasi kemiripan antar kalimat menggunakan cosine similarity berbasis TF-IDF. Hasil yang diperoleh: Algoritma LexRank berhasil menghasilkan ringkasan ekstraktif yang koheren pada 300 artikel berbahasa Indonesia dari berbagai topik. Evaluasi menggunakan metrik ROUGE menunjukkan bahwa LexRank mampu mengidentifikasi kalimat-kalimat paling

	representatif dan sentral dalam dokumen teks berbahasa Indonesia secara efektif. Persamaan : Sama-sama menggunakan algoritma LexRank sebagai metode utama extractive summarization pada teks berbahasa Indonesia dan mengevaluasi kualitas ringkasan menggunakan metrik ROUGE. Penelitian ini menjadi acuan langsung implementasi LexRank dalam penelitian yang dilakukan. Perbedaan : Penelitian tersebut menerapkan LexRank secara murni extractive pada artikel teks statis berbahasa Indonesia, sedangkan penelitian ini menggunakan LexRank sebagai tahap pertama dalam pipeline hybrid yang kemudian dilanjutkan dengan abstractive rewriting menggunakan Llama 3.2. Input pada penelitian ini juga berupa teks hasil transkripsi rapat (spoken text) yang jauh lebih beragam dan penuh noise dibanding artikel terstruktur.
4	Penulis/Tahun : Melvin Manuel, Aneetta Menon, Anakha Kallivayalil, Savio Isaac, Darel Lawrence KS / 2022 Judul Artikel : Automated Generation of Meeting Minutes Using Deep Learning Techniques Metode yang digunakan : Sistem AMBOC (Automated Minute Book Creation) menggunakan Deep Neural Networks untuk transkripsi audio (speech-to-text), MFCC (Mel Frequency Cepstral Coefficient) untuk speaker recognition, dan model Transformer untuk summarisasi. Hasil yang diperoleh : Sistem AMBOC berhasil mengotomatisasi pembuatan transkrip dan notulensi rapat dengan kemampuan pengenalan pembicara. Audio rapat dikonversi ke teks menggunakan DNN, kemudian diringkas menggunakan model Transformer yang terlatih. Sistem ini terbukti mengurangi ketergantungan pada pencatatan manual yang membutuhkan sumber daya manusia besar. Persamaan : Sama-sama membangun sistem otomatis end-to-end dari audio rapat menjadi notulensi terstruktur menggunakan kombinasi ASR dan teknik summarisasi berbasis deep learning, dengan tujuan mengurangi beban manual notulis rapat. Perbedaan : Penelitian tersebut menggunakan DNN generik dan model Transformer berbasis cloud/API yang membutuhkan koneksi internet, sedangkan penelitian ini menggunakan OpenAI Whisper untuk ASR dan Llama 3.2 via Ollama yang berjalan sepenuhnya secara lokal (offline) tanpa ketergantungan API berbayar. Selain itu, penelitian ini menyediakan antarmuka web terintegrasi dengan mode live transcription real-time.

5	Penulis/Tahun : Marc Arnold, Matthias Goldschmitt, Thomas Rigotti / 2023 Judul Artikel : Dealing with Information Overload: A Comprehensive Review Metode yang digunakan : Systematic Literature Review (SLR) terhadap penelitian-penelitian tentang information overload dalam konteks digitalisasi dan penggunaan teknologi informasi. Hasil yang diperoleh : Kajian sistematis mengonfirmasi bahwa information overload merupakan permasalahan nyata yang semakin diperparah oleh digitalisasi dunia kerja. Kondisi ini secara konsisten dikaitkan dengan menurunnya kinerja, meningkatnya stres kerja, dan terganggunya proses pengambilan keputusan yang efektif dalam organisasi. Penelitian ini menyoroti pentingnya sistem yang mampu menyaring dan mendistilasi informasi esensial dari volume data yang besar. Persamaan : Penelitian ini menjadi landasan teoritis yang menjelaskan urgensi pengembangan sistem notulensi otomatis. Fenomena information overload dari transkrip rapat verbatim yang sangat panjang menjadi salah satu motivasi utama penelitian yang dilakukan. Perbedaan : Penelitian Arnold et al. bersifat review teoritis tentang fenomena information overload secara umum tanpa mengembangkan solusi teknologi, sedangkan penelitian yang dilakukan secara konkret mengimplementasikan solusi teknologi berupa sistem notulensi otomatis berbasis hybrid summarization untuk mengatasi information overload dari transkrip rapat.
6	Penulis/Tahun : Hugo Touvron, Louis Martin, Kevin Stone, dkk. / 2023 Judul Artikel : Llama 2: Open Foundation and Fine-Tuned Chat Models Metode yang digunakan : Large Language Model (LLM) berbasis Transformer Decoder dengan teknik pelatihan Reinforcement Learning from Human Feedback (RLHF) dan Supervised Fine-Tuning (SFT) untuk menghasilkan model percakapan berkualitas tinggi. Hasil yang diperoleh : Model Llama 2 dalam berbagai ukuran parameter (7B, 13B, 34B, 70B) menunjukkan performa kompetitif dibanding model GPT-3.5 pada berbagai benchmark NLP. Versi chat model terbukti aman dan mampu menghasilkan teks berkualitas tinggi sesuai instruksi yang diberikan. Model ini menjadi fondasi arsitektur yang dikembangkan

	menjadi Llama 3.x dengan peningkatan signifikan pada kemampuan multilingualisme dan pemahaman konteks. Persamaan : Model Llama (yang berkembang menjadi Llama 3.2) digunakan dalam penelitian ini sebagai komponen abstractive rewriting — mengubah kalimat-kalimat ekstrak LexRank menjadi narasi ringkasan yang kohesif. Kemampuan Llama dalam mengikuti instruksi (instruction-following) menjadi kunci kualitas output notulensi. Perbedaan : Penelitian Touvron et al. berfokus pada pengembangan dan evaluasi model bahasa besar secara umum, sedangkan penelitian ini mengimplementasikan model Llama 3.2 (generasi lanjutan Llama 2) secara lokal melalui Ollama dalam konteks spesifik abstractive rewriting pada pipeline notulensi rapat berbahasa Indonesia, tanpa melakukan fine-tuning ulang model.
7	Penulis/Tahun : Divya S., Sripriya N., J. Andrew, Manuel Mazzara / 2024 Judul Artikel : Unified Extractive-Abstractive Summarization: A Hybrid Approach Utilizing BERT and Transformer Models for Enhanced Document Summarization Metode yang digunakan : Pendekatan hybrid extractive-abstractive summarization menggunakan representasi kalimat berbasis BERT untuk tahap ekstraksi, dilanjutkan dengan Transformer untuk abstractive rewriting. Hasil yang diperoleh : Metode hybrid yang mengombinasikan tahap extractive dan abstractive secara signifikan mengungguli pendekatan extractive-only maupun abstractive-only dalam hal kualitas ringkasan (diukur menggunakan ROUGE) pada dataset CNN/DailyMail dan DUC2004. Penelitian ini membuktikan bahwa integrasi kedua pendekatan menghasilkan ringkasan yang lebih relevan dan lebih mudah dibaca. Persamaan : Keduanya menggunakan paradigma hybrid extractive-abstractive summarization, di mana hasil ekstraksi digunakan sebagai masukan untuk proses abstractive rewriting, dengan tujuan menghasilkan ringkasan yang lebih kohesif dan readable dibanding pendekatan single-stage. Perbedaan : Penelitian tersebut menggunakan BERT untuk extractive stage dan Transformer encoder-decoder untuk abstractive stage pada dokumen teks terstruktur berbahasa Inggris, sedangkan penelitian ini menggunakan LexRank (graph-based) untuk extractive stage dan Llama 3.2 (generative LLM lokal) untuk abstractive stage pada spoken text rapat berbahasa Indonesia yang bersifat noisy dan tidak terstruktur.

8	Penulis/Tahun : Roshan Sharma, et al. / 2023 Judul Artikel : Multi-Person Speech Curator for Minutes of Meetings Along with Meeting Summarization and Language Translation Metode yang digunakan : Kombinasi OpenAI Whisper API untuk speech-to-text, algoritma speaker diarization untuk identifikasi pembicara, dan model BART (facebook/bart-large-cnn) fine-tuned pada dataset SAMSum untuk summarisasi teks rapat. Hasil yang diperoleh : Sistem berhasil menghasilkan notulensi rapat multi-pembicara secara otomatis dengan komponen transkripsi Whisper dan summarisasi BART. Evaluasi menunjukkan bahwa sistem mampu menghemat waktu dan sumber daya secara signifikan dibanding pencatatan manual, serta menyediakan fitur terjemahan ringkasan menggunakan model mBART. Persamaan : Keduanya menggunakan OpenAI Whisper sebagai model ASR untuk transkripsi audio rapat, mengimplementasikan pipeline otomatis dari audio ke ringkasan rapat, dan bertujuan untuk menggantikan proses pencatatan notulensi manual. Perbedaan : Penelitian tersebut menggunakan Whisper melalui API cloud dan model BART berbasis cloud untuk summarisasi, sedangkan penelitian ini menjalankan semua komponen secara lokal (Whisper small model + Llama 3.2 via Ollama) tanpa ketergantungan API berbayar. Penelitian ini juga menambahkan tahap extractive (LexRank) sebelum abstractive rewriting dan menyediakan mode live transcription real-time.
9	Penulis/Tahun : Ricardo Campos, et al. / 2026 Judul Artikel : NLP for Local Governance Meeting Records: A Focus Article on Tasks, Datasets, Metrics and Benchmark Metode yang digunakan : Kajian komprehensif tentang tugas-tugas NLP (Named Entity Recognition, Summarization, Sentiment Analysis, Question Answering) yang relevan untuk dokumen notulensi rapat pemerintahan lokal. Hasil yang diperoleh : Penelitian ini mengidentifikasi bahwa rekam rapat pemerintahan lokal bersifat padat, birokratis, dan sangat heterogen antar institusi, menyulitkan interpretasi komputasional. Penelitian juga menyajikan benchmark untuk berbagai tugas NLP pada domain notulensi rapat pemerintahan, mengisi kesenjangan riset yang sebelumnya kurang terwakili dalam literatur NLP.

	Persamaan : Keduanya berkaitan dengan otomasi pemrosesan dokumen notulensi rapat pemerintahan menggunakan teknik NLP, dengan penekanan pada tantangan kualitas dokumentasi dan akuntabilitas publik dari rapat organisasi. Perbedaan : Penelitian Campos et al. bersifat survey dan benchmark tanpa mengembangkan sistem produksi, sedangkan penelitian ini secara konkret membangun sistem notulensi otomatis berbasis web yang siap digunakan. Selain itu, penelitian ini berfokus pada pipeline transkripsi-ringkasan untuk rapat berbahasa Indonesia, sementara penelitian Campos et al. mencakup multi-bahasa dan multi-tugas NLP secara luas.
10	Penulis/Tahun : Arianto / 2022 Judul Artikel : Drafting of Online Meeting Minutes Based on Video Recording Using Topic Modelling Metode yang digunakan : Topic Modelling (Latent Dirichlet Allocation / LDA) untuk mengekstraksi topik-topik utama dari transkrip rekaman video rapat, kemudian digunakan sebagai dasar penyusunan draf notulensi. Hasil yang diperoleh : Sistem berhasil mengidentifikasi topik-topik pembahasan utama dalam rekaman video rapat secara otomatis menggunakan topic modelling, sehingga dapat membantu penyusunan draf notulensi berbasis topik. Pendekatan ini menunjukkan bahwa rekaman video rapat dapat diolah secara otomatis untuk menghasilkan dokumen yang lebih terstruktur. Persamaan : Keduanya membangun sistem otomatis untuk menghasilkan draf notulensi dari rekaman rapat (audio/video) dan memanfaatkan teknik NLP untuk mengidentifikasi informasi penting yang perlu didokumentasikan dalam notulensi. Perbedaan : Penelitian tersebut menggunakan topic modelling (LDA) yang bersifat unsupervised dan menghasilkan output berbasis topik, sedangkan penelitian ini menggunakan pipeline hybrid extractive-abstractive (LexRank + Llama 3.2) yang menghasilkan narasi ringkasan notulensi yang lebih kohesif dan siap baca. Penelitian ini juga mendukung mode live transcription real-time yang tidak tersedia pada penelitian sebelumnya.

2.2 Tinjauan Teoritis

Tinjauan teoritis merupakan landasan konseptual yang menjelaskan teori-teori, metode, dan teknologi yang menjadi dasar dalam pembangunan sistem notulensi rapat otomatis ini. Bagian ini mencakup uraian tentang notulensi rapat, *Automatic Speech Recognition* (ASR), model OpenAI Whisper, algoritma LexRank, *Large Language Model* (LLM) berbasis Llama 3.2, pendekatan hybrid summarization, serta metrik evaluasi yang digunakan.

2.1.1 Notulensi Rapat

Notulensi rapat adalah dokumen resmi tertulis yang merekam jalannya diskusi, keputusan, dan tindakan yang disepakati dalam suatu rapat. Secara etimologis, istilah ini berasal dari kata Latin *nota*, yang berarti catatan. Dalam praktik organisasi modern, notulensi berfungsi sebagai audit trail administratif—rekam jejak legal yang dapat dijadikan acuan akuntabilitas, verifikasi fakta, dan dasar pengambilan keputusan di kemudian hari. Dokumen ini setidaknya memuat: identitas rapat (tanggal, waktu, tempat, moderator), daftar hadir peserta, agenda pembahasan, ringkasan diskusi setiap agenda, keputusan yang ditetapkan, serta rencana tindak lanjut (action items) beserta penanggung jawabnya.

Proses pembuatan notulensi konvensional sangat bergantung pada kemampuan individual notulis yang harus mendengarkan, memahami konteks, menyaring informasi penting, sekaligus mencatat secara bersamaan—sebuah kondisi multitasking yang menimbulkan beban kognitif tinggi. Kajian mutakhir Campos et al. (2026) menegaskan bahwa rekam rapat pemerintahan lokal bersifat padat, birokratis, dan sangat heterogen antar institusi sehingga menyulitkan interpretasi manusia maupun analisis komputasional secara sistematis[2]. Kondisi ini memperkuat urgensi otomasi proses notulensi menggunakan teknologi kecerdasan buatan.

2.1.2 Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) adalah teknologi yang memungkinkan komputer mengonversi sinyal ucapan manusia menjadi teks tertulis secara otomatis. Proses kerja ASR modern umumnya melibatkan beberapa tahapan: (1) preprocessing sinyal audio untuk mengurangi derau, (2) ekstraksi fitur akustik—paling umum menggunakan Mel-Frequency Cepstral Coefficient (MFCC) atau representasi spektral berbasis mel, (3) decoding oleh model akustik dan bahasa untuk menerjemahkan fitur akustik menjadi kata-kata, dan (4) pasca-pemrosesan teks untuk koreksi ejaan dan tanda baca.

Generasi terbaru ASR berbasis arsitektur Transformer telah melampaui akurasi sistem berbasis Hidden Markov Model (HMM) konvensional secara signifikan. Model ASR modern dilatih secara end-to-end langsung memetakan spektrogram audio ke token teks, tanpa pipeline akustik-bahasa terpisah, sehingga lebih robust terhadap variasi aksen, kecepatan bicara, dan kondisi lingkungan yang beragam.

Akurasi sistem ASR diukur menggunakan metrik Word Error Rate (WER), yang diformulasikan sebagai berikut:

$$WER = \frac{S + D + I}{N} \times 100\%$$

Di mana S adalah jumlah substitusi (kata yang salah), D adalah jumlah delesi (kata yang hilang), I adalah jumlah insersi (kata tambahan yang tidak ada di referensi), dan N adalah total kata dalam teks referensi. Sehingga akurasi transkripsi dapat dihitung sebagai:

$$Akurasi\ ASR = (1 - WER) \times 100\%$$

Nilai WER yang lebih rendah menunjukkan akurasi transkripsi yang lebih baik. Sebagai acuan umum di industri, sistem ASR dianggap production-ready apabila mencapai WER di bawah 10% pada domain dan aksen yang sesuai[9]

2.1.3 Disfluensi dan Filler Words dalam Ucapan Spontan

Ucapan spontan dalam konteks rapat memiliki karakteristik yang berbeda signifikan dengan ucapan yang direncanakan dan dilatih, terutama dalam hal tingginya frekuensi disfluensi—fenomena linguistik yang mencakup pengulangan kata, jeda terisi (filled pauses), hesitasi, dan koreksi diri[10]. Kehadiran disfluencies dalam sinyal audio dapat secara serius menghambat akurasi pengenalan kata sistem ASR karena meningkatkan tingkat kesalahan insertion, deletion, dan rejection pada output transkripsi[10]. Dalam praktiknya, filled pauses seperti "uh" dan "uhm" merupakan jenis disfluency yang paling umum ditemui dalam ucapan spontan berbahasa Inggris, sementara dalam Bahasa Indonesia fenomena serupa muncul dalam bentuk "ehm", "ya", "jadi", "kan", "gitu", "sebenarnya", "mungkin", dan "nggak tahu"[3].

Penelitian mutakhir telah mengembangkan berbagai pendekatan untuk mendeteksi dan mengeliminasi filler words dari transkripsi otomatis. Sebuah benchmark khusus untuk deteksi dan klasifikasi filler words yang dikembangkan pada tahun 2022 membuktikan bahwa pengenalan kata-kata pengisi memerlukan integrasi pendekatan akustik dan linguistik karena karakteristik fonetik unik yang dimiliki oleh elemen-elemen tersebut[11]. Lebih lanjut, pendekatan multi-task self-supervised learning yang diperkenalkan untuk deteksi disfluency telah memanfaatkan representasi terlatih dari data transkripsi berlabel dan membuktikan bahwa model yang dilatih secara end-to-end mampu mengenali pola disfluency dengan akurasi yang kompetitif dibandingkan sistem pipeline konvensional[12].

Dalam konteks teks transkripsi verbatim, keberadaan filler words dan disfluencies meningkatkan volume dokumen secara signifikan tanpa menambah informasi semantik yang substantif, sehingga memberatkan proses peringkasan dan analisis selanjutnya[4]. Transkrip yang mempertahankan setiap hesitasi dan pengulangan meskipun secara teknis lengkap, seringkali membutuhkan waktu lebih lama untuk ditinjau dan dibersihkan secara manual. Oleh karena itu, penghapusan disfluencies menjadi sangat penting ketika teks bersih diperlukan untuk penyuntingan konten

atau ketika transkrip digunakan sebagai dasar untuk subtitling maupun localisation [13].

Sebuah penelitian penting tentang pengenalan ucapan dan penghapusan disfluency secara terintegrasi menegaskan bahwa sistem ASR yang dilatih untuk langsung memetakan ucapan disfluent ke transkrip fluent menawarkan efisiensi komputasi yang signifikan meskipun performanya sedikit lebih rendah dibandingkan pendekatan pipeline yang terpisah. Penelitian tersebut juga mengusulkan metrik evaluasi baru—Fluent Error Rate (FER) dan Disfluent Error Rate (DER)—yang dirancang khusus untuk mengukur kualitas output sistem terintegrasi secara lebih akurat daripada WER konvensional[14].

2.1.4 Word Filler dan Noise Filter

Noise filter merupakan komponen pasca-pemrosesan transkripsi yang dirancang untuk mengidentifikasi dan menghapus elemen-elemen non-informatif dari output ASR sebelum teks tersebut diproses lebih lanjut dalam pipeline peringkasan. Pendekatan noise filtering yang diterapkan dalam sistem ini mengkombinasikan dua strategi utama yang terinspirasi dari literatur disfluency removal kontemporer: pertama, lexicon-based filtering yang menggunakan daftar stopwords dan filler words yang dikurasi khusus untuk domain rapat formal Bahasa Indonesia; dan kedua, pattern-based filtering untuk mendeteksi pengulangan karakter, token akustik yang tidak bermakna seperti "mmm", "ahh", dan "ehh", serta fragmentasi kalimat akibat tumpang tindih bicara (cross-talk)[10][12].

Proses noise filtering diterapkan pada dua tahap dalam pipeline sistem. Tahap pertama dilakukan segera setelah output transkripsi Whisper diperoleh, dengan tujuan membersihkan teks dari filler words dan token akustik sebelum masuk ke tahap deduplikasi Jaccard dan ekstraksi LexRank. Tahap kedua dilakukan pada level kalimat terpilih oleh LexRank, untuk memastikan kalimat-kalimat yang masuk ke tahap abstraktif Llama 3.2 memiliki densitas informasi optimal.

Eliminasi word fillers secara signifikan mengurangi information overload pada input model LLM, sekaligus meningkatkan kualitas ringkasan akhir karena model generatif tidak lagi terdistraksi oleh pola-pola repetitif dari bahasa lisan yang tidak relevan dengan substansi rapat [4]. Hal ini didukung oleh penelitian tentang Disfluency Augmented Curriculum Learning (DACL) yang membuktikan bahwa pembelajaran dengan kurikulum yang diperkaya disfluency secara signifikan meningkatkan kemampuan model dalam menghasilkan teks yang bersih dari elemen-elemen non-informatif[15].

Strategi fine-tuning model Whisper dengan label-masking pada data latih yang mengandung disfluency telah menunjukkan hasil yang sangat mengesankan dalam mengurangi kesalahan transkripsi. Dalam sebuah kompetisi pengenalan ucapan internasional, pendekatan ini berhasil menurunkan deletion error dari 4.62% menjadi 1.15% karena model yang di-fine-tune belajar mengenali fenomena disfluency seperti pengulangan kata dan menghasilkan transkrip yang lebih akurat. Strategi encoder-freezing yang diterapkan secara bersamaan juga membuktikan bahwa mempertahankan representasi akustik multilingual sambil melatih ulang decoder untuk mengabaikan disfluency merupakan pendekatan yang sangat efektif [16].

Dalam konteks sistem yang dikembangkan, mitigasi disfluency diimplementasikan melalui tiga mekanisme utama. Pertama, lexical filtering menggunakan daftar kata pengisi Bahasa Indonesia yang dikurasi manual berdasarkan frekuensi kemunculan dalam corpus rapat. Kedua, acoustic pattern detection untuk token-token non-linguistik seperti "mmm" dan "ehh" yang sering muncul akibat hesitasi pembicara. Ketiga, syntactic validation untuk mendeteksi fragmentasi kalimat akibat tumpang tindih bicara antar peserta rapat. Mekanisme-mekanisme ini bekerja sinergis untuk memastikan integritas data transkripsi sebelum masuk ke pipeline peringkasan

2.1.5 Model OpenAI Whisper

OpenAI Whisper adalah model ASR berbasis Transformer encoder-decoder yang dikembangkan oleh OpenAI menggunakan metode weak supervision berskala

besar. Radford et al. (2022) melatih Whisper pada 680.000 jam data audio berlabel dari internet yang mencakup lebih dari 97 bahasa, termasuk Bahasa Indonesia [9]. Pendekatan weak supervision ini memungkinkan model belajar dari data yang jauh lebih besar dan beragam dibanding dataset ASR konvensional yang dikurasi secara manual, menghasilkan robustness yang luar biasa terhadap variasi aksen, kecepatan bicara, dan kondisi akustik yang bervariasi.

Arsitektur Whisper menggunakan komponen-komponen berikut: (1) Log-Mel spectrogram dengan 80 saluran sebagai representasi input audio, (2) encoder Transformer yang memproses representasi akustik menjadi embedding laten, (3) decoder Transformer yang menghasilkan token teks secara autoregresif melalui mekanisme cross-attention terhadap output encoder. Whisper tersedia dalam enam varian ukuran model—tiny, base, small, medium, large, large-v3—dengan pertukaran antara akurasi dan kecepatan inferensi yang dapat disesuaikan dengan sumber daya komputasi yang tersedia.

1. Whisper Hallucination

Meskipun Whisper menunjukkan performa transkripsi yang superior, model ini tidak kebal terhadap fenomena *hallucination*—yaitu produksi teks yang tidak terdengar dalam sinyal audio input. Barański *et al.* (2025) mendefinisikan *hallucinations* dalam konteks ASR sebagai prediksi tanpa koneksi fonetik atau semantik terhadap referensi, yang terjadi ketika sistem ASR menghasilkan transkrip untuk audio yang tidak mengandung konten ucapan apapun [17]. Fenomena ini berbeda dengan *error* konvensional (substitusi atau delesi) karena teks yang dihasilkan bersifat koheren secara linguistik namun secara akustik tidak terjustifikasi [16]. Dalam konteks transkripsi rapat, hallucination Whisper dapat berupa: penambahan kata penghubung yang tidak diucapkan, pengulangan frasa dari pembicara sebelumnya yang tidak relevan dengan pembicara aktual, atau bahkan penambahan nama, angka, atau terminologi teknis yang tidak disebutkan dalam audio. Konsekuensinya sangat kritis untuk

sistem notulensi karena hallucinated content dapat disalahartikan sebagai keputusan rapat atau action item yang sah.

Sistem yang dikembangkan mengimplementasikan tiga mekanisme mitigasi hallucination: (1) segment-level verification menggunakan confidence score per token untuk mendeteksi segmen dengan probabilitas rendah; (2) cross-speaker consistency check yang membandingkan transkripsi antar segmen untuk mendeteksi pengulangan anomali; dan (3) post-hoc filtering menggunakan aturan heuristik untuk menghapus kalimat yang memiliki pola repetitif tidak wajar atau yang mengandung informasi yang kontradiktif dengan konteks rapat. Mitigasi ini bekerja sinergis dengan komponen noise filter (Subbab 2.2.3) untuk memastikan integritas data transkripsi sebelum masuk ke pipeline peringkasan.

Dalam konteks penelitian ini, model Whisper digunakan sebagai komponen ASR utama untuk mengonversi audio rapat berbahasa Indonesia menjadi teks transkrip. Keunggulan Whisper dalam mengenali Bahasa Indonesia—termasuk istilah teknis dan frasa berbahasa Inggris yang umum digunakan dalam konteks rapat organisasi dan akademis—menjadikannya pilihan yang paling sesuai untuk keperluan sistem notulensi otomatis, dengan catatan mitigasi hallucination harus diimplementasikan secara rigor.

2.1.6 Information Overload

Information overload (beban informasi berlebih) adalah kondisi di mana volume informasi yang diterima seseorang melampaui kapasitas kognitifnya untuk memproses secara efektif, sehingga justru menghambat—bukan membantu—proses pengambilan keputusan. Konsep ini pertama kali diperkenalkan dalam literatur manajemen dan psikologi pada era 1970-an, namun relevansinya semakin meningkat seiring dengan ekspansi teknologi informasi dan komunikasi di lingkungan kerja modern. Arnold, Goldschmitt, dan Rigotti (2023) dalam kajian sistematis mereka menegaskan bahwa information overload merupakan permasalahan nyata yang semakin diperparah oleh digitalisasi dunia kerja dan

secara konsisten dikaitkan dengan menurunnya kinerja, meningkatnya stres kerja, serta terganggunya proses pengambilan keputusan yang efektif dalam organisasi [4].

Dalam konteks dokumentasi rapat, information overload muncul ketika hasil transkripsi verbatim dari rekaman rapat—yang dapat mencapai puluhan halaman untuk satu sesi rapat—disajikan langsung kepada pimpinan atau peserta rapat tanpa proses distilasi. Dokumen transkripsi verbatim penuh dengan filler words, pengulangan kalimat, dan percakapan basa-basi yang tidak relevan dengan substansi keputusan—menjadikan proses peninjauan ulang isi rapat sangat tidak efisien. Fenomena inilah yang menjadi motivasi utama dikembangkan komponen automatic text summarization dalam sistem yang dibangun pada penelitian ini.

2.1.7 Pemrosesan Bahasa Alami (Natural Language Processing)

Natural Language Processing (NLP) adalah cabang kecerdasan buatan yang mengkaji interaksi antara komputer dan bahasa manusia. NLP mencakup spektrum tugas yang luas, mulai dari analisis tingkat rendah seperti tokenisasi dan penandaan kelas kata (part-of-speech tagging), hingga tugas tingkat tinggi seperti penerjemahan mesin, menjawab pertanyaan (question answering), analisis sentimen, dan peringkasan teks. Kemajuan besar dalam NLP modern didorong oleh mekanisme self-attention yang diperkenalkan oleh Vaswani et al. (2017) dalam arsitektur Transformer, yang kini menjadi fondasi utama hampir seluruh model bahasa mutakhir (state-of-the-art)[13].

Dalam pipeline sistem notulensi otomatis, NLP berperan pada dua tahap utama: (1) analisis dan pemilihan kalimat penting dari teks transkripsi melalui extractive summarization, dan (2) penulisan ulang kalimat-kalimat terpilih menjadi narasi

yang koheren melalui abstractive summarization. Kedua tahap ini memerlukan pemahaman semantik mendalam terhadap teks berbahasa Indonesia, termasuk kemampuan menangani ciri khas teks spoken language seperti kalimat tidak lengkap, kata pengisi, dan pengulangan.

2.1.8 Peringkasan Teks Otomatis (Automatic Text Summarization)

Automatic Text Summarization (ATS) adalah proses komputasional untuk menghasilkan versi ringkas dari satu atau beberapa dokumen teks yang mempertahankan informasi paling penting. Berdasarkan pendekatan yang digunakan, ATS dibagi menjadi dua paradigma utama:

a. Extractive Summarization

Pendekatan ekstraktif memilih kalimat-kalimat yang dianggap paling representatif langsung dari dokumen asli tanpa mengubah susunan kata apapun. Algoritma menilai kepentingan setiap kalimat berdasarkan berbagai fitur: frekuensi kata kunci, posisi kalimat dalam dokumen, kemiripan dengan kalimat-kalimat lain, dan skor sentralitas dalam representasi graf dokumen. Output ringkasan merupakan himpunan subset kalimat asli yang disusun ulang. Keunggulan pendekatan ini adalah ketepatan fakta (tidak ada informasi baru yang ditambahkan) dan efisiensi komputasi, namun kelemahannya adalah hasil ringkasan dapat terasa kaku dan kehilangan koherensi transisional[18].

b. Abstractive Summarization

Pendekatan abstraktif menghasilkan teks ringkasan baru yang dapat mengandung kata dan frasa yang tidak ada dalam dokumen asli, serupa dengan cara manusia meringkas dokumen. Model generatif—umumnya berbasis arsitektur Transformer seq2seq atau LLM—membaca seluruh dokumen, membangun representasi semantik, kemudian menghasilkan teks ringkasan secara autoregresif. Pendekatan ini menghasilkan narasi yang lebih alami dan kohesif, namun berisiko

menghasilkan informasi yang tidak akurat (hallucination) apabila model tidak dikontrol dengan baik[19].

2.1.9 Automatic Correction of Indonesian Grammatical

Automatic Text Correction merupakan salah satu penerapan *Natural Language Processing* (NLP) yang bertujuan untuk mendeteksi dan memperbaiki kesalahan pada teks secara otomatis, baik berupa kesalahan ejaan, tata bahasa, penggunaan tanda baca, maupun pemilihan kata. tugas *Grammatical Error Correction* (GEC) pada bahasa Indonesia dapat dimodelkan sebagai permasalahan *sequence-to-sequence* menggunakan arsitektur Transformer yang mampu memahami konteks kalimat secara menyeluruh melalui mekanisme *self-attention*. Penelitian tersebut menunjukkan bahwa pendekatan Transformer yang dipadukan dengan *copy mechanism* efektif dalam memperbaiki berbagai jenis kesalahan pada teks bahasa Indonesia, sehingga dapat meningkatkan kualitas, keterbacaan, dan kejelasan informasi yang disampaikan. Oleh karena itu, konsep *automatic text correction* dapat dimanfaatkan sebagai proses pascapemrosesan (*post-processing*) untuk menyempurnakan hasil transkripsi otomatis agar lebih mudah dipahami dan sesuai dengan kaidah penulisan bahasa Indonesia[20].

2.1.10 Algoritma LexRank

LexRank adalah algoritma extractive summarization berbasis graf yang diperkenalkan oleh Erkan dan Radev (2004). Algoritma ini merepresentasikan seluruh kalimat dalam dokumen sebagai simpul (node) dalam sebuah graf berbobot, di mana bobot antar simpul ditentukan oleh nilai kemiripan kosinus (cosine similarity) berbasis representasi TF-IDF dari setiap pasangan kalimat[21].

Kemiripan kosinus antara dua kalimat i dan j dihitung menggunakan rumus:

$$\cos(\vec{d}_i, \vec{d}_j) = \frac{\vec{d}_i \cdot \vec{d}_j}{|\vec{d}_i| \times |\vec{d}_j|}$$

Dimana d_i dan d_j adalah vektor TF-IDF dari kalimat i dan j . Setelah graf terbentuk, LexRank menerapkan algoritma power iteration serupa dengan PageRank untuk menghitung skor sentralitas leksikal setiap kalimat:

$$p(u) = \frac{d}{N} + (1 - d) \times \sum_{v \in \text{adj}(u)} \frac{\text{sim}(u, v)}{\text{deg}(v)} \times p(v)$$

Di mana d adalah faktor redaman (damping factor, umumnya 0.85), N adalah total kalimat, $\text{sim}(u, v)$ adalah kemiripan kosinus antara kalimat u dan v , dan $\text{deg}(v)$ adalah jumlah total bobot sisi yang terhubung ke simpul v (out-degree). Kalimat-kalimat dengan skor $p(u)$ tertinggi dipilih sebagai output ringkasan ekstraktif.

Pada konteks teks berbahasa Indonesia, Halimah, Agustian, dan Ramadhani (2022) membuktikan bahwa LexRank mampu menghasilkan ringkasan ekstraktif yang koheren dan relevan pada 300 artikel berbahasa Indonesia dari berbagai topik, dengan evaluasi ROUGE yang kompetitif[22]. Hal ini menjadikan LexRank sebagai pilihan yang tepat untuk tahap ekstraksi kalimat dalam pipeline sistem notulensi berbahasa Indonesia.

Akurasi LexRank diukur menggunakan metrik ROUGE-1 dan ROUGE-L. Rumus ROUGE-1 (berbasis unigram):

$$\text{Precision} = \frac{|\text{kata_cocok}|}{|\text{total_kata_sistem}|}$$

$$\text{Recall} = \frac{|\text{kata_cocok}|}{|\text{total_kata_referensi}|}$$

$$\text{F1-Score} = \frac{2 \times (P \times R)}{P + R}$$

Sedangkan ROUGE-L berbasis Longest Common Subsequence (LCS):

$$\text{LCS_Precision} = \frac{\text{LCS}(X, Y)}{|Y|}$$

$$\text{LCS_Recall} = \frac{\text{LCS}(X, Y)}{|X|}$$

$$\text{ROUGE-L F1} = \frac{(1 + \beta^2) \times \text{LCS_R} \times \text{LCS_P}}{\text{LCS_R} + \beta^2 \times \text{LCS_P}}$$

Di mana X adalah ringkasan referensi dan Y adalah ringkasan sistem. Parameter β mengontrol keseimbangan antara precision dan recall; pada kasus standar $\beta = 1$, rumus ini ekuivalen dengan F1 harmonik biasa[23].

2.1.11 Large Language Model (LLM)

Large Language Model (LLM) adalah model bahasa berbasis arsitektur Transformer decoder yang dilatih pada korpus teks berskala sangat besar dengan parameter mencapai miliaran hingga ratusan miliar. Model ini belajar distribusi statistik bahasa melalui tugas next-token prediction secara autoregresif: diberikan sekuens token sebelumnya, model memprediksi token berikutnya yang paling mungkin muncul. Setelah pra-pelatihan (pretraining) pada korpus generik, LLM umumnya menjalani tahap fine-tuning dengan teknik Reinforcement Learning from Human Feedback (RLHF) dan Supervised Fine-Tuning (SFT) untuk meningkatkan kemampuan mengikuti instruksi dan menghasilkan respons yang aman[8].

LLM modern seperti keluarga Llama dari Meta AI menunjukkan kemampuan emergent yang luar biasa: mampu menyelesaikan tugas-tugas yang tidak secara eksplisit dilatih, termasuk peringkasan teks, terjemahan, analisis sentimen, dan penulisan ulang (rewriting). Kemampuan instruksi-following yang kuat ini menjadikan LLM kandidat ideal untuk tahap abstractive rewriting dalam pipeline hybrid summarization.

2.1.12 Model Llama 3.2 dan Platform Ollama

Llama (Large Language Model Meta AI) adalah keluarga LLM sumber terbuka yang dikembangkan oleh Meta AI. Llama 2 yang diperkenalkan oleh Touvron et al. (2023) tersedia dalam ukuran 7B, 13B, 34B, dan 70B parameter dan menunjukkan performa kompetitif dibanding model GPT-3.5 pada berbagai benchmark NLP[8]. Llama 3.2 sebagai generasi lanjutannya menghadirkan peningkatan signifikan dalam kemampuan multilingualisme, pemahaman konteks yang lebih panjang, dan kualitas output teks yang lebih natural, sehingga lebih sesuai untuk tugas peringkasan teks berbahasa Indonesia.

Ollama adalah platform inferensi LLM sumber terbuka yang memungkinkan model seperti Llama 3.2 dijalankan secara lokal pada perangkat keras konsumen (CPU atau GPU) tanpa memerlukan koneksi internet atau langganan API berbayar. Ollama menyediakan antarmuka REST API yang kompatibel yang dapat dipanggil oleh aplikasi Python, Node.js, maupun aplikasi web lainnya secara langsung. Dalam konteks penelitian ini, Ollama digunakan untuk menjalankan inferensi Llama 3.2 secara lokal melalui backend Python Flask, memastikan privasi data rapat—karena seluruh pemrosesan dilakukan di infrastruktur lokal tanpa data dikirimkan ke server pihak ketiga.

2.1.13 Pendekatan Hybrid Extractive-Abstractive Summarization

Pendekatan hybrid extractive-abstractive summarization menggabungkan keunggulan kedua paradigma: ketepatan fakta dari pendekatan ekstraktif dan kelancaran narasi dari pendekatan abstraktif. Dalam pipeline hybrid, tahap ekstraktif terlebih dahulu menyaring kalimat-kalimat paling relevan dan informatif dari dokumen sumber, sehingga volume teks yang perlu diproses oleh model generatif berkurang secara drastis. Selanjutnya, model abstraktif menerima kalimat-kalimat terpilih sebagai konteks dan menuliskannya kembali menjadi narasi yang kohesif dan mudah dipahami.

Divya et al. (2024) membuktikan bahwa metode hybrid yang mengombinasikan BERT untuk tahap ekstraksi dan Transformer untuk tahap abstraksi secara signifikan mengungguli pendekatan single-stage (extractive-only maupun abstractive-only) dalam hal kualitas ringkasan yang diukur menggunakan ROUGE pada dataset CNN/DailyMail dan DUC2004[24]. Penelitian ini menjadi landasan konseptual utama bagi arsitektur pipeline LexRank + Llama 3.2 yang diterapkan dalam sistem notulensi yang dikembangkan, di mana LexRank berperan sebagai tahap ekstraktif dan Llama 3.2 sebagai tahap abstraktif.

2.1.14 TF-IDF dengan Sublinear TF dan Smooth IDF

Term Frequency-Inverse Document Frequency (TF-IDF) adalah metode pembobotan kata yang digunakan secara luas dalam information retrieval dan NLP untuk mengukur seberapa penting suatu kata dalam sebuah dokumen relatif terhadap sekumpulan dokumen lainnya (corpus). TF-IDF menjadi representasi vektor kalimat dalam algoritma LexRank.

Namun, dalam sistem yang dikembangkan pada penelitian ini, digunakan varian TF-IDF yang diimplementasikan melalui library scikit-learn dengan konfigurasi khusus untuk mengoptimalkan representasi teks transkripsi rapat. Implementasi ini menggunakan tiga komponen utama:

a. Sublinear TF (Term Frequency)

Berbeda dengan rumus TF standar yang linear, sistem menggunakan sublinear term frequency untuk mencegah dominasi kata yang muncul berulang kali dalam satu kalimat:

$$\text{TF}(t, d) = \begin{cases} 1 + \log(f(t, d)) & \text{jika } f(t, d) > 0 \\ 0 & \text{jika } f(t, d) = 0 \end{cases}$$

Di mana $f(t, d)$ adalah frekuensi kemunculan kata t dalam dokumen d . Fungsi logaritmik ini mengurangi bobot tambahan untuk kata yang muncul sangat sering, sehingga frekuensi tinggi tidak mendominasi representasi vektor secara

tidak proporsional. Hal ini penting dalam konteks transkripsi rapat di mana pembicara sering mengulangi kata kunci untuk penekasan, namun pengulangan tersebut tidak boleh memberikan bobot berlebihan dalam perhitungan kemiripan antar kalimat.

b. Smooth IDF (Inverse Document Frequency)

Untuk menghindari pembagian dengan nol ketika suatu kata tidak muncul di seluruh corpus, digunakan smooth IDF dengan penambahan konstanta 1 pada pembilang dan penyebut:

$$IDF(t, D) = \log \left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|} \right) + 1$$

Di mana $|D|$ adalah total jumlah dokumen dalam corpus dan $|\{d \in D : t \in d\}|$ adalah jumlah dokumen yang mengandung kata t . Penambahan 1 pada kedua sisi rasio memastikan tidak terjadi pembagian dengan nol ketika kata tersebut muncul di semua dokumen, sekaligus memberikan nilai IDF minimum yang tetap bernilai positif.

c. Normalisasi L2

Setelah perhitungan TF-IDF, vektor hasil dinormalisasi menggunakan L2-norm (Euclidean normalization) sehingga panjang vektor menjadi satuan:

$$\vec{v}_{\text{normalized}} = \frac{\vec{v}}{\|\vec{v}\|_2} = \frac{\vec{v}}{\sqrt{\sum_{i=1}^n v_i^2}}$$

Normalisasi ini memastikan bahwa panjang dokumen tidak mempengaruhi perhitungan kemiripan kosinus, sehingga kalimat pendek dan panjang dapat dibandingkan secara adil dalam algoritma LexRank.

2.1.15 Jaccard Similarity untuk Deduplikasi Kalimat

Dalam konteks transkripsi rapat, fenomena pengulangan kalimat oleh pembicara sangat umum terjadi—baik karena penekasan, klarifikasi, maupun karakteristik spoken language yang repetitif. Keberadaan kalimat duplikat atau hampir identik dalam input LexRank dapat mengaburkan diversitas informasi dan menghasilkan ringkasan yang redundan. Untuk mengatasi masalah ini, sistem menerapkan tahap deduplikasi kalimat sebelum proses ekstraksi LexRank menggunakan ukuran kemiripan Jaccard.

Jaccard Similarity mengukur rasio irisan terhadap gabungan dari dua himpunan, yang dalam konteks NLP diimplementasikan sebagai himpunan kata unik (unique tokens) dari dua kalimat. Rumus Jaccard Similarity antara kalimat A dan B adalah:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Di mana A dan B adalah himpunan kata unik dari masing-masing kalimat, $|A \cap B|$ adalah jumlah kata yang muncul di kedua kalimat (irisan), dan $|A \cup B|$ adalah jumlah total kata unik dari gabungan kedua kalimat. Nilai $J(A, B)$ berada dalam rentang $[0, 1]$, di mana 0 menunjukkan tidak ada kesamaan kata dan 1 menunjukkan identitas sempurna.

Dalam pipeline sistem, pasangan kalimat yang memiliki nilai Jaccard Similarity melebihi threshold 0.65 diklasifikasikan sebagai duplikat. Dari setiap pasangan duplikat tersebut, satu kalimat dibuang secara selektif—umumnya kalimat yang lebih pendek atau yang muncul kemudian—sementara kalimat lainnya dipertahankan untuk diproses oleh LexRank. Pemilihan threshold 0.65 didasarkan pada keseimbangan antara efektivitas penghilangan redundansi dan pelestarian variasi leksikal yang masih mengandung informasi berbeda. Threshold yang terlalu rendah akan menghapus kalimat yang sebenarnya mengandung informasi komplementer, sementara threshold yang terlalu tinggi akan gagal mengeliminasi repetisi yang signifikan.

Tahap deduplikasi ini secara substansial meningkatkan kualitas input untuk algoritma LexRank karena: (1) graf kalimat menjadi lebih sparse dan representatif, (2) probabilitas terpilihnya kalimat duplikat dalam ringkasan berkurang, serta (3) komputasi power iteration menjadi lebih efisien dengan jumlah node yang lebih sedikit.

2.1.16 Metrik Evaluasi Akurasi Sistem

Evaluasi akurasi sistem notulensi otomatis menggunakan tiga metrik utama yang saling melengkapi:

a. Word Error Rate (WER)

WER merupakan metrik standar industri untuk mengukur akurasi sistem ASR dengan cara membandingkan teks hasil transkripsi dengan teks referensi (ground truth) yang diketik manual secara verbatim. WER dihitung menggunakan algoritma edit distance (Levenshtein distance) pada tingkat kata. Formula lengkap WER dan akurasi ASR telah dijelaskan pada Subbab 2.2.2.

Interpretasi nilai WER: WER < 5% = sangat baik, 5-10% = baik, 10-20% = dapat diterima, > 20% = perlu perbaikan [5].

b. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE, yang diperkenalkan oleh Lin (2004), adalah metrik evaluasi otomatis untuk ringkasan teks yang mengukur tumpang tindih kata antara ringkasan yang dihasilkan sistem dengan ringkasan referensi buatan manusia [22]. Dua varian yang paling umum digunakan:

ROUGE-1 mengukur tumpang tindih pada level unigram (kata tunggal):

$$[\text{ROUGE-1 Recall} = \frac{\sum_{\text{unigram} \in \text{Ref}} \text{Count}_{\text{match}}(\text{unigram})}{\sum_{\text{unigram} \in \text{Ref}} \text{Count}(\text{unigram})}]$$

$$[\text{ROUGE-1 Precision} = \frac{\sum_{\text{unigram} \in \text{Sis}} \text{Count}_{\text{match}}(\text{unigram})}{\sum_{\text{unigram} \in \text{Sis}} \text{Count}(\text{unigram})}]$$

$$[\text{ROUGE-1 F1} = \frac{2 \times (P \times R)}{P + R}]$$

ROUGE-L berbasis *Longest Common Subsequence* (LCS) yang lebih sensitif terhadap urutan kata:

$$[\text{ROUGE-L F1} = \frac{(1 + \beta^2) \times R_{\text{lcs}} \times P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 \times P_{\text{lcs}}}]$$

c. Human Evaluation (Skala Likert)

Evaluasi manusia dilakukan menggunakan skala Likert 1-5 pada empat dimensi kualitas: (1) Keterbacaan—seberapa alami dan mudah dipahami narasi notulensi, (2) Kelengkapan—seberapa lengkap poin-poin penting rapat tertangkap, (3) Struktur—seberapa rapi dan terorganisasi penyajian notulensi, dan (4) Akurasi Fakta—seberapa tepat informasi yang disajikan dibanding transkrip asli. Skor keseluruhan per dokumen dihitung sebagai rata-rata dari keempat dimensi:

$$[\text{Skor}_{\text{dokumen}} = \frac{\text{Keterbacaan} + \text{Kelengkapan} + \text{Struktur} + \text{Akurasi_Fakta}}{4}]$$

$$[\text{Persentase} = \frac{\text{Skor}_{\text{dokumen}}}{5} \times 100\%]$$

d. Akurasi Sistem Keseluruhan (Composite Score)

Untuk mengukur akurasi keseluruhan pipeline sistem, digunakan formula komposit berbobot yang menggabungkan akurasi transkripsi Whisper dan akurasi ringkasan Ollama:

$$[\text{Akurasi}_{\text{Sistem}} = w_1 \times \text{Akurasi}_{\text{Whisper}} + w_2 \times \text{Akurasi}_{\text{Ringkasan}}]$$

Di mana $w_1=0.40$ adalah bobot komponen transkripsi, $w_2=0.60$ adalah bobot komponen ringkasan

$$[\text{Akurasi}_{\text{Whisper}} = (1 - \text{WER}) \times 100\%]$$

$$[\text{Akurasi}_{\text{Ringkasan}} = \text{ROUGE-L F1} \times 100\%]$$

dari output final Ollama.

BAB III

METODOLOGI PENELITIAN

3.1 Data dan Pengumpulan Data

Sumber data dalam penelitian ini bersifat primer, di mana pengambilan data dilakukan secara langsung dari interaksi pengguna dengan sistem tanpa menggunakan dataset publik. Fokus pengambilan data terletak pada inputan suara yang diterima oleh sistem dari user yang berbicara dalam rapat. Rincian dan klasifikasi data tersebut dapat dilihat pada Tabel 1.1:

Tabel 1.1 Fitur Data Notulensi

No	Fitur	Deskripsi
1	Durasi rapat	Lama berlangsungnya rapat atau rekaman yang digunakan sebagai data penelitian, dihitung dalam satuan menit.
2	Jumlah pembicara	Banyaknya peserta yang terlibat dan aktif berbicara selama proses rapat berlangsung.
3	Tingkat kebisingan	Kondisi lingkungan audio pada saat rapat berlangsung yang dikategorikan menjadi tingkat rendah, sedang, dan tinggi berdasarkan intensitas suara gangguan di sekitar.
4	Kejelasan suara	Tingkat kejernihan suara pembicara yang diterima oleh sistem sehingga mempengaruhi kualitas proses transkripsi.

5	Tumpang tindih suara	Kondisi ketika dua atau lebih pembicara berbicara secara bersamaan dalam satu waktu selama rapat berlangsung.
6	Format File	Jenis format audio atau video yang digunakan sebagai input sistem, seperti mp3, wav, atau mp4.
7	Topik Rapat	Tema atau pokok pembahasan yang dibicarakan selama rapat atau simulasi berlangsung seperti meeting basis data, zoom meeting pelatihan keselamatan kerja, zoom materi psikologi, dan meeting lainnya seperti ilmu komunikasi.

Sumber data dalam penelitian ini bersifat primer, di mana pengambilan data dilakukan secara langsung dari proses interaksi pengguna dengan sistem notulensi rapat otomatis tanpa menggunakan dataset publik. Data yang digunakan berupa audio percakapan rapat yang diproses melalui dua mode sistem, yaitu mode live transcription dan mode upload rekaman.

Pada mode live transcription, data diperoleh secara langsung melalui interaksi pengguna dengan sistem menggunakan mikrofon yang terhubung ke perangkat audio. Proses pengambilan data dilakukan dalam bentuk simulasi rapat atau podcast dengan topik yang telah ditentukan sebelumnya. Simulasi tersebut melibatkan beberapa pembicara untuk menghasilkan percakapan yang menyerupai kondisi rapat nyata. Selain itu, pengujian dilakukan pada beberapa kondisi lingkungan dengan tingkat kebisingan berbeda, yaitu rendah, sedang, dan tinggi, untuk mengukur performa sistem dalam kondisi akustik yang bervariasi.

Sedangkan pada mode upload rekaman, data diperoleh dari hasil rekaman rapat berupa file audio maupun video yang kemudian diunggah ke sistem untuk diproses. Sumber rekaman diambil dari dokumentasi rapat dan video rapat yang tersedia pada

platform seperti YouTube sebagai bahan pengujian kemampuan sistem dalam melakukan transkripsi dan summarization pada data non-realtime.

3.2 Analisis kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk merumuskan spesifikasi dan cakupan sistem yang akan dibangun, sehingga sistem dapat berjalan sesuai dengan tujuan yang telah ditetapkan. Analisis ini dibagi menjadi dua bagian, yaitu kebutuhan fungsional dan kebutuhan non-fungsional.

3.2.1 Analisis Kebutuhan Fungsional

Kebutuhan fungsional merupakan kumpulan kebutuhan yang mencakup seluruh entitas yang terlibat dalam sistem, serta berbagai proses atau aktivitas yang dijalankan oleh masing-masing entitas tersebut. Berikut kebutuhan fungsional untuk User pada sistem ini.

1. Admin
 - a. Melakukan Login
 - b. Memulai live Rapat/ upload
 - c. Melihat Hasil summarize Rapat
 - d. Mendownload hasil PDF
 - e. Add user baru
 - f. Melakukan logout
2. User
 - a. Melakukan login
 - b. Melihat hasil summarize rapat
 - c. Melakukan logout

3.2.2 Analisis Kebutuhan Non Fungsional

Kebutuhan non-fungsional merupakan jenis kebutuhan yang mencakup aspek-aspek pendukung sistem yakni perangkat lunak (software) dan perangkat keras (hardware). Software yang digunakan pada penelitian ini dapat dilihat pada Tabel 1.2.

Tabel 1.2 Spesifikasi Perangkat lunak (*software*)

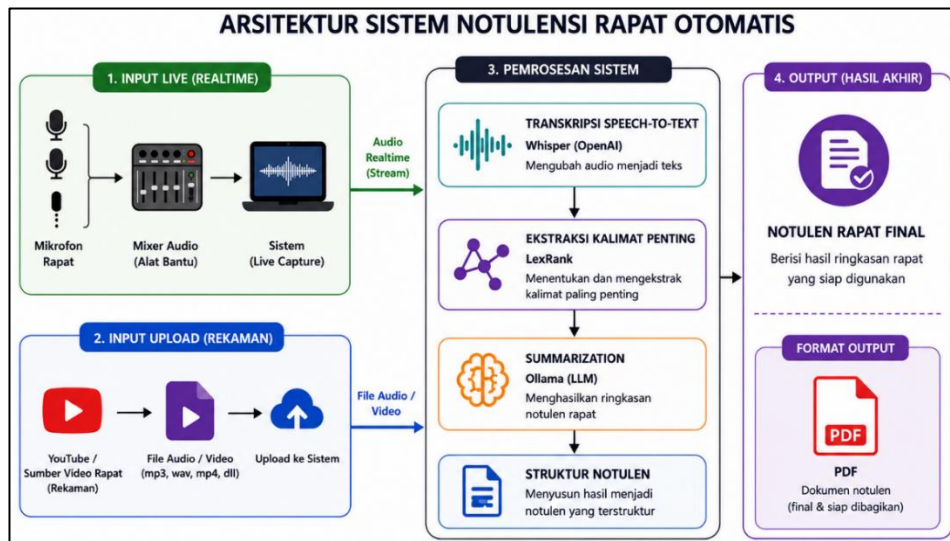
No.	Nama Software	Spesifikasi/Fungsi
1	Apache	Sebagai <i>web server</i>
2	MySQL 5.7.39	Sebagai <i>database</i>
3	Visual Studio Code	Sebagai <i>text editor</i>
4	Drawio	Merancang Diagram
5	Figma	Merancang UI/UX
6	PHP 8.1.10	Tampilan website
7	Ollama 3.2 3B	Untuk <i>Text Summarization</i>

Hardware yang digunakan dalam penelitian ini adalah satu buah laptop dengan spesifikasi processor Intel Core i7, RTX 3050Ti, RAM 16GB, dan SSD 512GB

3.3 Rancangan Sistem

3.3.1 Arsitektur Sistem

Arsitektur sistem pada penelitian ini dirancang dalam beberapa modul fungsional untuk memisahkan proses deteksi perintah, pengolahan suara, ekstraksi data, dan penyimpanan basis data, sebagaimana ditampilkan pada Gambar 1.1



Gambar 1.1 Arsitektur sistem Notulensi Rapat

Proses kerja sistem notulensi rapat otomatis diawali dari tahap pengambilan input yang terdiri dari dua mode, yaitu live transcription (realtime) dan upload rekaman. Pada mode live, sistem menerima input berupa suara langsung dari pengguna melalui mikrofon yang terhubung ke sistem, sehingga percakapan rapat dapat diproses secara realtime. Sedangkan pada mode upload, pengguna memasukkan file audio atau video rekaman rapat ke dalam sistem. Kedua jenis input ini menjadi sumber utama data yang akan diproses lebih lanjut dalam sistem notulensi otomatis.

Setelah data audio diterima, sistem menjalankan menggunakan model Whisper, yang berfungsi mengubah sinyal audio menjadi teks transkripsi. Hasil transkripsi ini masih berupa teks mentah yang berisi seluruh percakapan rapat. Selanjutnya, teks proses speech-to-text tersebut diproses menggunakan metode LexRank untuk mengekstraksi kalimat-kalimat penting berdasarkan tingkat keterkaitan dan relevansi antar kalimat. Tahap ini bertujuan untuk menyaring informasi utama dari keseluruhan percakapan sehingga hanya bagian yang paling penting yang dipertahankan.

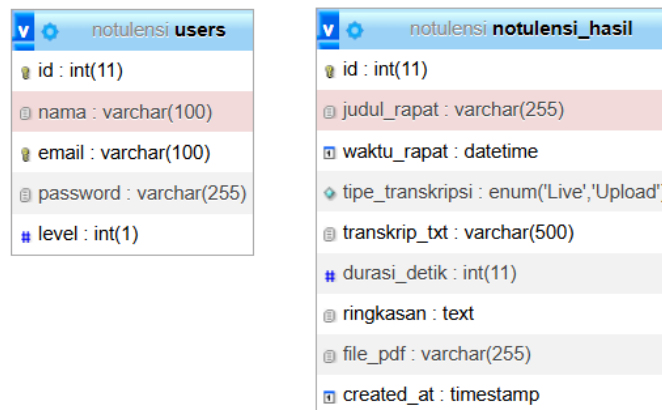
Hasil ekstraksi kalimat penting kemudian diproses lebih lanjut pada tahap summarization menggunakan model Ollama (LLM). Pada tahap ini, sistem

melakukan penyusunan ulang dan peringkasan teks menjadi bentuk notulen yang lebih terstruktur dan mudah dipahami. Output akhir dari sistem berupa notulen rapat final yang berisi ringkasan hasil rapat secara keseluruhan. Notulen tersebut kemudian disajikan dalam bentuk dokumen digital yang dapat diunduh, yaitu dalam format PDF, sehingga memudahkan pengguna dalam menyimpan, membagikan, dan mendokumentasikan hasil rapat secara formal dan sistematis.

3.3.2 Rancangan Database

a. Class Diagram

Database yang digunakan pada sistem Notulensi rapat akan dijelaskan diagram pada Gambar



Gambar 1.2 Rancangan Database

Gambar 1.2 merupakan Class Diagram yang menggambarkan struktur dan relasi antar kelas pada sistem notulensi rapat berbasis transkripsi dan peringkasan

otomatis. Diagram ini mendeskripsikan entitas utama, atribut yang dimiliki masing-masing entitas, entitas pada database ini berdifat independen dan tidak memiliki relasi.

1. Kelas Users

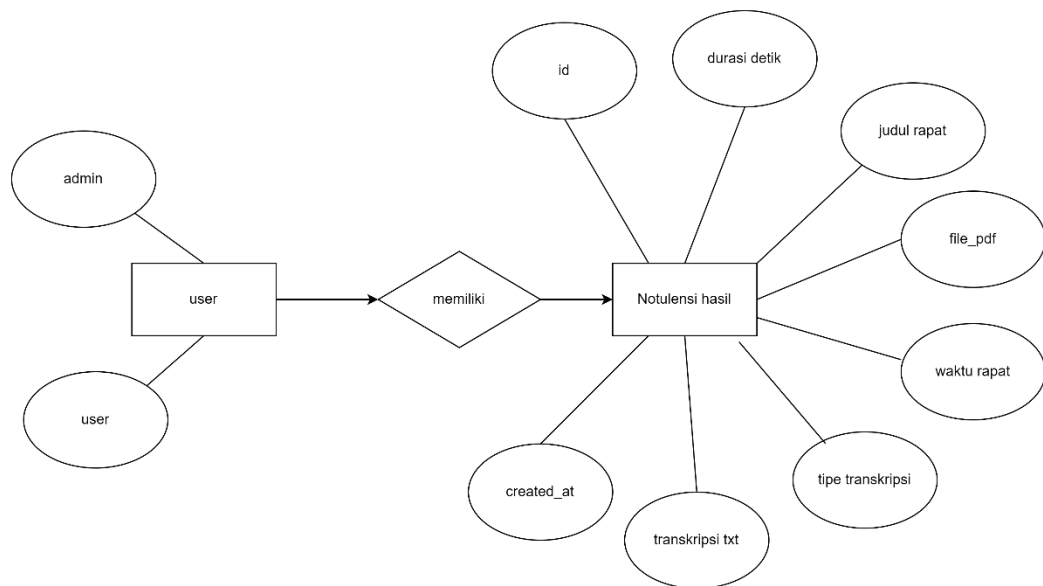
Kelas *Users* menyimpan informasi pengguna yang memiliki akses ke dalam sistem. Atribut yang terdapat pada kelas ini meliputi *id*, *nama*, *email*, *password*, dan *level*. Setiap pengguna memiliki peran tertentu, yaitu sebagai **Admin** atau **User**, yang ditentukan berdasarkan nilai pada atribut *level*. Dalam pengembangan sistem, kelas ini dapat memiliki relasi satu-ke-banyak terhadap data *notulensi_hasil*, di mana satu pengguna dapat menghasilkan banyak data notulensi.

2. Kelas Notulensi Hasil

Kelas *Notulensi_Hasil* merupakan entitas utama yang digunakan untuk menyimpan hasil pengolahan notulensi rapat. Atribut yang terdapat pada kelas ini meliputi *id*, *judul_rapat*, *waktu_rapat*, *tipe_transkripsi*, *transkrip_txt*, *durasi_detik*, *ringkasan*, *file_pdf*, dan *created_at*.

Diagram ini menunjukkan keterkaitan antara data pengguna dan hasil notulensi dalam sistem. Relasi antar kelas menggambarkan alur proses mulai dari transkripsi hingga penyimpanan dan penyajian hasil notulensi. Model ini mendukung integritas data serta efisiensi pengolahan data secara sistematis.

b. Rancangan Tabel Entity Relationship Diagram (ERD)



Gambar 1.3 Rancangan ERD

Gambar 3.10 menunjukkan *Entity Relationship Diagram (ERD)* pada sistem notulensi rapat. Pada sistem ini terdapat 2 entitas utama yaitu **users** dan **notulensi_hasil**. Tabel **users** digunakan untuk menyimpan data pengguna sistem, sedangkan tabel **notulensi_hasil** digunakan untuk menyimpan hasil notulensi rapat yang meliputi transkrip, ringkasan, dan file hasil. Pada rancangan database ini, tidak terdapat relasi langsung antara tabel **users** dan **notulensi_hasil**, sehingga kedua entitas berdiri secara independen.

c. Rancangan Tabel Database

Deskripsi tabel data yang digunakan pada sistem notulensi rapat terdiri dari beberapa tabel, yakni:

1) Tabel Users

Tabel *Users* berfungsi untuk menyimpan data pengguna yang digunakan dalam proses autentikasi saat login ke dalam sistem notulensi. Setiap atribut pada tabel ini dirancang untuk merepresentasikan informasi penting yang berkaitan dengan identitas pengguna, seperti nama, email, serta keamanan akun melalui

password. Selain itu, tabel ini juga mengatur hak akses pengguna melalui atribut *level* yang membedakan peran sebagai admin dan user. Penjelasan rinci mengenai atribut-atribut yang terdapat pada tabel *Users* disajikan pada Tabel 1.3.

Tabel 1.3 Rancangan Tabel *Users*

No	Nama Field	Tipe Data	Panjang	Keterangan
1	id	int	11	Primary key, ID pengguna
2	nama	varchar	100	Nama pengguna
3	email	varchar	100	Email pengguna (unik)
4	password	varchar	255	Password yang telah dienkripsi
5	level	int	1	Hak akses (1: Admin, 2: User)

2) Tabel Notulensi Hasil

Tabel **notulensi_hasil** berfungsi untuk menyimpan data hasil proses notulensi rapat yang dihasilkan oleh sistem. Tabel ini mencakup informasi seperti identitas rapat, hasil transkripsi, ringkasan, serta file output dalam bentuk PDF. Penjelasan rinci mengenai atribut pada tabel **notulensi_hasil** disajikan pada Tabel 1.4.

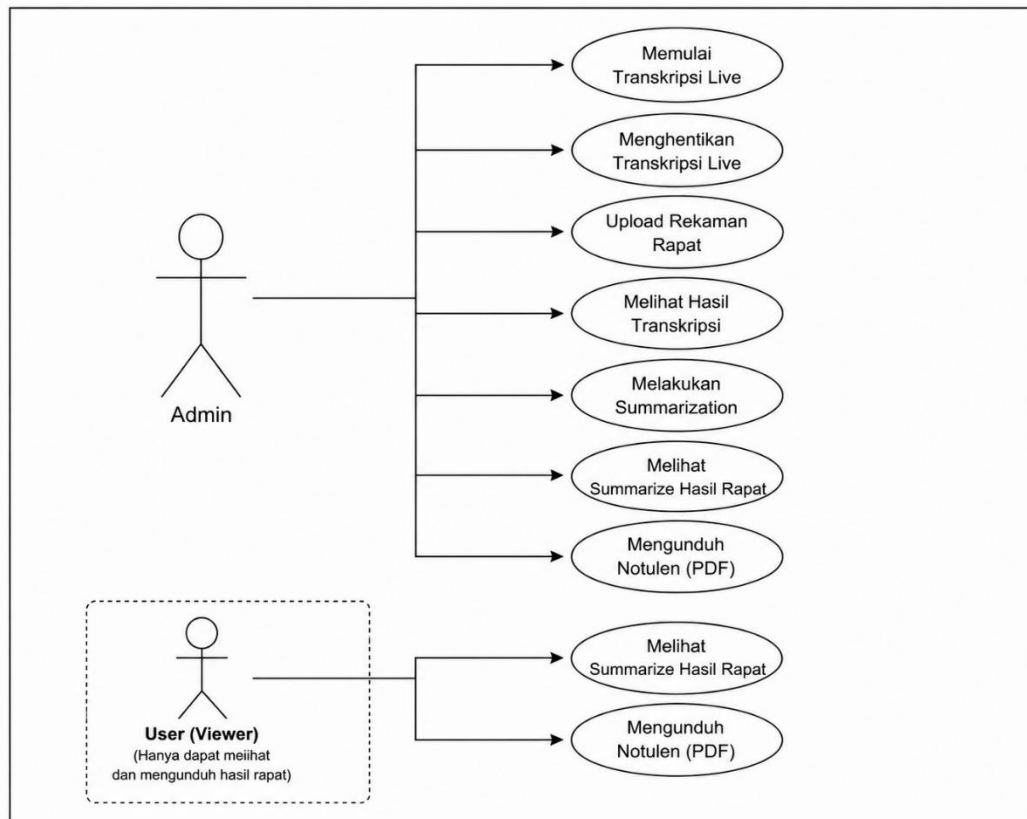
Tabel 1.4 Rancangan Tabel Notulensi Hasil

No	Nama Field	Tipe Data	Panjang	Keterangan
1	id	int	11	Primary key
2	judul_rapat	varchar	255	Judul rapat
3	waktu_rapat	datetime	-	Waktu pelaksanaan rapat

4	tipe_transkripsi	enum	-	Jenis transkripsi (Live / Upload)
5	transkrip_txt	varchar	500	Nama file hasil transkrip (.txt)
6	durasi_detik	int	11	Durasi audio/video dalam detik
7	ringkasan	text	-	Hasil ringkasan notulensi
8	file_pdf	varchar	255	File hasil notulensi dalam format PDF

3.3.3 Use Case Diagram

Use Case Diagram pada sistem notulensi rapat otomatis melibatkan satu aktor utama, yaitu **Admin**, yang berinteraksi dengan sistem melalui beberapa use case utama, yaitu memulai transkripsi live, menghentikan transkripsi, mengunggah rekaman rapat, melihat hasil transkripsi, melakukan summarization, melihat hasil notulen, serta mengunduh notulen dalam bentuk dokumen (PDF); rangkaian use case tersebut menggambarkan alur penggunaan sistem mulai dari proses input audio hingga menghasilkan notulen rapat yang terstruktur dan siap digunakan, sebagaimana ditunjukkan pada Gambar 1.4.



Gambar 1.4 Use case diagram admin dan user

Use Case Diagram pada sistem notulensi rapat otomatis ini melibatkan dua aktor, yaitu *Admin* sebagai pengguna utama sistem dan *User (Viewer)* sebagai pengguna dengan hak akses terbatas. Admin memiliki kontrol penuh terhadap sistem, termasuk memulai dan menghentikan proses transkripsi live, mengunggah rekaman rapat, melihat hasil transkripsi, melakukan proses summarization, melihat hasil ringkasan rapat, serta mengunduh notulen dalam format PDF. Alur ini menggambarkan proses utama sistem mulai dari pengolahan input audio hingga menghasilkan dokumen notulen yang siap digunakan.

Sementara itu, aktor *User (Viewer)* hanya memiliki akses terbatas, yaitu melihat hasil ringkasan rapat dan mengunduh notulen yang telah dihasilkan oleh sistem. Pembagian peran ini menunjukkan adanya pemisahan hak akses dalam sistem, di mana hanya Admin yang dapat melakukan proses pengolahan data, sedangkan User (Viewer) berperan sebagai pihak yang mengonsumsi hasil akhir. Diagram ini secara

keseluruhan merepresentasikan interaksi antara aktor dengan sistem dalam menghasilkan notulen rapat otomatis yang terstruktur dan efisien.

3.1.1.1 Definisi actor

Definisi actor pada system dijelaskan untuk menggambarkan peran setiap pihak yang terlibat dalam penggunaan sistem. Deskripsi aktor dalam sistem dapat dilihat pada tabel 1.5

Tabel 1.5 Definisi actor

No	Aktor	Deskripsi
1	Admin	Pengguna dengan akses penuh: transkripsi live, upload rekaman, melihat hasil, mengunduh notulen
2	User (viewer)	Pengguna dengan akses terbatas: melihat hasil ringkasan dan mengunduh notulen

3.1.1.2 Definisi Use Case

Definisi *Use Case* pada sistem dijelaskan untuk menggambarkan fungsi-fungsi utama yang dapat dilakukan oleh aktor dalam sistem serta interaksi antara aktor dan sistem. Uraian *Use Case* tersebut disajikan pada Tabel 1.6.

Tabel 1.6 Definisi Use Case

No	Use Case	Deskripsi
1	Mulai Transkripsi Live	Admin memulai proses transkripsi real-time dari mikrofon
2	Upload Rekaman	Admin mengunggah file audio/video rapat untuk diproses

3	Lihat Hasil Transkripsi	Admin / User melihat hasil transkripsi dari rapat
4	Lihat Ringkasan Rapat	Admin/User melihat ringkasan otomatis hasil summarization
5	Unduh kesimpulan Hasil Rapat	Admin/User mengunduh notulen dalam format PDF

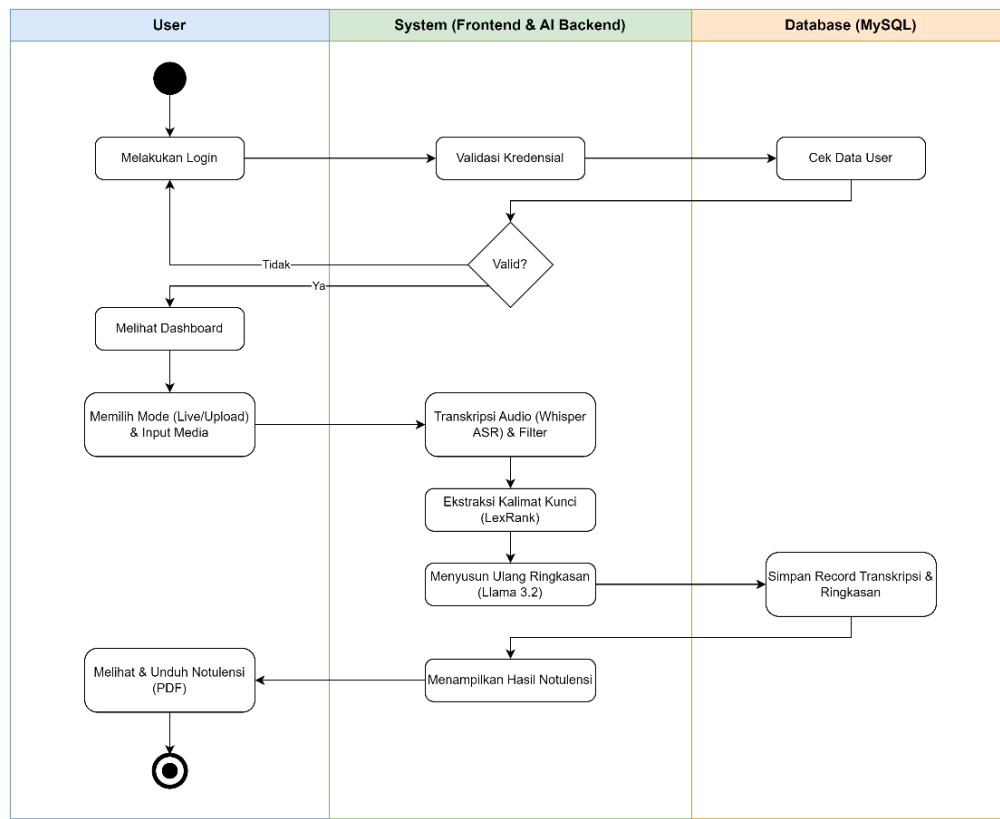
3.4 Preprocessing Data

Masih kosong

3.5 Activity Diagram

3.4.1 Activity Diagram Sistem Notulensi

Activity Diagram pada sistem notulensi rapat otomatis digunakan untuk menggambarkan alur kerja sistem secara dinamis, mulai dari input audio, proses transkripsi menggunakan Whisper, hingga peringkasan dengan LexRank dan model LLM. Diagram ini membantu memvisualisasikan proses secara jelas dan terstruktur, sebagaimana ditunjukkan pada Gambar 1.6



Gambar 1.6 Activity Diagram Sistem Notulensi Rapat

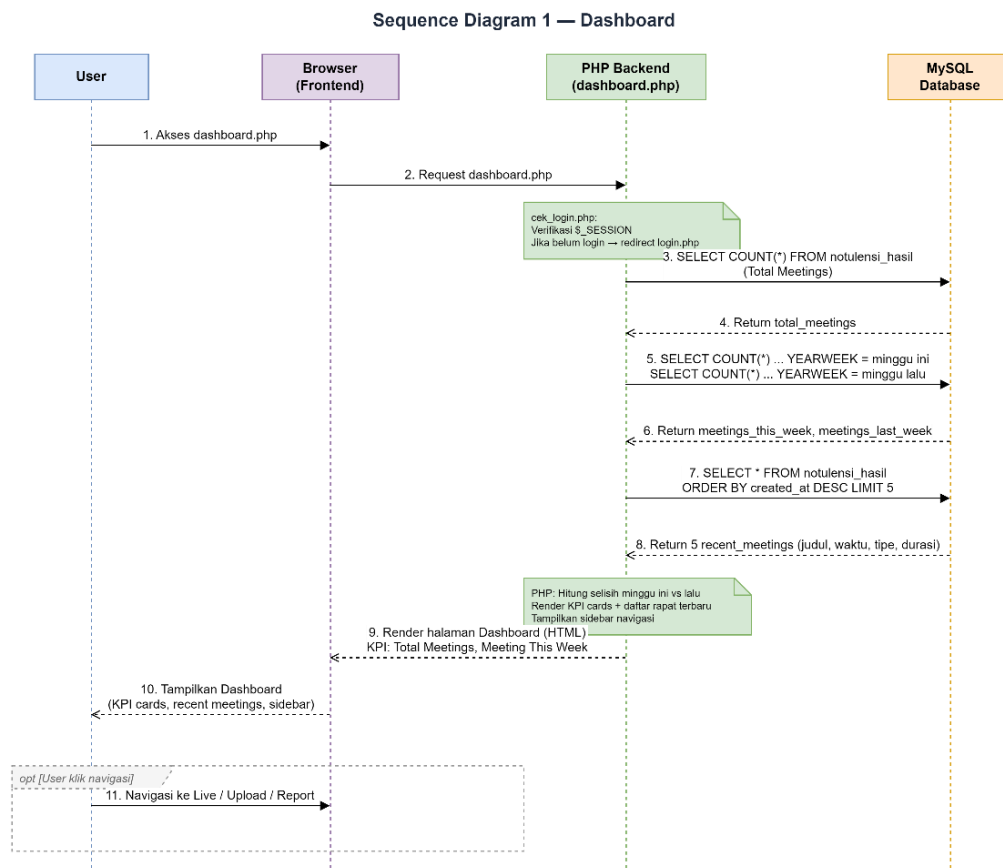
3.4.2 Sequence Diagram

Sequence diagram merupakan representasi visual dari alur interaksi yang terjadi antara pengguna dengan proses atau sistem dalam urutan waktu tertentu, dan menggambarkan pesan atau data dikirim dan diterima antar komponen.

1. Sequence Diagram Dashboard

Gambar 3.11 menunjukkan sequence diagram yang menggambarkan alur interaksi untuk menampilkan halaman Dashboard pada sistem notulensi rapat. Proses dimulai ketika *User* mengakses halaman Dashboard melalui *Browser (Frontend)*. Selanjutnya, Browser mengirimkan permintaan ke *Backend (PHP)* untuk memproses data yang dibutuhkan. Backend kemudian mengirimkan permintaan ke *Database* untuk mengambil data terkait, seperti data hasil notulensi atau ringkasan rapat. Database mengembalikan data tersebut ke

Backend, yang selanjutnya melakukan proses pengolahan data. Setelah proses selesai, Backend mengirimkan hasilnya kembali ke Browser untuk ditampilkan dalam bentuk halaman Dashboard kepada User.

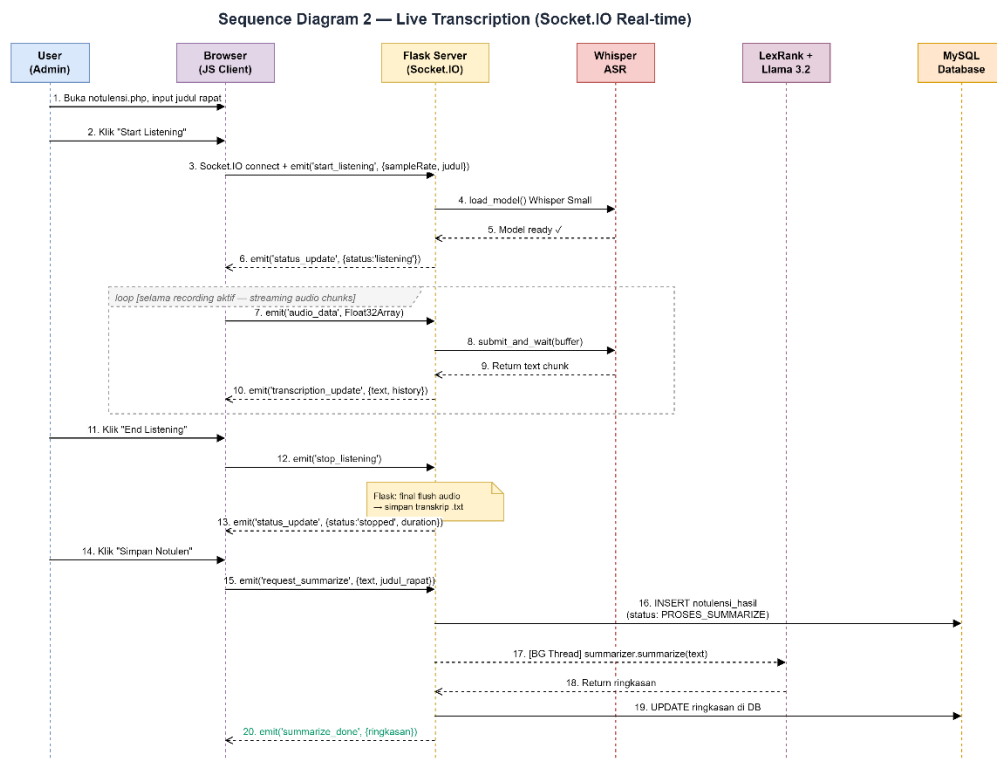


Gambar 1.7 *sequence Diagram Dashboard*

2. Sequence Diagram Live Transcription

Gambar 3.12 menunjukkan sequence diagram yang menggambarkan proses transkripsi rapat secara langsung (live transcription). Proses dimulai ketika User mengakses fitur Live Transcription melalui Browser. Browser kemudian mengirimkan data audio secara real-time ke Backend untuk diproses. Backend selanjutnya meneruskan data tersebut ke *API Speech-to-Text* (*Whisper/OpenAI*) untuk dilakukan proses transkripsi. API mengembalikan

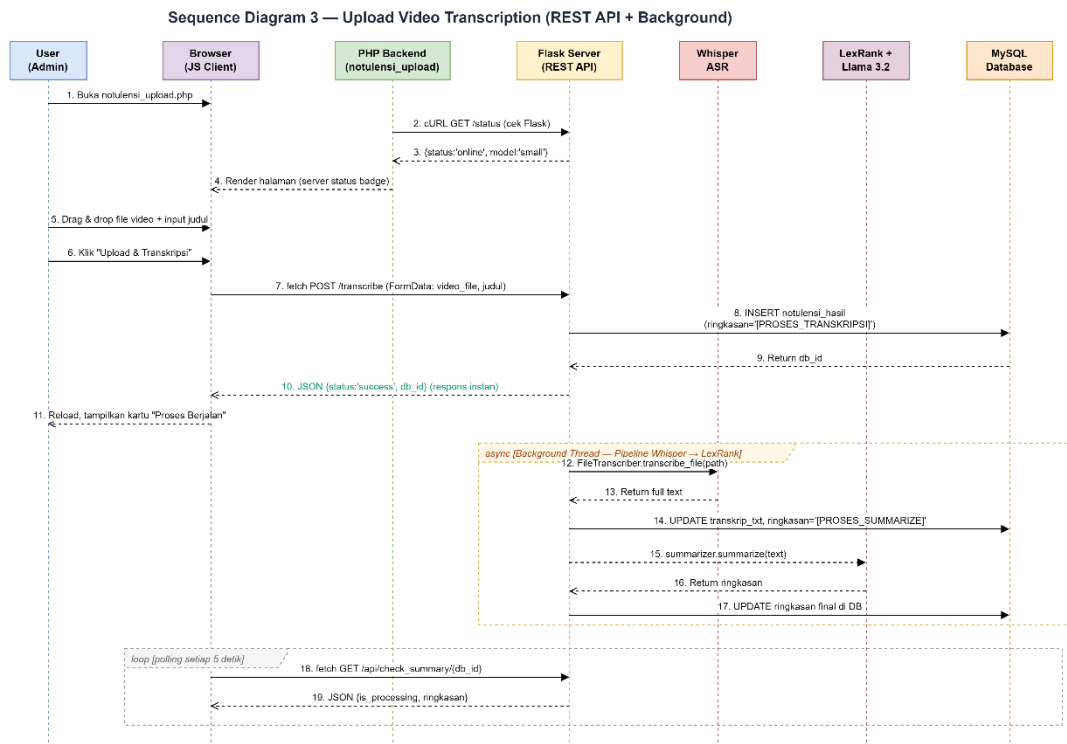
hasil transkripsi ke Backend, yang kemudian mengirimkan hasil tersebut ke Browser untuk ditampilkan secara langsung kepada User. Selain itu, hasil transkripsi juga dapat disimpan ke dalam Database sebagai data notulensi.



Gambar 1.8 Sequence diagram Live Transcription

3. Sequence Diagram Upload Transcription

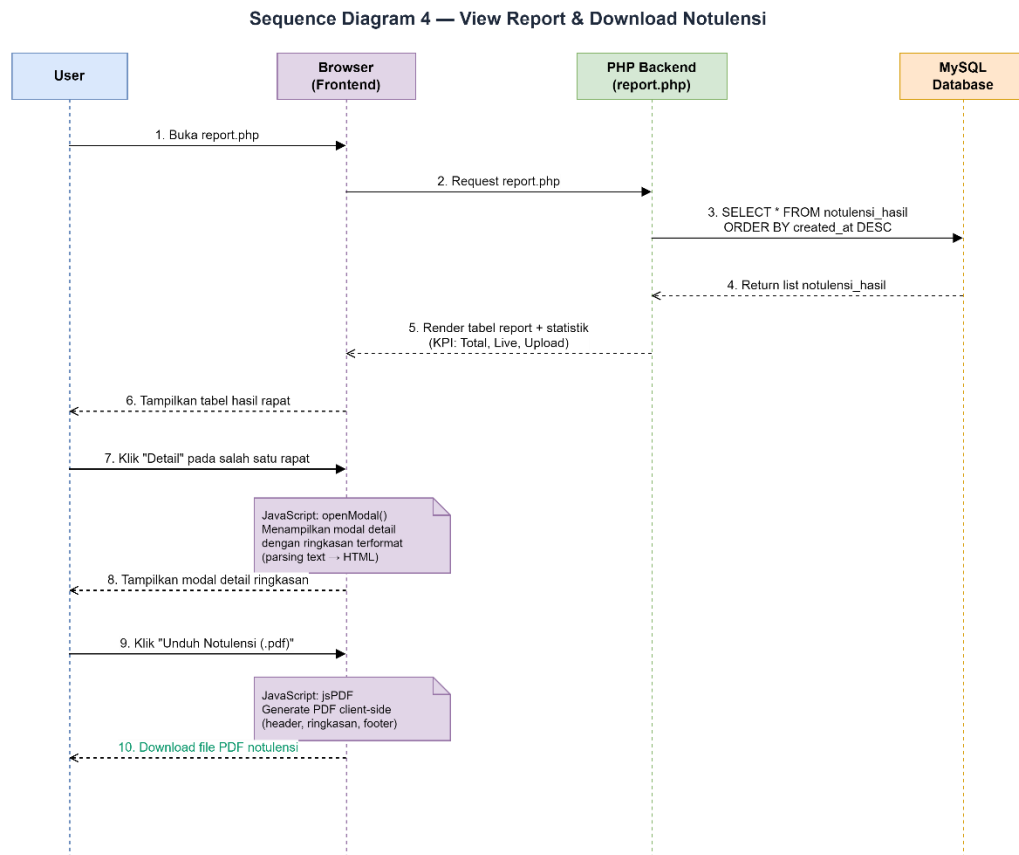
Gambar 3.13 menunjukkan sequence diagram yang menggambarkan proses upload file rekaman untuk ditranskripsikan. Proses dimulai ketika User mengakses halaman upload dan memilih file audio melalui Browser. File tersebut kemudian dikirim ke Backend untuk diproses. Backend meneruskan file audio ke API Speech-to-Text untuk dilakukan proses transkripsi. Setelah proses selesai, API mengembalikan hasil transkripsi ke Backend. Backend kemudian menyimpan hasil tersebut ke Database dan mengirimkan hasil transkripsi ke Browser untuk ditampilkan kepada User.



Gambar 1.9 Sequence Diagram Upload Transcription

4. Sequence Diagram View Report & Download Notulensi

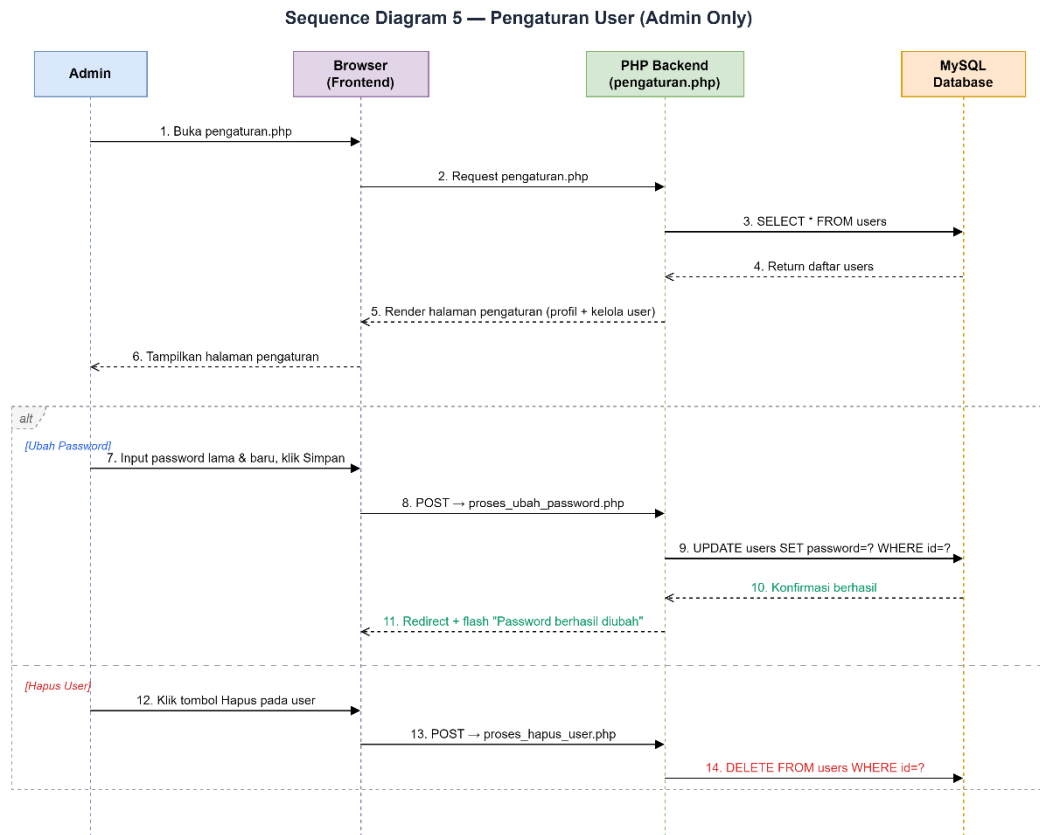
Gambar 3.14 menunjukkan sequence diagram yang menggambarkan proses melihat hasil laporan notulensi. Proses dimulai ketika User memilih menu laporan melalui Browser. Browser mengirimkan permintaan ke Backend untuk mengambil data laporan. Backend kemudian meminta data ke Database yang berisi hasil notulensi rapat yang telah disimpan sebelumnya. Database mengembalikan data tersebut ke Backend, yang kemudian mengolah data jika diperlukan. Setelah itu, Backend mengirimkan data laporan ke Browser untuk ditampilkan kepada User dalam bentuk laporan notulensi.



Gambar 1.10 Diagram View Report & Download Notulensi

5. Sequence Diagram Pengaturan (*Admin Only*)

Gambar 3.15 menunjukkan sequence diagram yang menggambarkan proses pengelolaan data pengguna. Proses dimulai ketika User mengakses halaman pengaturan melalui Browser. User dapat melakukan perubahan data seperti profil atau pengaturan akun. Data tersebut dikirim ke Backend untuk diproses dan divalidasi. Selanjutnya, Backend menyimpan perubahan data ke Database. Setelah proses penyimpanan berhasil, Backend mengirimkan respon ke Browser untuk menampilkan notifikasi bahwa perubahan data berhasil dilakukan kepada User.



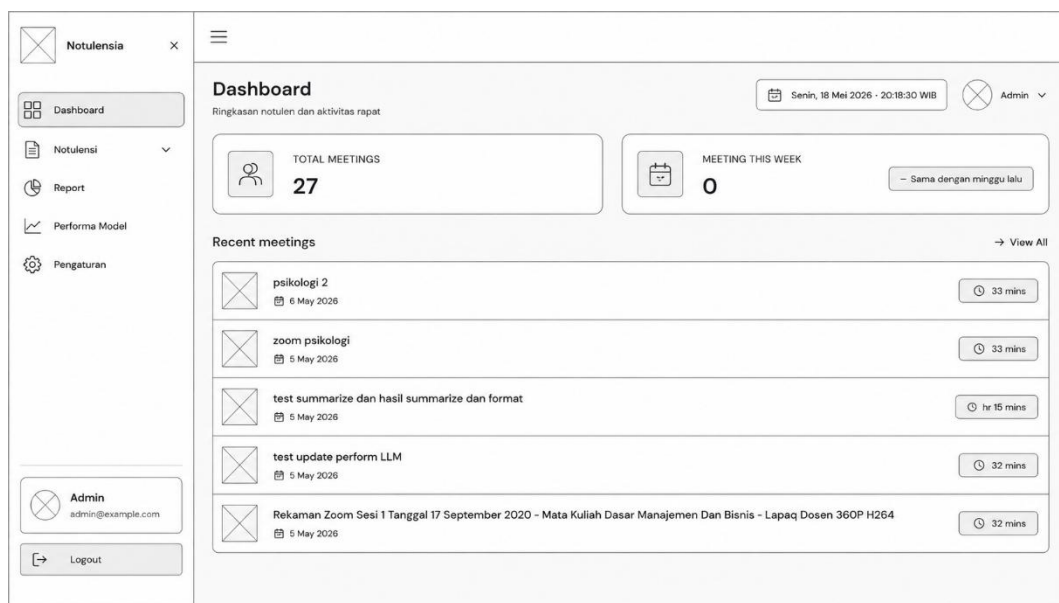
Gambar 1.11 Diagram Pengaturan (*Admin Only*)

3.6 Wireframe Sistem Notulensi

Untuk mengimplementasikan teknologi *speech-to-text* menggunakan OpenAI Whisper pada sistem notulensi rapat, diperlukan rancangan antarmuka yang menggambarkan alur kerja mulai dari perekaman atau unggah audio, transkripsi otomatis, hingga peringkasan menggunakan metode LexRank atau TextRank. Wireframe ini memvisualisasikan interaksi pengguna secara terstruktur dalam mengelola transkripsi, ringkasan, serta penyimpanan notulen secara efektif.

3.5.1 Tampilan Halaman Dashboard

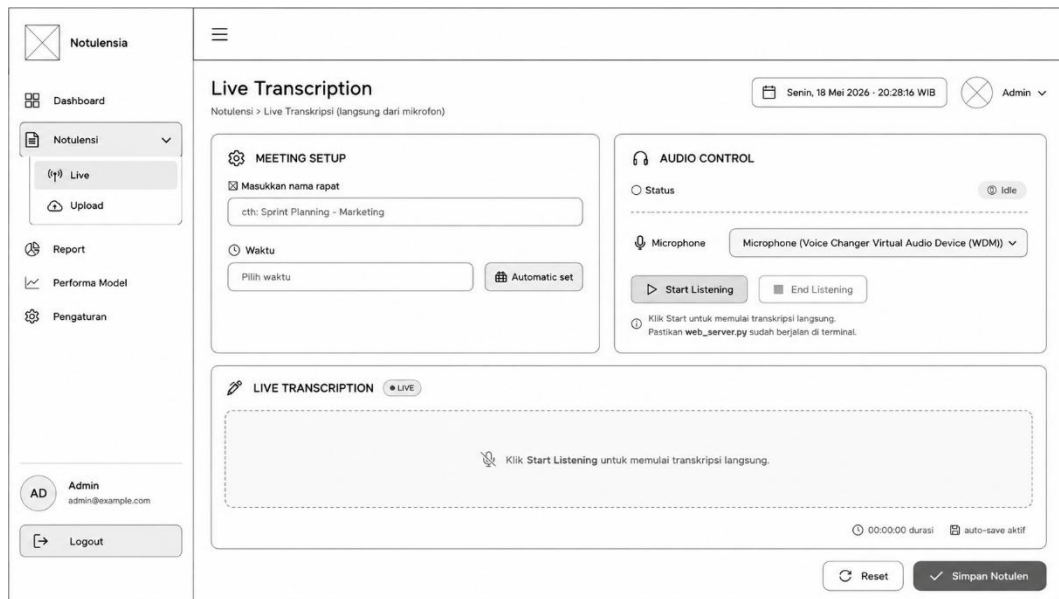
Halaman Dashboard merupakan antarmuka utama dalam sistem notulensi rapat otomatis yang mengintegrasikan fitur perekaman atau unggah audio, transkripsi, serta ringkasan notulen menggunakan OpenAI Whisper dan metode LexRank atau TextRank. Berikut merupakan tampilan rancangan (wireframe) yang menunjukkan struktur dan penempatan komponen utama sistem.



Gambar 1.12 Dashboard Sistem Notulensi

3.5.2 Tampilan Halaman live

Halaman Live Transcription merupakan antarmuka yang digunakan pengguna untuk melakukan pencatatan rapat secara langsung melalui input suara. Pada halaman ini, pengguna dapat memulai proses perekaman, melihat hasil transkripsi secara real-time, serta mengelola hasil notulen sebelum disimpan atau diproses lebih lanjut.

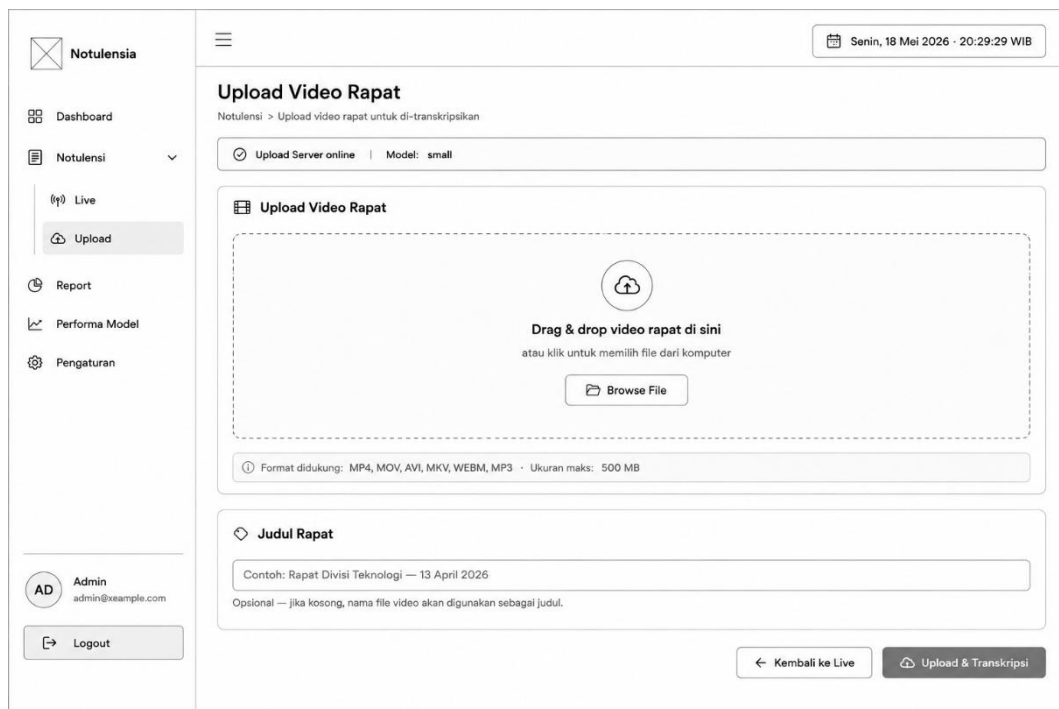


Gambar 1.13 Wireframe fitur live sistem notulensi rapat

Wireframe ini mengilustrasikan antarmuka halaman Live Transcription pada sistem notulensi rapat, di mana fitur perekaman suara berperan sebagai input utama. Pengguna dapat melihat hasil transkripsi rapat secara real-time pada area yang tersedia, serta melakukan penyuntingan sebelum data disimpan dan dilanjutkan ke proses peringkasan atau penyimpanan sebagai dokumen notulen.

3.5.3 Tampilan Halaman upload

Halaman Upload Notulensi merupakan antarmuka yang digunakan pengguna untuk mengunggah rekaman rapat dalam bentuk audio atau video ke dalam sistem. Pada halaman ini, pengguna dapat memilih atau melakukan *drag and drop* file, mengisi informasi terkait rapat, serta melanjutkan proses transkripsi dan pembuatan notulen secara otomatis.

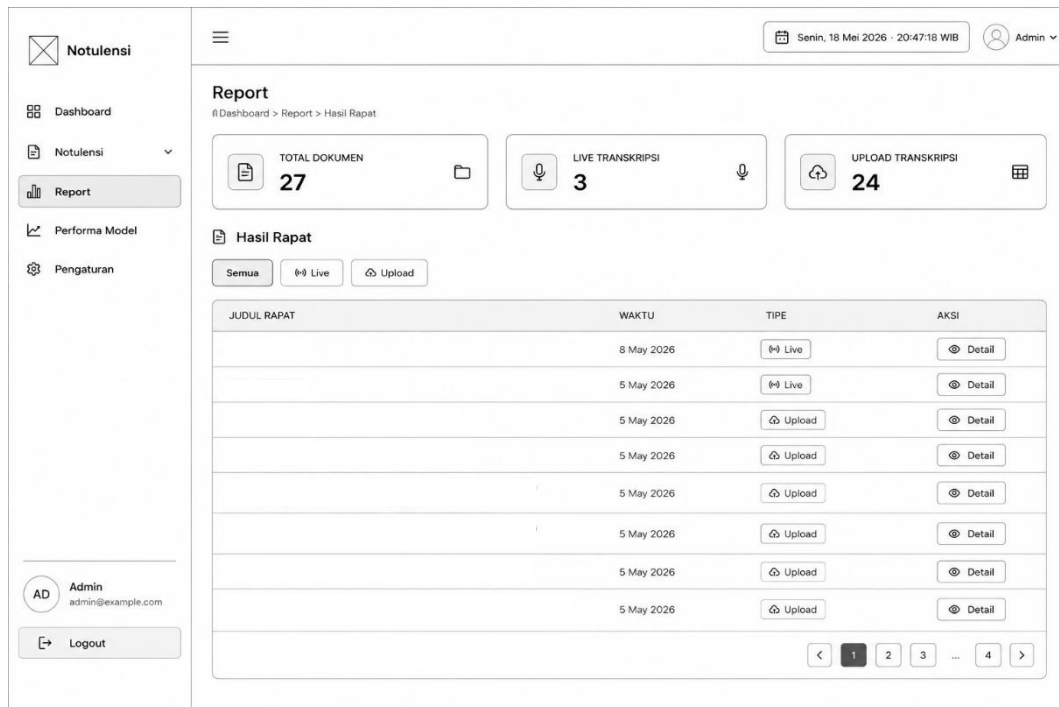


Gambar 1.14 Wireframe fitur upload sistem notulensi rapat

Halaman Upload Notulensi berfungsi untuk mengolah rekaman rapat dalam format audio atau video (seperti MP4 atau WAV) yang diunggah oleh pengguna. Sistem akan mengekstrak audio, mengubahnya menjadi teks transkripsi menggunakan OpenAI Whisper, kemudian merangkum isi rapat dengan metode LexRank atau TextRank, dan menyimpannya sebagai dokumen notulensi.

3.5.4 Tampilan Halaman Report

Halaman Report merupakan antarmuka yang digunakan untuk menampilkan dan mengelola hasil notulensi rapat yang telah diproses dalam sistem. Pada halaman ini, pengguna dapat melihat daftar dokumen notulen, mengakses detail hasil rapat, serta melakukan pengelolaan data notulensi secara terstruktur.

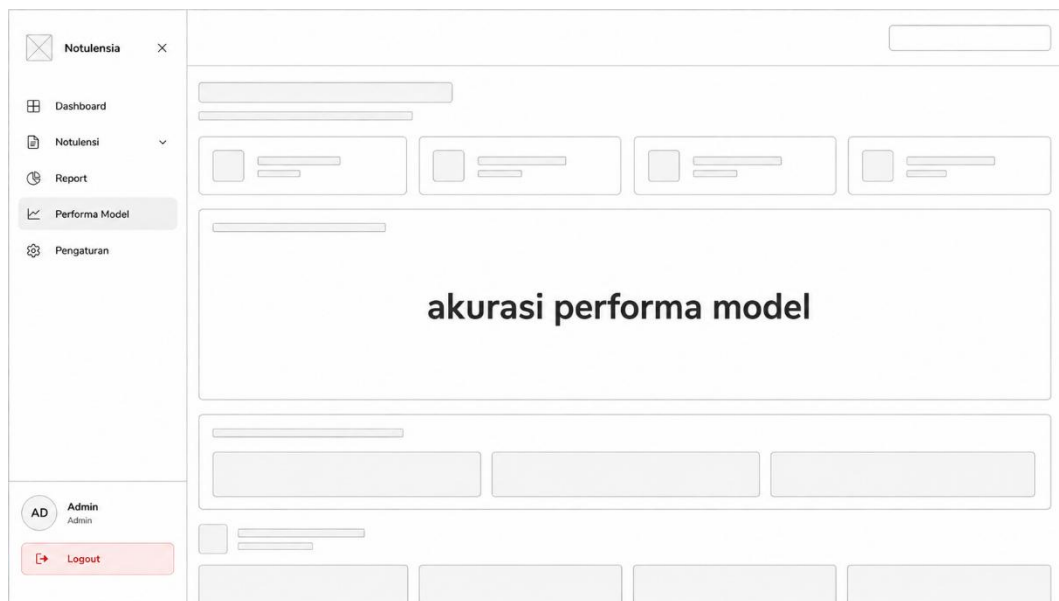


Gambar 1.15 Wireframe Report hasil summarization

Halaman Report berfungsi untuk menampilkan hasil akhir dari proses notulensi rapat yang telah dilakukan, baik melalui fitur live transcription maupun upload rekaman. Data yang ditampilkan merupakan hasil transkripsi yang telah diverifikasi dan diringkas menggunakan metode LexRank atau TextRank, sehingga pengguna dapat melihat, mengelola, dan mengakses kembali dokumen notulen rapat secara terstruktur.

3.5.5 Tampilan Halaman Performa Model

Halaman Performa Model merupakan antarmuka yang digunakan untuk menampilkan dan mengevaluasi kinerja model yang digunakan dalam sistem notulensi rapat. Pada halaman ini, pengguna dapat melihat metrik evaluasi seperti Word Error Rate (WER), akurasi (WACC), Character Error Rate (CER), serta waktu inferensi, sekaligus mengakses visualisasi grafik performa model dan informasi spesifikasi model secara terstruktur.

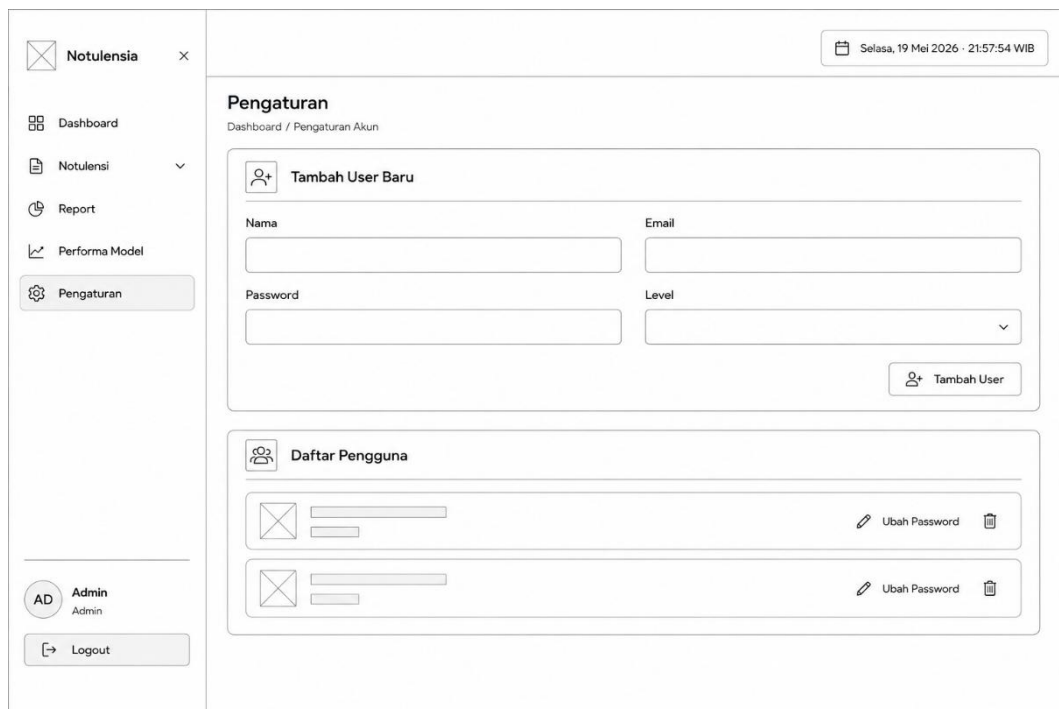


Gambar 1.16 Wireframe performa model Whisper, Lexrank, LLM

Halaman Performa Model berfungsi untuk menampilkan hasil evaluasi kinerja model yang digunakan dalam sistem notulensi rapat, khususnya model transkripsi seperti OpenAI Whisper dan metode peringkasan seperti LexRank atau TextRank. Data yang ditampilkan berupa metrik evaluasi seperti Word Error Rate (WER), akurasi, Character Error Rate (CER), serta waktu inferensi, sehingga pengguna dapat menganalisis, membandingkan, dan memahami performa model secara terstruktur.

3.5.6 Tampilan Halaman pengaturan

Halaman Pengaturan merupakan antarmuka yang digunakan untuk mengelola konfigurasi sistem serta data pengguna dalam sistem notulensi rapat. Pada halaman ini, pengguna dapat menambahkan dan mengelola akun baru, mengatur informasi seperti nama, email, dan kata sandi, serta menentukan level atau hak akses pengguna. Selain itu, halaman ini juga menyediakan fitur untuk melakukan pembaruan data dan pengelolaan keamanan akun secara terstruktur, sehingga sistem dapat digunakan dengan lebih terkontrol dan aman.



Gambar 1.17 Wireframe halaman pengaturan sistem notulensi

Halaman Pengaturan berfungsi untuk mengelola data pengguna dan konfigurasi dasar dalam sistem notulensi rapat. Pada halaman ini, pengguna dapat menambahkan akun baru, mengatur informasi seperti nama, email, dan kata sandi, serta menentukan level atau hak akses pengguna. Data yang ditampilkan memungkinkan pengguna untuk mengelola, memperbarui, dan menjaga keamanan akun secara terstruktur sehingga penggunaan sistem menjadi lebih terkontrol dan efisien.

3.7 Metode dan Variabel Penelitian

3.6.1 Whisper

Whisper adalah sistem pengenalan suara otomatis (*Automatic Speech Recognition / ASR*) yang dilatih menggunakan 680.000 jam data multibahasa dan multitugas yang diawasi serta dikumpulkan dari web. Model ini dirancang untuk mampu melakukan berbagai tugas seperti transkripsi, penerjemahan, dan deteksi bahasa dalam satu arsitektur berbasis Transformer *encoder-decoder*. Dengan skala

data pelatihan yang besar, Whisper memiliki kemampuan generalisasi yang baik, sehingga mampu mengenali berbagai aksen, variasi bahasa, serta tetap akurat pada kondisi lingkungan yang bising. Hal ini menjadikan Whisper sangat sesuai digunakan dalam sistem notulensi rapat otomatis yang membutuhkan hasil transkripsi yang konsisten dan andal.

Spesifikasi Implementasi:

1. Model Whisper: Whisper Small (244 juta parameter)
2. Input: Audio berbahasa Indonesia (mono atau stereo)
3. Sampling rate: 16 kHz

Representasi Whisper dalam memetakan sinyal audio ke teks adalah sebagai berikut:

$$Y = f_{\theta}(X)$$

Keterangan :

1. XXX = sinyal audio (waveform)
2. YYY = urutan token teks hasil transkripsi
3. f_{θ} = model Whisper berbasis Transformer
4. θ = parameter model

Jika dirinci dalam arsitektur encoder-decoder:

$$\begin{aligned} H &= \text{Encoder}(X), \\ Y &= \text{Decoder}(H) \end{aligned}$$

Evaluasi menggunakan Word Error Rate (WER):

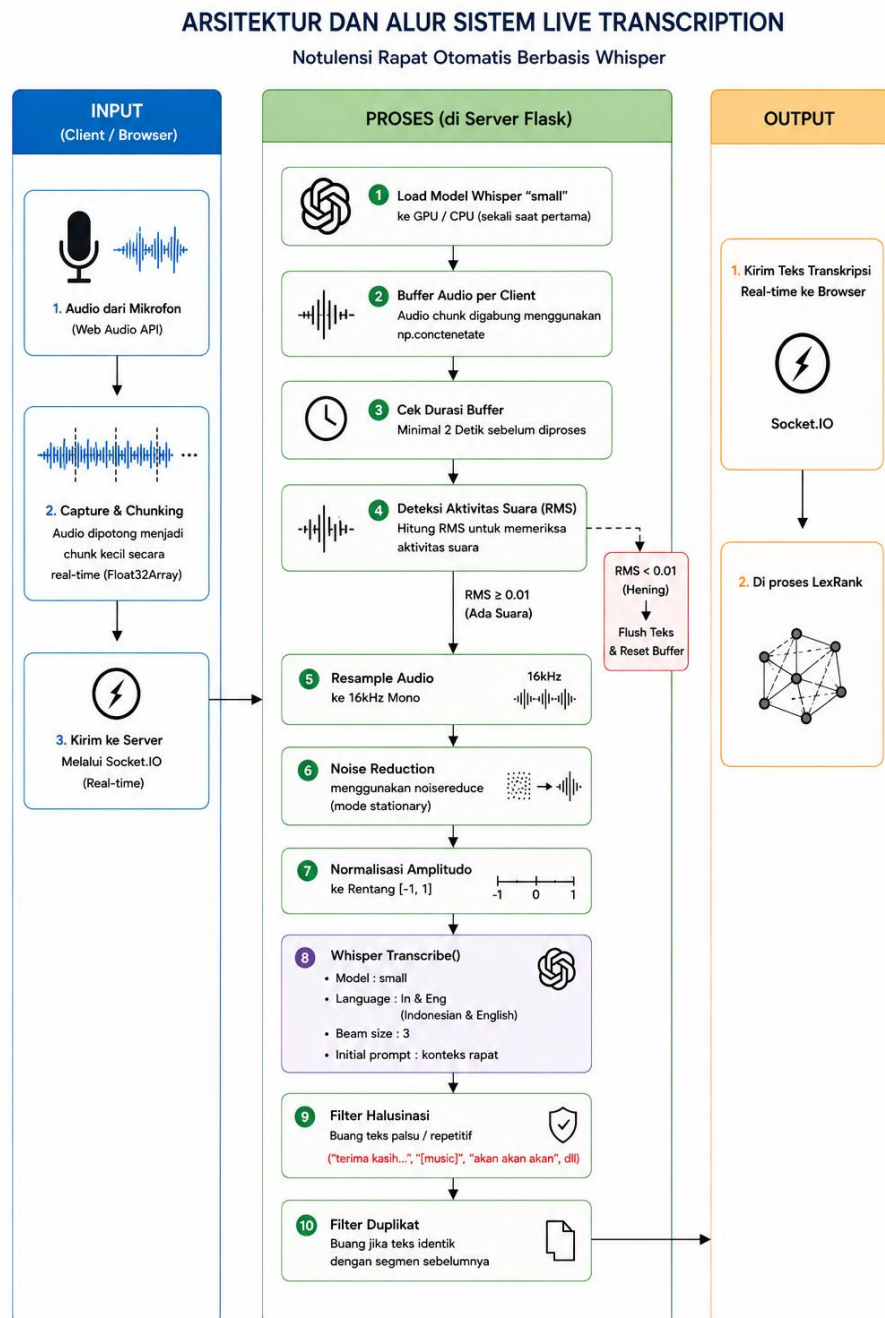
$$WER = \frac{S + D + I}{N} \times 100\%$$

Keterangan:

1. S = Substitusi

2. D = Delesi
3. I = Inseri
4. N = Total Kata referensi

dan akurasi: $\text{Accuracy} = (1 - \text{WER})$

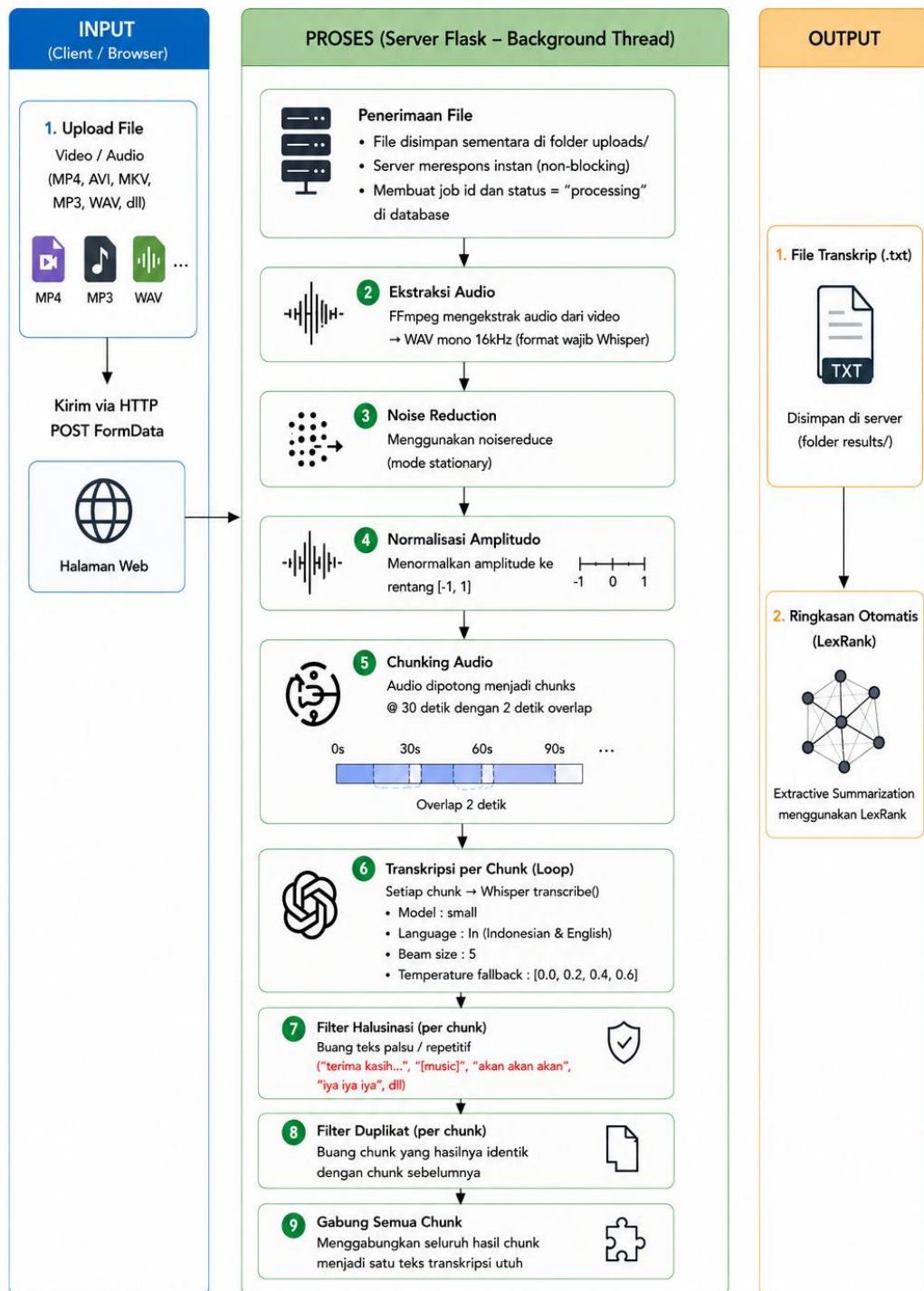


Gambar 1.18 Arsitektur *Whisper Live Transkripsi*

Kemudian ada fitur kedua yaitu fitur upload rapat yang digunakan dengan cara mengupload hasil rekaman rapat, berikut adalah arsitektur nya

ARSITEKTUR DAN ALUR SISTEM WHISPER – MODE UPLOAD FILE

Notulensi Rapat Otomatis



Gambar 1.19 Arsitektur *Whisper* Upload Rapat

3.6.2 Word filler dan Noise filter

3.1.1.3 Word filler removal

Word filler adalah kata-kata pengisi (anu, ya, jadi, kan, gitu) yang sering muncul dalam ucapan spontan dan tidak mengandung informasi substantif. Tahap ini menghilangkan noise non-informatif sebelum ekstraksi kalimat penting.

Komponen Noise Filtering

1. Lexicon-based filtering menggunakan daftar kata pengisi Bahasa Indonesia
2. Pattern-based filtering mendeteksi pengulangan karakter dan token akustik
3. Syntactic validation mendeteksi fragmentasi akibat tumpang tindih bicara

Representasi Word filler removal dalam penghapusan kalimat yang tidak penting adalah sebagai berikut:

$$T' = T \setminus F$$

Atau

$$[T' = \{w \in T \mid w \notin F\}]$$

Keterangan:

1. T = himpunan kata hasil transkripsi dari Whisper
2. F = himpunan kata *word filler* (misalnya: “anu”, “ya”, “jadi”, “kan”, “gitu”)
3. W_i = kata ke- i dalam teks
4. T' = teks setelah penghapusan word filler

3.1.1.4 Noise filter

Noise Filter adalah proses penyaringan atau pembersihan teks transkripsi dari elemen-elemen yang tidak informatif atau mengganggu. Dalam konteks pengolahan ucapan spontan, noise filter bertugas mengidentifikasi kebisingan dan mengurangi kebisingan pada input suara yang diterima.

Representasi noise filter dalam pereda kebisingan dalam input rapat pada live transkripsi adalah sebagai berikut:

$$T = f_{\theta}(X(t) - N(t))$$

Keterangan:

1. $X(t)$ = sinyal audio yang diterima secara real-time (live microphone)
2. $N(t)$ = noise/kebisingan (suara latar, gangguan, dll)
3. $X(t) - N(t)$ = sinyal audio yang telah difilter (noise reduction)
4. F_{θ} = model Whisper (*speech-to-text*)
5. T = Hasil transkripsi teks

3.6.3 Hallucination filter

Hallucination Filter adalah proses untuk mendeteksi dan menghapus teks hasil transkripsi Whisper yang tidak sesuai dengan audio asli (teks "khayalan" model), seperti kata berulang, frasa asing, atau kalimat yang tidak pernah diucapkan, biasanya berdasarkan skor kepercayaan (confidence score).

Variabel yang digunakan dalam proses ini direpresentasikan sebagai berikut:

$$T_{clean} = \{w_i \in T \mid P(w_i \mid X) \geq \tau\}$$

Keterangan

1. T = teks hasil transkripsi awal dari model Whisper
2. w_i = kata ke- i dalam teks transkripsi
3. $P(w_i \mid X)$ = probabilitas atau tingkat kepercayaan model terhadap kata w_i berdasarkan input audio X
4. τ = nilai ambang batas (*threshold*) untuk menentukan validitas kata
5. T_{clean} = teks hasil setelah dilakukan proses penyaringan

3.6.4 Algoritma Lexrank (Extractive Summarization)

LexRank adalah algoritma berbasis graf yang mengidentifikasi kalimat penting berdasarkan sentralitas leksikal dalam dokumen. Algoritma ini terdiri dari beberapa tahap pemrosesan:

Tahap 1: Deduplikasi kalimat dengan jaccard similiraty

$$J(A,B) = |A \cap B| / |A \cup B|$$

Keterangan:

1. A, B = Himpunan kata unik dari dua kalimat
2. $|A \cap B|$ = Jumlah kata irisan (kesamaan)
3. $|A \cup B|$ = Jumlah kata gabungan (union)
4. Threshold: 0.65 (duplikat jika $J > 0.65$)
5. Output: Daftar kalimat unik tanpa redundansi

Penjelasan: Tahap deduplikasi membuang kalimat-kalimat yang sangat mirip (similarity > 0.65) dari transkripsi untuk menghilangkan redundansi sebelum proses ekstraksi.

Tahap 2: Representasi vektor dengan TF-IDF

- Sublinear TF: $TF(t, d) = 1 + \log(f(t, d))$
- Smooth IDF: $IDF(t) = \log\left(\frac{|D|+1}{|\{d \in D: t \in d\}|+1}\right) + 1$
- Normalisasi L2: $v_{\text{normalized}} = \frac{v}{|v|_2}$

Keterangan:

1. $f(t, d)$ Frekuensi kata t dalam dokumen d
2. $|D|$ Total jumlah dokumen dalam corpus
3. $|\{d \in D: t \in d\}|$ Jumlah dokumen yang mengandung kata t
4. $|v|_2$ Euclidean norm (L2 normalization)
5. Output: Representasi vektor TF-IDF untuk setiap kalimat

Penjelasan: Sublinear TF mencegah kata yang sering muncul mendominasi representasi. Smooth IDF menghindari pembagian dengan nol. Normalisasi L2 memastikan panjang dokumen tidak mempengaruhi perhitungan kemiripan.

Tahap 3: Perhitungan cosine similarity antar kalimat

$$\cos(d_i, d_j) = \frac{d_i \cdot d_j}{|d_i| \times |d_j|}$$

Keterangan:

1. d_i, d_j = Vektor TF-IDF dari kalimat i dan j
2. $d_i \cdot d_j$ = Dot product antara dua vektor
3. $|d_i|, |d_j|$ = Magnitude (panjang) vektor
4. Rentang nilai: $[0, 1]$
5. Output: Matriks similarity M ($N \times N$) untuk N kalimat

Penjelasan: Cosine similarity mengukur kesamaan semantik antar kalimat berdasarkan representasi TF-IDF. Nilai 1 berarti kalimat identik, nilai 0 berarti tidak ada kesamaan.

Tahap 4: Power Iteration (PageRank Like)

$$p(u) = \frac{d}{N} + (1 - d) \times \sum_{v \in adj(u)} \frac{sim(u, v)}{deg(v)} \times p(v)$$

Keterangan:

1. d = Damping factor (0.85, menentukan probabilitas random walk)
2. N = Total jumlah kalimat dalam dokumen
3. $sim(u, v)$ = Cosine similarity antara kalimat u dan v
4. $deg(v)$ = Out-degree dari node v (jumlah bobot sisi terhubung)
5. $adj(u)$ = Himpunan node tetangga u dalam graf
6. Iterasi: Dilakukan hingga konvergensi (biasanya 20-30 iterasi)
7. Output: Skor sentralitas $p(u)$ untuk setiap kalimat u

Penjelasan: Power iteration menghitung skor sentralitas setiap kalimat dalam graf berdasarkan importance kalimat-kalimat tetangga. Kalimat dengan skor tinggi berarti kalimat tersebut bersifat sentral dalam diskusi.

Tahap 5: Seleksi kalimat penting

Strategi: Pilih K kalimat dengan skor $p(u)$ tertinggi

1. K = 30-40% dari total kalimat N (dapat disesuaikan)
2. Pengurutan: Jaga urutan kalimat asli dalam dokumen (preservasi konteks)
3. Output: Daftar K kalimat penting untuk tahap abstractive

Penjelasan: Kalimat dengan skor power iteration tertinggi dianggap paling representatif dan sentral. Persentase 30-40% dipilih untuk menyeimbangkan antara kompresi informasi dan pelestarian detail penting.



Gambar 1.20 Arsitektur Lexrank

3.6.5 Llama 3.2 (Abstractive Summarization)

Llama 3.2 adalah Large Language Model yang dijalankan secara lokal melalui platform Ollama untuk melakukan abstractive rewriting kalimat-kalimat hasil ekstraksi LexRank.

Spesifikasi model:

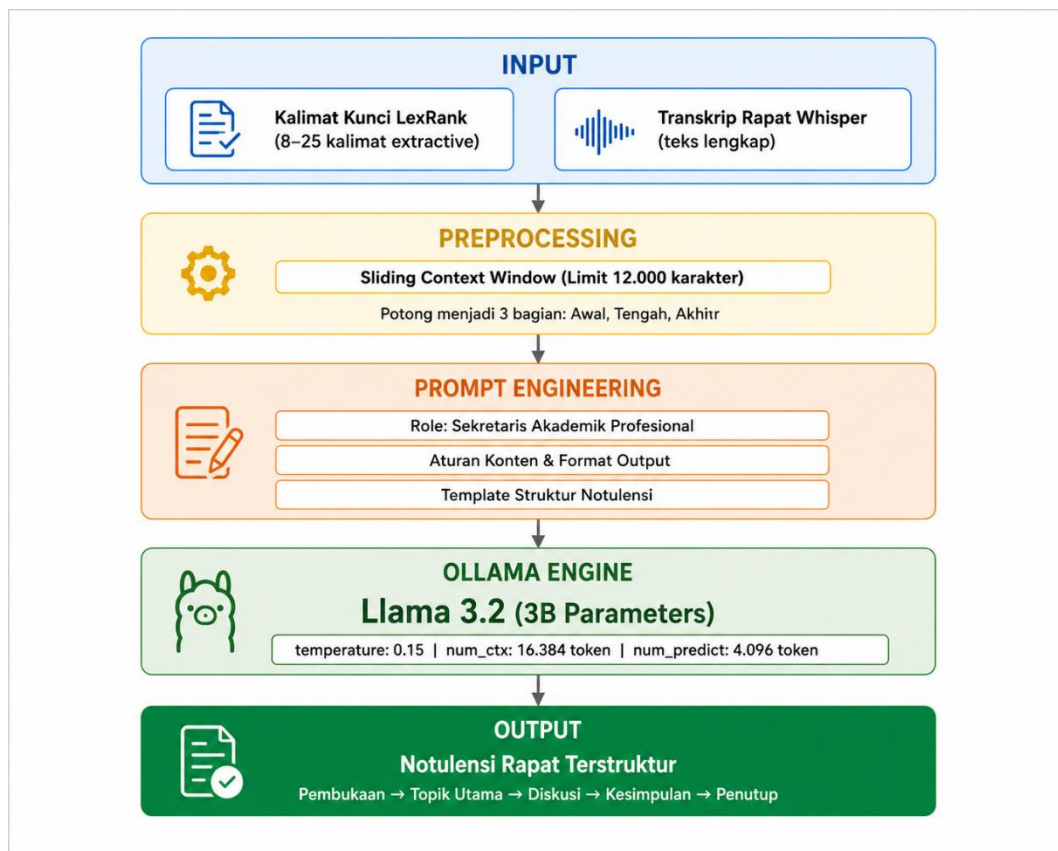
1. Model: Llama 3.2 (Instruct variant)
2. Platform: Ollama (inferensi lokal)
3. Variant: 3B-parameter (efisien untuk dijalankan secara lokal pada komputer/laptop standar)
4. Mode: Instruction-following dengan prompt engineering

Parameter generasi

1. Temperature: 0.15 → lebih rendah untuk determinisme tinggi agar format notulen konsisten
2. Max Tokens: 4096 → mencegah teks terpotong pada rapat panjang
3. Top-P: 0.9 → sesuai
4. Repetition Penalty: 1.15 → mengurangi pengulangan kata

Proses Abstractive Rewriting:

Input “K kalimat hasil ekstraksi Lexrank”



Gambar 1.21 Arsitektur llama

Arsitektur *Abstractive Summarization* menggunakan Llama 3.2 dirancang untuk mengubah hasil ringkasan ekstraktif dari LexRank menjadi notulensi rapat yang lebih terstruktur dan mudah dipahami. Proses dimulai dengan masukan berupa kalimat-kalimat kunci hasil ekstraksi LexRank serta transkrip rapat lengkap dari Whisper. Selanjutnya, dilakukan tahap *preprocessing* menggunakan *sliding context window* untuk mengelola panjang konteks agar tetap sesuai dengan kapasitas model. Data yang telah diproses kemudian dikombinasikan dengan *prompt engineering* yang berisi peran model, aturan penulisan, dan format notulensi yang diinginkan. Seluruh informasi tersebut diproses oleh Llama 3.2 melalui Ollama untuk menghasilkan ringkasan abstraktif yang lebih koheren dan terstruktur. Hasil akhir berupa notulensi rapat yang tersusun secara sistematis mulai dari pembukaan, topik utama, diskusi, kesimpulan, hingga penutup, sehingga memudahkan pengguna dalam memahami hasil rapat secara ringkas dan jelas.

3.8 Teknik pengujian

3.7.1 Evaluasi Transkripsi Whisper

Evaluasi akurasi transkripsi dilakukan menggunakan metrik berbasis error rate yang umum digunakan pada sistem Automatic Speech Recognition (ASR).

1. Word Error Rate (WER)

$$WER = \frac{S + D + I}{N}$$

Keterangan:

- S = Substitusi (kata salah)
- D = Deletion (kata hilang)
- I = Insertion (kata tambahan)
- N = Jumlah kata pada referensi

Semakin kecil nilai WER, semakin baik akurasi transkripsi.

2. Character Error Rate (CER)

$$CER = \frac{S_c + D_c + I_c}{N_c}$$

CER mirip dengan WER tetapi dihitung pada tingkat karakter. Cocok digunakan untuk bahasa dengan variasi penulisan atau kesalahan kecil pada kata.

3. Word Accuracy (WAcc)

$$WAcc = 1 - WER$$

Word Accuracy merupakan kebalikan dari WER yang menunjukkan tingkat akurasi dalam bentuk positif. Nilai mendekati 1 (atau 100%) menunjukkan hasil transkripsi yang sangat akurat.

Setelah nilai *Word Accuracy* diperoleh dari masing-masing data uji, dilakukan perhitungan rata-rata akurasi untuk mengetahui tingkat kinerja sistem transkripsi secara keseluruhan. Rata-rata akurasi dihitung dengan menjumlahkan seluruh nilai *Word Accuracy* dari setiap data uji, kemudian membaginya dengan

jumlah data uji yang digunakan. Persamaan rata-rata akurasi dapat dinyatakan sebagai berikut.

$$\overline{WAcc} = \frac{\sum_{i=1}^n WAcc_i}{n}$$

Keterangan:

1. $\overline{WAcc}$ = rata-rata *Word Accuracy*,
2. $WAcc_i$ = nilai *Word Accuracy* pada data uji ke-iii,
3. n = jumlah data uji.

Nilai n menyatakan jumlah data uji yang digunakan dalam evaluasi sistem. Rata-rata *Word Accuracy* digunakan untuk menggambarkan tingkat akurasi transkripsi secara keseluruhan, di mana semakin tinggi nilainya, semakin baik kinerja sistem.

3.7.2 Evaluasi LexRank

Evaluasi pada penelitian ini dilakukan terhadap tiga aspek utama, yaitu kualitas hasil transkripsi, tingkat kemiripan semantik antara hasil ringkasan dan ringkasan referensi, serta tingkat kompresi ringkasan yang dihasilkan. Metrik yang digunakan meliputi *Word Error Rate (WER)* untuk evaluasi transkripsi, *Cosine Similarity* untuk mengukur kesamaan isi ringkasan, dan *Compression Ratio* untuk mengetahui tingkat pemadatan teks.

1. *Cosine Similarity* (TF-IDF)

Cosine Similarity digunakan untuk mengukur tingkat kemiripan semantik antara ringkasan hasil sistem dengan ringkasan referensi berdasarkan representasi bobot *Term Frequency–Inverse Document Frequency (TF-IDF)*. Nilai *Cosine Similarity* berada pada rentang 0 hingga 1. Semakin mendekati 1, semakin tinggi tingkat kesamaan kedua ringkasan.

$$\text{Cosine Similarity} = \frac{A \cdot B}{|A| \times |B|}$$

Keterangan:

- A : vektor TF-IDF dari ringkasan hasil sistem,
- B : vektor TF-IDF dari ringkasan referensi,
- $A \cdot B$: hasil perkalian titik (*dot product*) antara kedua vektor,
- $\|A\|$ dan $\|B\|$: norma atau panjang masing-masing vektor.

Nilai Cosine Similarity yang tinggi menunjukkan bahwa kedua ringkasan memiliki representasi kata yang serupa sehingga informasi yang disampaikan cenderung sama.

2. Sentence Coverage

Sentence Coverage digunakan untuk mengukur persentase informasi penting dari ringkasan referensi yang berhasil tercakup dalam ringkasan sistem.

$$[\text{Sentence Coverage} = \frac{\text{Jumlah kalimat referensi yang tercakup}}{\text{Jumlah total kalimat referensi}} \times 100\%$$

Semakin tinggi nilai Sentence Coverage, semakin banyak informasi penting yang berhasil dipertahankan.

3. Jaccard Similarity

Jaccard Similarity digunakan untuk mengukur tingkat kemiripan kata unik antara ringkasan sistem dan ringkasan referensi.

$$[\text{Jaccard Similarity} = \frac{|A \cap B|}{|A \cup B|}$$

Keterangan:

A = himpunan kata pada ringkasan sistem,

B = himpunan kata pada ringkasan referensi.

Semakin mendekati 1, semakin tinggi tingkat kemiripan kedua ringkasan.

Setelah nilai masing-masing metrik diperoleh pada setiap data uji, dilakukan perhitungan rata-rata untuk menggambarkan performa sistem secara keseluruhan. Rata-rata nilai setiap metrik dihitung dengan menjumlahkan seluruh nilai hasil pengujian, kemudian membaginya dengan jumlah data uji yang digunakan. Persamaan rata-rata evaluasi dapat dinyatakan sebagai berikut.

$$\overline{M} = \frac{\sum_{i=1}^n M_i}{n}$$

Keterangan:

- a. $\overline{M}$ = nilai rata-rata metrik evaluasi,
- b. M_i = nilai metrik pada data uji ke- i ,
- c. n = jumlah data uji.

Apabila diterapkan pada masing-masing metrik evaluasi ringkasan, maka diperoleh persamaan sebagai berikut.

1. Rata rata Cosine Similarity

$$\overline{CS} = \frac{\sum_{i=1}^n CS_i}{n}$$

Keterangan:

- a. $\overline{CS}$ = rata-rata *Cosine Similarity*,
- b. CS_i = nilai *Cosine Similarity* pada data uji ke- i ,
- c. n = jumlah data uji.

2. Rata-rata Sentence Coverage

$$\overline{SC} = \frac{\sum_{i=1}^n SC_i}{n}$$

Keterangan:

- a. $\overline{SC}$ = rata-rata *Sentence Coverage*,
- b. SC_i = nilai *Sentence Coverage* pada data uji ke-iii,
- c. n = jumlah data uji.

3. Rata-rata Jaccard Similarity

$$\overline{JS} = \frac{\sum_{i=1}^n JS_i}{n}$$

Keterangan:

- a. $\overline{JS}$ = rata-rata *Jaccard Similarity*,
- b. JS_i = nilai *Jaccard Similarity* pada data uji ke-iii,
- c. n = jumlah data uji.

Nilai rata-rata dari masing-masing metrik digunakan untuk merepresentasikan kualitas ringkasan yang dihasilkan sistem secara keseluruhan. Semakin tinggi nilai rata-rata *Cosine Similarity*, *Sentence Coverage*, dan *Jaccard Similarity*, maka semakin baik kemampuan sistem dalam menghasilkan ringkasan yang memiliki kesamaan informasi dengan ringkasan referensi.

3.7.3 Evaluasi LLM (Ringkasan Abstraktif)

Model LLM (seperti Llama 3.2), evaluasi tetap menggunakan metrik ROUGE karena output berupa teks ringkasan.

1. Precision (LLM)

$$\text{Precision}_{LLM} = \frac{\text{Overlap dengan referensi}}{\text{Total kata output LLM}}$$

Menunjukkan ketepatan informasi yang dihasilkan oleh LLM dalam ringkasan abstraktif.

2. Recall (LLM)

$$\text{Recall}_{LLM} = \frac{\text{Overlap dengan referensi}}{\text{Total kata referensi}}$$

Mengukur kemampuan LLM dalam menangkap informasi penting dari referensi.

3. F1-Score (LLM)

$$F_{LLM} = 2 \times \frac{P \times R}{P + R}$$

Menggabungkan precision dan recall untuk memberikan evaluasi keseluruhan kualitas ringkasan LLM.

Setelah nilai *Precision*, *Recall*, dan *F1-Score* diperoleh pada setiap data uji, dilakukan perhitungan rata-rata untuk mengetahui performa keseluruhan sistem. Rata-rata masing-masing metrik dihitung menggunakan persamaan berikut.

1. Rata-rata Precision

$$\overline{P} = \frac{\sum_{i=1}^n P_i}{n}$$

Keterangan:

- $\overline{P}$ = rata-rata *Precision*,
- P_i = nilai *Precision* pada data uji ke-*i*,
- n = jumlah data uji.

2. Rata-rata Recall

$$\overline{R} = \frac{\sum_{i=1}^n R_i}{n}$$

Keterangan:

- a. $\overline{R}$ = rata-rata *Recall*,
- b. R_i = nilai *Recall* pada data uji ke-*i*,
- c. n = jumlah data uji.

3. Rata-rata F1-Score

$$\overline{F_1} = \frac{\sum_{i=1}^n F_{1i}}{n}$$

Keterangan:

- a. $\overline{F_1}$ = rata-rata *F1-Score*,
- b. F_{1i} = nilai *F1-Score* pada data uji ke-*i*,
- c. n = jumlah data uji.

Nilai rata-rata *Precision*, *Recall*, dan *F1-Score* digunakan untuk merepresentasikan kualitas ringkasan abstraktif yang dihasilkan oleh LLM secara keseluruhan. Semakin tinggi nilai ketiga metrik tersebut, semakin baik kemampuan model dalam menghasilkan ringkasan yang relevan dan sesuai dengan ringkasan referensi.

3.7.4 Skenario pengujian Blackbox (Fungsional)

Pengujian *Black Box Testing* dilakukan untuk memastikan bahwa setiap fungsi pada aplikasi notulensi rapat otomatis berjalan sesuai dengan kebutuhan yang telah ditentukan. Pengujian ini berfokus pada kesesuaian antara masukan yang diberikan dengan keluaran yang dihasilkan tanpa memperhatikan struktur kode program.

1. Modul Login

Tabel 3.4 Skenario Pengujian Modul *Login*

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan
1	Login dengan data valid	Buka halaman login → isi email dan password → klik Login	Email: Admin@admin.com, Password: admin123	Berhasil masuk ke halaman Dashboard, muncul pesan "Selamat datang"
2	Login dengan password salah	Buka halaman login → isi email benar, password salah → klik Login	Email: Admin@admin.com, Password: salah123	Tetap di halaman login, muncul pesan error "Email atau Password salah!"
3	Login dengan email tidak terdaftar	Buka halaman login → isi email yang tidak ada di database → klik Login	Email: tidak@ada.com, Password: test123	Tetap di halaman login, muncul pesan error "Email atau Password salah!"
4	Login dengan field kosong	Buka halaman login → biarkan email dan password kosong → klik Login	Email: (kosong), Password: (kosong)	Tetap di halaman login, muncul pesan error "Email dan Password harus diisi!"
5	Akses halaman tanpa login	Langsung buka URL dashboard.php tanpa login	(tidak ada)	Di-redirect ke halaman login

2. Modul Dashboard

Tabel 3.5 Pengujian Modul Dashboard

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan
1	Menampilkan statistik dashboard	Login sebagai Admin → buka Dashboard	(akun Admin)	Menampilkan statistik jumlah rapat, chart mingguan, dan daftar rapat terbaru
2	Akses dashboard sebagai User biasa	Login sebagai User (level 2) → buka Dashboard	(akun User)	Dashboard tampil dengan data sesuai hak akses User

3. Modul Live Transcription

Tabel 3.6 Pengujian Modul Live Transcription

No	Skenario Pengujian	Prosedur Pengujian	<i>Test Data</i>	Hasil yang Diharapkan
1	Memulai live transcription	Isi nama rapat → klik "Start Listening" → izinkan akses mikrofon → bicara	Nama rapat: "Rapat Sprint Planning", Input suara: kalimat Bahasa Indonesia	Status berubah ke "Listening", teks transkripsi muncul secara real-time di layar
2	Menghentikan live transcription	Setelah live berjalan → klik "End Listening"	(live sedang berjalan)	Status berubah ke "Stopped", teks final

				ditampilkan, file .txt tersimpan
3	Menyimpan notulensi live	Setelah stop listening → klik "Simpan Notulen"	Teks transkripsi hasil live	Data tersimpan ke database, ringkasan otomatis diproses, panel ringkasan muncul
4	Live transcription tanpa server Python	Klik "Start Listening" saat server Python (port 5000) tidak berjalan	(server offline)	Muncul pesan error bahwa server tidak tersedia
5	Reset transcription	Setelah transkripsi selesai → klik "Reset"	(ada teks transkripsi)	Area transkripsi dikosongkan, siap untuk sesi baru
6	Akses halaman notulensi sebagai User biasa	Login sebagai User (level 2) → buka halaman Notulensi	(akun User)	Di-redirect ke Dashboard (hanya Admin yang boleh akses)

4. Modul Upload Video

Tabel 3.7 Pengujian Modul Upload File

No	Skenario Pengujian	Prosedur Pengujian	Test Data	Hasil yang Diharapkan
----	--------------------	--------------------	-----------	-----------------------

1	Upload file video valid	Drag & drop atau browse file MP4 → isi judul rapat → klik "Upload & Transkripsi"	File: rapat.mp4 (50MB), Judul: "Rapat Divisi IT"	Progress bar muncul, file dikirim ke server, status "Proses Berjalan" tampil
2	Upload file format tidak didukung	Pilih file dengan format tidak valid → klik Upload	File: dokumen.pdf	Muncul pesan error "Format tidak didukung"
3	Upload tanpa memilih file	Klik "Upload & Transkripsi" tanpa memilih file	(tidak ada file)	Tombol Upload tetap disabled, tidak bisa diklik
4	Upload tanpa judul rapat	Pilih file video → biarkan judul kosong → klik Upload	File: rapat.mp4, Judul: (kosong)	Upload berhasil, judul otomatis diambil dari nama file video
5	Melihat progress proses background	Setelah upload → lihat card "Proses Berjalan"	(proses sedang berjalan)	Card menampilkan status fase (Mentranskripsi/Meringkas) dengan persentase progress

5. Modul Report

Tabel 3.8 Pengujian Modul Report

No	Skenario Pengujian	Prosedur Pengujian	Test Data	Hasil yang Diharapkan
----	--------------------	--------------------	-----------	-----------------------

1	Melihat daftar hasil rapat	Buka halaman Report	(data tersedia di database)	Tabel menampilkan semua hasil rapat dengan judul, waktu, tipe (Live/Upload), dan tombol Detail
2	Filter report berdasarkan tipe	Klik filter "Live" atau "Upload"	(data tersedia)	Tabel hanya menampilkan hasil sesuai filter yang dipilih
3	Melihat detail ringkasan rapat	Klik tombol "Detail" pada salah satu hasil rapat	(ringkasan tersedia)	Modal muncul menampilkan ringkasan terstruktur (Pembukaan, Topik, Kesimpulan, Penutup)
4	Download notulensi sebagai PDF	Klik "Unduh Notulensi (.pdf)" di modal detail	(ringkasan tersedia)	File PDF ter-download dengan format notulensi yang rapi

6. Modul Pengaturan

Tabel 3.9 Pengujian Modul Pengaturan

No	Skenario Pengujian	Prosedur Pengujian	<i>Data test</i>	Hasil yang Diharapkan
1	Tambah user baru (Admin)	Login Admin → Pengaturan → isi form Tambah User	1	Tambah user baru (Admin)

		→ klik Tambah		
2	Ubah password sendiri	Klik "Ubah Password" → isi password baru → klik Simpan	2	Ubah password sendiri
3	Hapus user (Admin)	Login Admin → Pengaturan → klik icon hapus pada user lain → konfirmasi	3	Hapus user (Admin)

3.7.5 Skenario Pengujian Whitebox (Struktural)

Pengujian *White Box Testing* dilakukan untuk mengevaluasi logika dan alur program pada aplikasi notulensi rapat otomatis. Pengujian ini berfokus pada struktur internal kode untuk memastikan setiap jalur eksekusi program dapat berjalan dengan baik.

1. Proses Login

Tabel 7. Rencana Skenario Pengujian White-Box — B.1 Proses Login
(proses_login.php)

No	Skenario Pengujian	<i>Data Test</i>	Hasil yang Diharapkan
1	Jalur: sudah login → redirect	Session sudah aktif	Langsung redirect ke dashboard.php, proses login tidak dijalankan

2	Jalur: method bukan POST → redirect	Akses langsung via GET	Redirect ke login.php
3	Jalur: input kosong → error	Email: "", Password: ""	Set session error, redirect ke login.php
4	Jalur: email ditemukan + password bcrypt valid	Email ada, password cocok dengan hash bcrypt	Set session (id, nama, email, level), redirect ke dashboard.php
5	Jalur: email ditemukan + password plain text → auto-upgrade	Email ada, password masih plain text di DB	Login berhasil + password di-upgrade ke bcrypt hash secara otomatis
6	Jalur: email ditemukan + password salah	Email ada, password tidak cocok	Set session error "Email atau Password salah!", redirect ke login.php
7	Jalur: email tidak ditemukan	Email tidak ada di database	Set session error "Email atau Password salah!", redirect ke login.php

2. Validasi Upload File (server.py — api_transcribe)

Tabel 8. Skenario Pengujian White-Box — B.2 Validasi Upload File
(server.py — api_transcribe)

No	Skenario Pengujian	<i>Data test</i>	Hasil yang Diharapkan
1	Jalur: field video_file tidak ada	POST tanpa field video_file	Return JSON error 400: "Field 'video_file' tidak ditemukan"

2	Jalur: filename kosong	Field ada tapi tidak ada file terpilih	Return JSON error 400: "Tidak ada file yang dipilih"
3	Jalur: ekstensi tidak diizinkan	File: dokumen.exe	Return JSON error 400: "Format 'exe' tidak didukung"
4	Jalur: file valid → proses background	File: rapat.mp4	File disimpan, record DB dibuat, background thread dimulai, return JSON success

3. Voice Activity Detection/VAD (server.py_Process_audio)

Tabel 9. Skenario Pengujian White-Box — B.3 Voice Activity Detection / VAD (server.py — process_live_audio)

No	Skenario Pengujian	<i>Data Test</i>	Hasil yang Diharapkan
1	Jalur: buffer terlalu pendek → skip	Buffer audio 0.5 detik	Fungsi return tanpa proses, menunggu buffer lebih panjang
2	Jalur: RMS di bawah threshold → flush silence	Audio hening (RMS = 0.002)	current_text dipindahkan ke history_text, buffer di-trim
3	Jalur: RMS di atas threshold → proses Whisper	Audio ada suara (RMS = 0.05)	Buffer dikirim ke Whisper submit_and_wait(), hasil diproses

4	Jalur: buffer melebihi max → force flush	Buffer audio 25 detik	current_text → history_text, buffer direset ke kosong
---	--	-----------------------	---

4. Hallucination Filter (Hallucination Filter.py)

Tabel 10. Skenario Pengujian White-Box — B.4 Hallucination Filter
(hallucination_filter.py)

No	Skenario Pengujian	<i>Data test</i>	Hasil yang Diharapkan
1	Jalur: teks kosong → halusinasi	Teks: "terima kasih sudah menonton"	Return True (frasa ada di daftar halusinasi)
2	Jalur: exact match frasa halusinasi	Teks: "subscribe subscribe subscribe"	Return True (frasa muncul >2 kali = repetisi)
3	Jalur: frasa berulang >2 kali → halusinasi	Teks: "jadi kita harus jadi kita harus"	Return True (pengulangan terdeteksi)
4	Jalur: separuh pertama == separuh kedua → halusinasi	Teks: "Kita perlu menyelesaikan fitur dashboard minggu ini"	Return False (teks valid, tidak dibuang)
5	Jalur: teks normal → bukan halusinasi	Teks: "terima kasih sudah menonton"	Return True (frasa ada di daftar halusinasi)

5. File Transcription — Chunking & Loop (transcriber.py)

Tabel 11. Skenario Pengujian White-Box — B.5 File Transcription — Chunking & Loop (transcriber.py)

No	Skenario Pengujian	<i>Data Test</i>	Hasil yang Diharapkan
1	Jalur: chunk terlalu pendek → skip	Chunk audio < 0.5 detik	Chunk dilewati, lanjut ke chunk berikutnya
2	Jalur: chunk normal → transkripsi	Chunk audio 30 detik	model.transcribe() dipanggil, hasil ditambahkan ke <code>all_text_parts</code>
3	Jalur: chunk hasil = halusinasi → skip	Chunk menghasilkan "[music]"	Chunk dilewati, tidak masuk ke hasil akhir
4	Jalur: chunk hasil = duplikat → skip	Chunk menghasilkan teks identik dengan chunk sebelumnya	Chunk dilewati (mencegah teks berulang)

6. Lexrank Pipeline (summarize/Lexrank.py)

Tabel 12. Skenario Pengujian White-Box — B.6 LexRank Pipeline
(summarize/lexrank.py)

No	Skenario Pengujian	<i>Data Test</i>	Hasil yang Diharapkan
1	Jalur: teks terlalu pendek → return langsung	Teks: "Rapat selesai." (15 karakter)	Return teks apa adanya tanpa proses LexRank
2	Jalur: kalimat ≤ target → skip LexRank	Teks pendek menghasilkan 5 kalimat, target 8	Semua kalimat digunakan langsung tanpa scoring LexRank

3	Jalur: Ollama tersedia → abstractive rewriting	Ollama server berjalan di localhost:11434	Kalimat extractive dikirim ke LLM untuk ditulis ulang menjadi notulensi berstruktur
4	Jalur: Ollama tidak tersedia → fallback extractive	Ollama server tidak berjalan	Output format extractive: kalimat di-polish dan dikelompokkan per 3 kalimat per paragraf

7. Operasi Database (server.py)

Tabel 13. Skenario Pengujian White-Box — B.7 Operasi Database (server.py)

No	Skenario Pengujian	<i>Data Test</i>	Hasil yang Diharapkan
1	Jalur: insert DB berhasil	Data valid: judul, tipe, txt_path	Record baru tersimpan, return lastrowid > 0
2	Jalur: insert DB gagal → return 0	Database offline atau tabel tidak ada	Print error log, return 0 (tidak crash)

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Data Sistem

Implementasi data sistem pada penelitian ini dilakukan dengan mengumpulkan dan mengelola data rekaman rapat yang digunakan sebagai masukan dalam proses transkripsi dan peringkasan otomatis. Data yang diperoleh terdiri dari data hasil perekaman secara langsung (live recording) serta data hasil unggah berkas audio atau video (upload file). Seluruh data tersebut digunakan sebagai bahan pengujian sistem untuk mengevaluasi kemampuan proses transkripsi, preprocessing, dan peringkasan yang telah diimplementasikan. Rincian data yang digunakan pada masing-masing fitur dapat dilihat pada tabel berikut.

4.1.1 Data Pengujian Fitur Live Recording

4.1.2 Data Pengujian Fitur Upload File

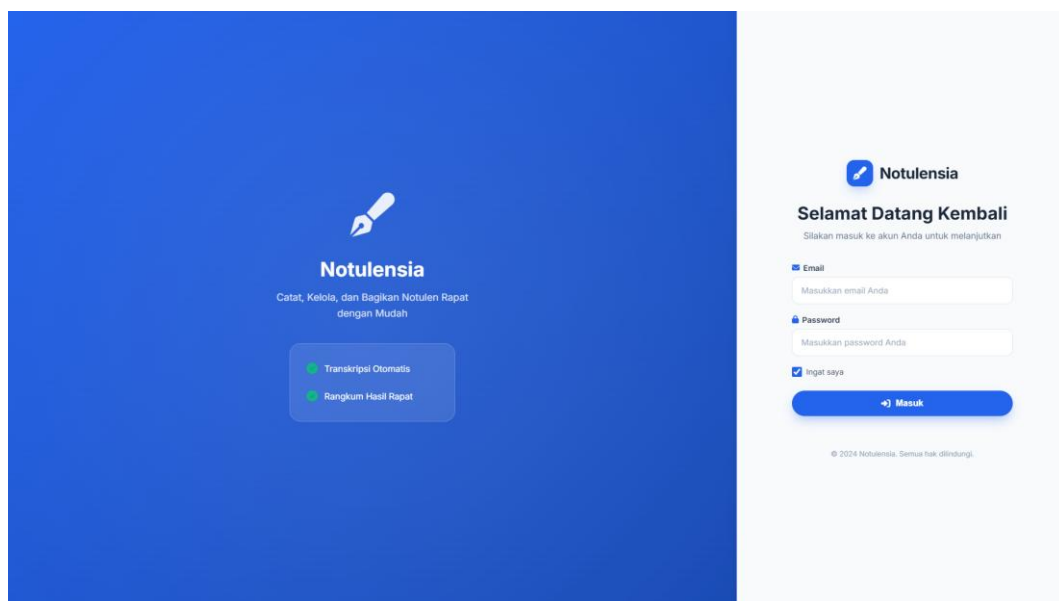
4.2 Implentasi Web application

Dashboard Sistem Notulensi Rapat Otomatis dikembangkan sebagai pusat kendali yang digunakan oleh Admin dan User untuk mengelola proses notulensi secara terpadu. Antarmuka ini dirancang berbasis web dengan memanfaatkan PHP sebagai teknologi sisi server, serta HTML, CSS, dan Vanilla JavaScript pada sisi klien. Dengan pendekatan tersebut, pengguna dapat mengakses seluruh fitur sistem melalui peramban web tanpa perlu melakukan instalasi aplikasi tambahan. Berbagai fitur yang tersedia pada dashboard telah terhubung dengan layanan backend AI melalui implementasi Socket.IO dan REST API sehingga memungkinkan pertukaran data secara real-time. Fungsionalitas yang disediakan meliputi transkripsi rapat secara langsung (live transcription), transkripsi melalui unggahan file rekaman (upload transcription), pengelolaan riwayat notulensi, penyajian

informasi terkait performa model yang digunakan, serta pengaturan sistem untuk mendukung kebutuhan operasional aplikasi secara menyeluruh.

1. Halaman login

Halaman login Merupakan sebagai sarana autentikasi pengguna untuk memastikan bahwa hanya Admin dan User yang memiliki hak akses yang dapat menggunakan fitur-fitur pada Sistem Notulensi Rapat Otomatis.

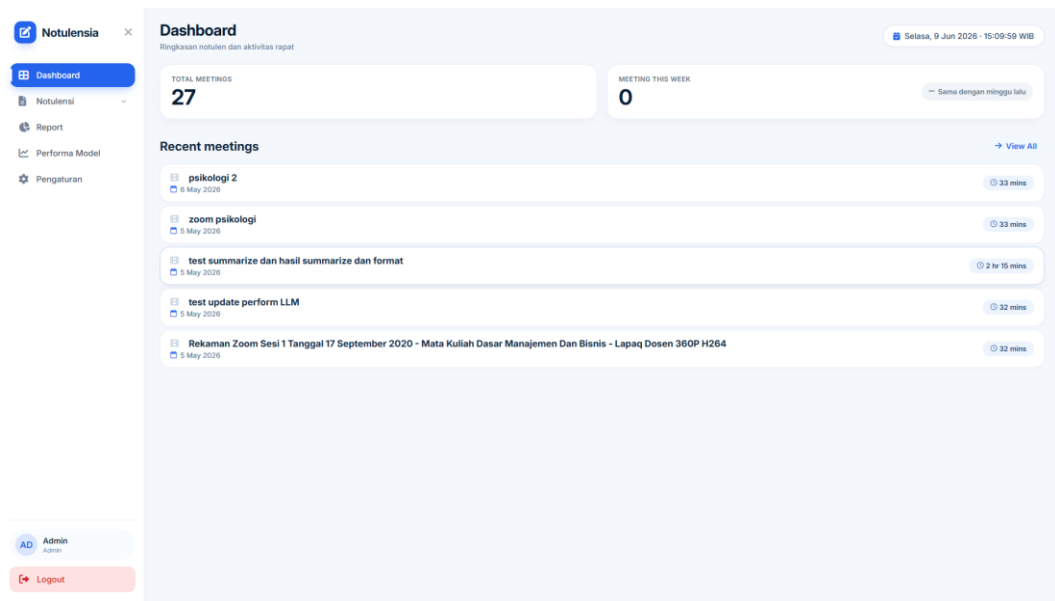


Gambar 4.1 Halaman login web

Gambar 4.1 menunjukkan halaman login yang berfungsi sebagai gerbang autentikasi pengguna sebelum mengakses Sistem Notulensi Rapat Otomatis. Setelah proses autentikasi berhasil, pengguna akan diarahkan ke dashboard untuk menggunakan fitur-fitur yang tersedia dalam sistem. Halaman ini juga menampilkan identitas aplikasi *Notulensia* beserta informasi singkat mengenai fungsi utama sistem, yaitu melakukan transkripsi otomatis dan menghasilkan ringkasan hasil rapat. Tampilan antarmuka dirancang sederhana dan mudah digunakan agar pengguna dapat mengakses sistem dengan nyaman.

2. Halaman Dashboard Utama

Halaman Dashboard utama merupakan halaman yang menampilkan informasi aktivitas sistem notulensi yang telah dilakukan.

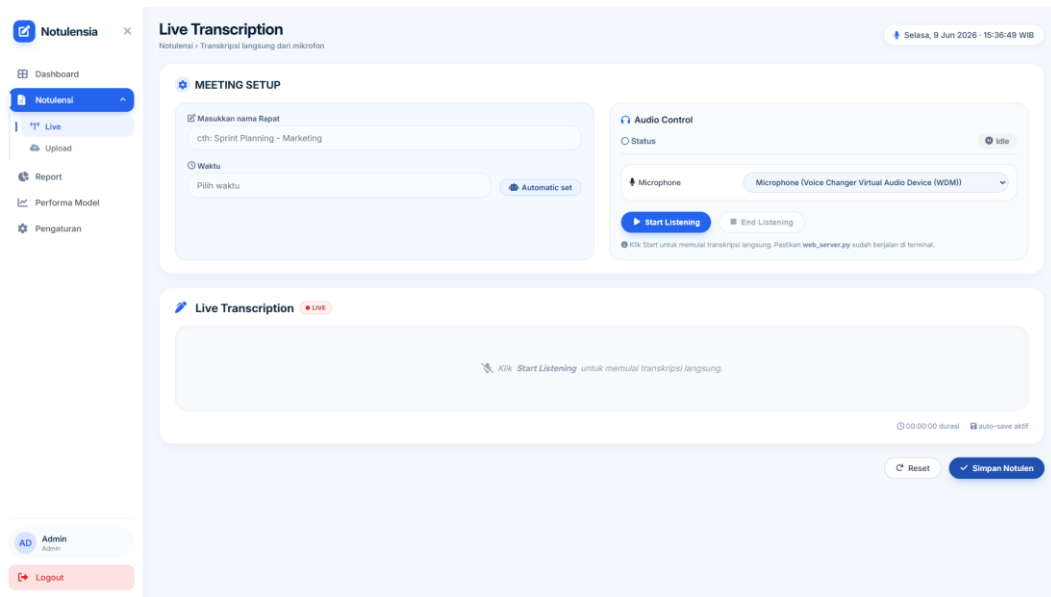


Gambar 4.2 Tampilan Halaman Dashboard Utama

Gambar 4.2 menunjukkan tampilan halaman dashboard yang menjadi pusat informasi dan navigasi utama pada Sistem Notulensi Rapat Otomatis. Halaman ini menyajikan ringkasan aktivitas rapat, seperti jumlah total rapat yang telah diproses serta jumlah rapat dalam periode tertentu. Pada bagian menu navigasi, pengguna dapat mengakses berbagai fitur sistem, meliputi proses notulensi, laporan hasil rapat, informasi performa model, dan pengaturan sistem. Selain itu, dashboard juga menampilkan daftar rapat terbaru beserta informasi singkat, seperti tanggal pelaksanaan dan durasi rapat, sehingga memudahkan pengguna dalam memantau serta mengelola hasil notulensi yang telah dibuat.

3. Halaman live Rapat

Halaman live rapat merupakan halaman yang berfungsi untuk menjalankan live rapat secara realtime dan merangkum hasil rapat yang dilakukan.

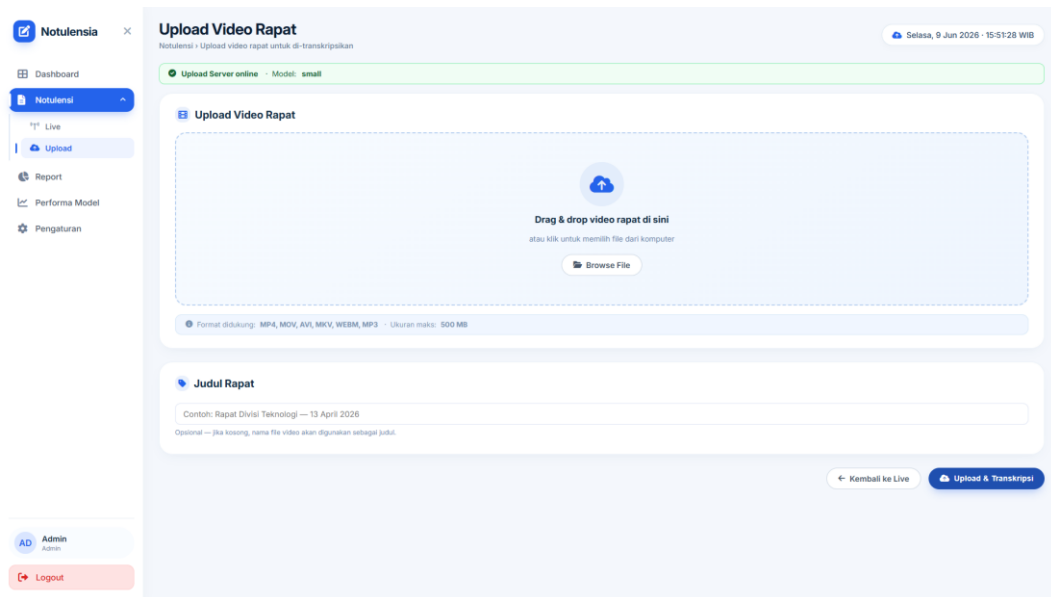


Gambar 4.3 Tampilan Halaman live Rapat

Gambar 4.3 menunjukkan halaman *Live Transcription* yang digunakan untuk melakukan transkripsi rapat secara langsung melalui mikrofon. Pada halaman ini, pengguna dapat mengisi informasi rapat, seperti nama rapat dan waktu pelaksanaan, sebelum memulai proses transkripsi. Selain itu, tersedia fitur pengaturan perangkat mikrofon serta tombol *Start Listening* dan *End Listening* untuk mengendalikan proses transkripsi secara real-time. Hasil transkripsi akan ditampilkan pada area yang telah disediakan sehingga pengguna dapat memantau jalannya proses notulensi. Setelah rapat selesai, pengguna dapat menggunakan tombol *Simpan Notulen* untuk menyimpan hasil transkripsi, sedangkan tombol *Reset* berfungsi untuk mengosongkan data transkripsi dan mengembalikan halaman ke kondisi awal sebelum memulai sesi transkripsi baru.

4. Halaman Upload Rapat

Halaman upload rapat merupakan fitur summarize rapat dengan cara mengupload video hasil rekaman rapat yang sudah di download atau disimpan dalam format mp4 atau mkv.

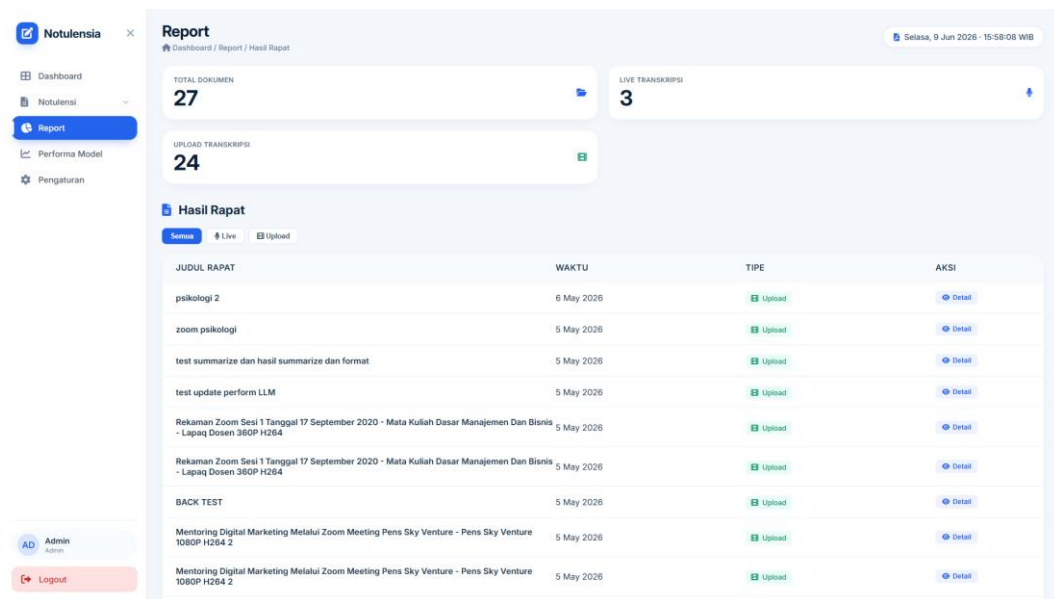


Gambar 4.4 Tampilan halaman upload rapat

Gambar 4.4 menunjukkan halaman *Upload Video Rapat* yang digunakan untuk melakukan proses transkripsi melalui berkas rekaman rapat yang telah tersedia. Pengguna dapat mengunggah file dengan cara *drag and drop* atau memilih file secara langsung dari perangkat melalui tombol *Browse File*. Pada halaman ini juga ditampilkan informasi mengenai format file yang didukung beserta batas maksimum ukuran file yang dapat diproses oleh sistem. Selain itu, pengguna dapat menambahkan judul rapat untuk memudahkan pengelolaan hasil notulensi. Setelah seluruh data telah sesuai, tombol *Upload & Transkripsi* dapat digunakan untuk memulai proses transkripsi secara otomatis, sedangkan tombol *Kembali ke Live* berfungsi untuk berpindah ke halaman transkripsi langsung.

5. Halaman Report

Halaman report merupakan halaman yang memuat informasi hasil rangkuman rapat yang telah di proses oleh sistem notulemsi

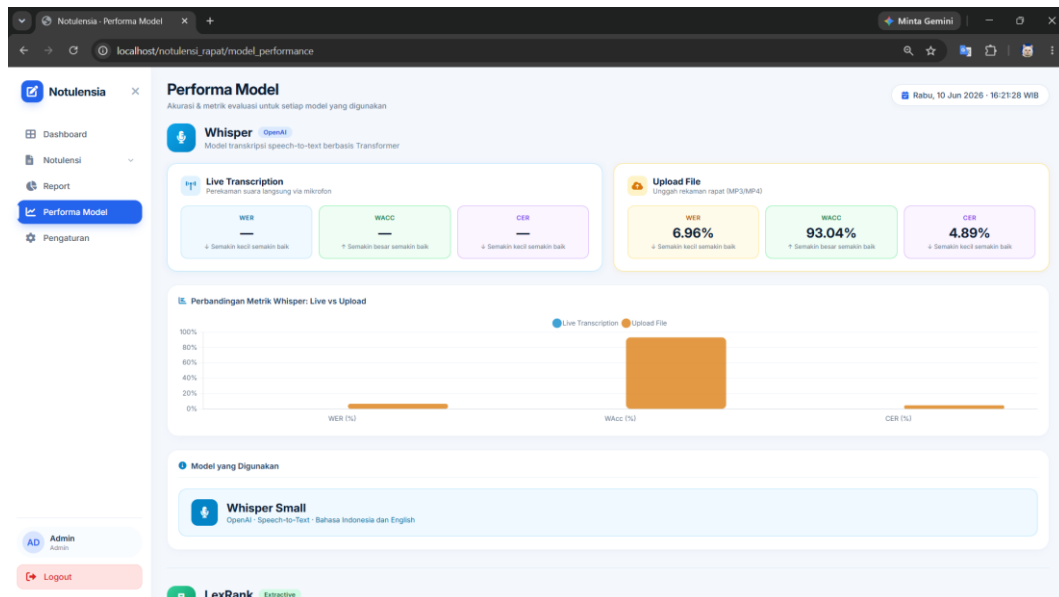


Gambar 4.5 Tampilan Halaman Report Sistem Notulensia

Halaman *Report* berfungsi untuk mengelola dan menampilkan riwayat hasil notulensi rapat yang telah diproses oleh sistem. Pada halaman ini, pengguna dapat melihat informasi statistik berupa jumlah total dokumen, jumlah hasil transkripsi langsung (*live transcription*), serta jumlah hasil transkripsi melalui unggahan berkas (*upload transcription*). Selain itu, halaman ini menyediakan daftar hasil rapat yang dilengkapi dengan informasi judul rapat, waktu pelaksanaan, dan jenis transkripsi yang digunakan. Fitur penyaringan berdasarkan kategori tertentu juga disediakan untuk memudahkan pengguna dalam menemukan data yang dibutuhkan. Pada setiap data notulensi, tombol *Detail* dapat digunakan untuk melihat informasi hasil rapat secara lebih lengkap.

6. Halaman Performa model

Halaman ini menampilkan performa dari model yang digunakan sejauh mana model berfungsi dengan baik serta nilai akurasi yang dihasilkan dari pengujian model whisper, lextank, dan ollama pada fitur live rapat dan fitur upload file rapat.

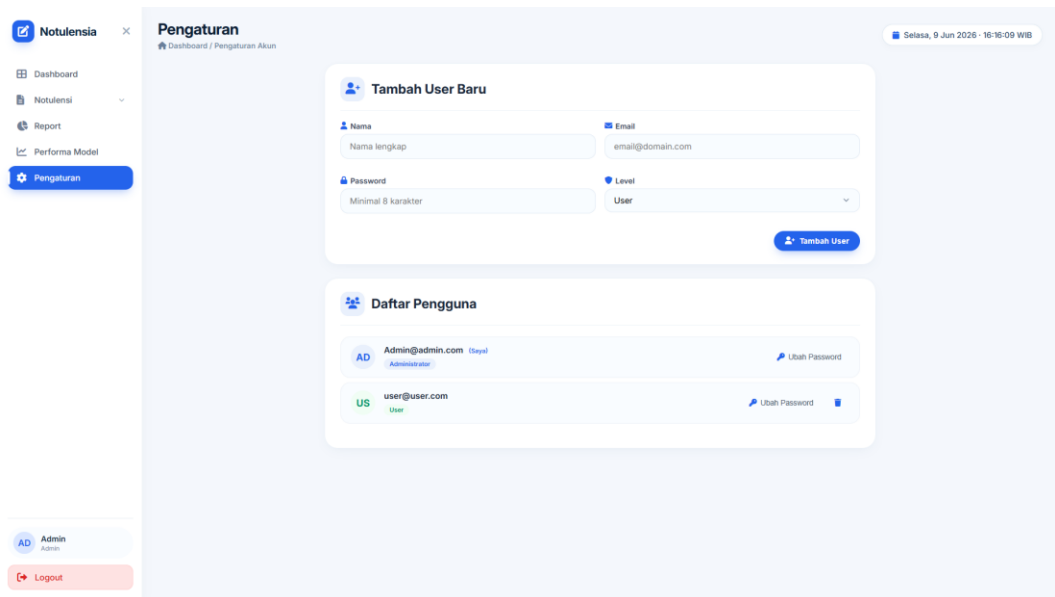


Gambar 4.6 Tampilan Halaman performa model sistem

Halaman *Performa Model* berfungsi untuk menyajikan hasil evaluasi terhadap model kecerdasan buatan yang digunakan pada sistem notulensi rapat otomatis. Pengguna dapat melihat tingkat akurasi model *Whisper* pada proses transkripsi, baik untuk *live transcription* maupun *upload transcription*, berdasarkan metrik evaluasi seperti *Word Error Rate (WER)*, *Word Accuracy (WAcc)*, dan *Character Error Rate (CER)*. Selain evaluasi model transkripsi, halaman ini juga menampilkan informasi performa model *LexRank* dan *Large Language Model (LLM)* yang digunakan dalam proses peringkasan hasil rapat. Visualisasi dalam bentuk grafik disediakan untuk mempermudah pengguna dalam membandingkan hasil evaluasi antara metode transkripsi langsung dan transkripsi melalui unggahan berkas. Informasi mengenai model yang digunakan turut ditampilkan sebagai referensi terhadap teknologi yang diterapkan pada sistem.

7. Halaman Pengaturan Sistem

Halaman pengaturan sistem merupakan halaman yang memuat informasi akun dari sistem notulensi yang memiliki fitur *add user*, *delete user*, dan *edit password*

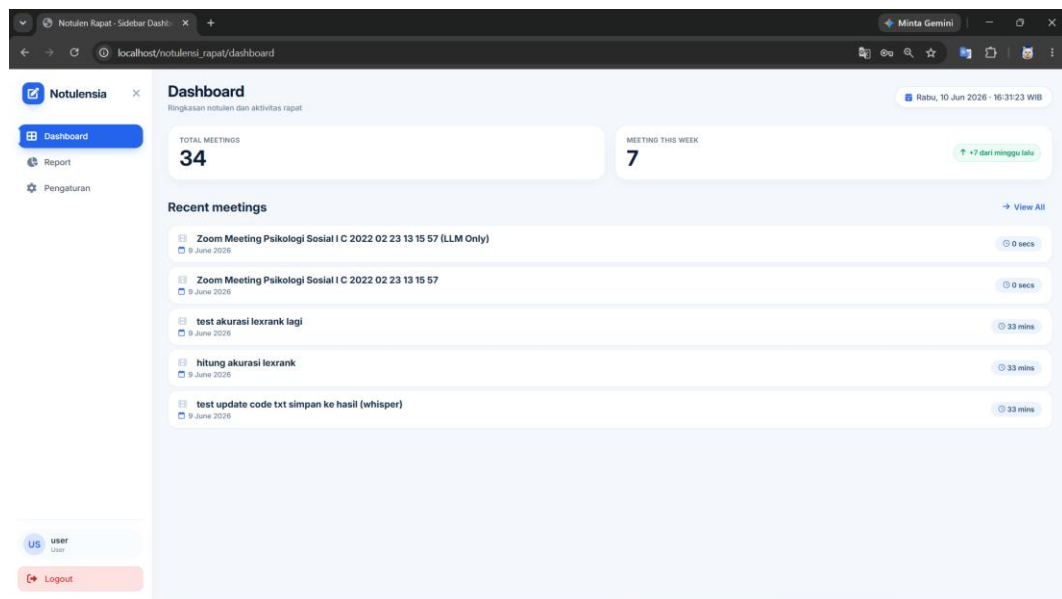


Gambar 4.7 Tampilan Halaman pengaturan

Halaman *Pengaturan* berfungsi untuk mengelola akun pengguna yang dapat mengakses Sistem Notulensi Rapat Otomatis. Pada halaman ini, administrator dapat menambahkan pengguna baru dengan mengisi informasi yang diperlukan, seperti nama, alamat email, kata sandi, dan level hak akses pengguna. Selain itu, halaman ini juga menyediakan daftar pengguna yang telah terdaftar dalam sistem. Administrator dapat melakukan pengelolaan akun, seperti mengubah kata sandi pengguna serta menghapus akun tertentu sesuai dengan kebutuhan pengelolaan sistem.

8. Halaman Dashboard User

Halaman dashboard user merupakan halaman khusus untuk user sistem notulensi yang memuat informasi hasil rapat atau hasil rangkuman rapat yang telah selesai dilakukan.

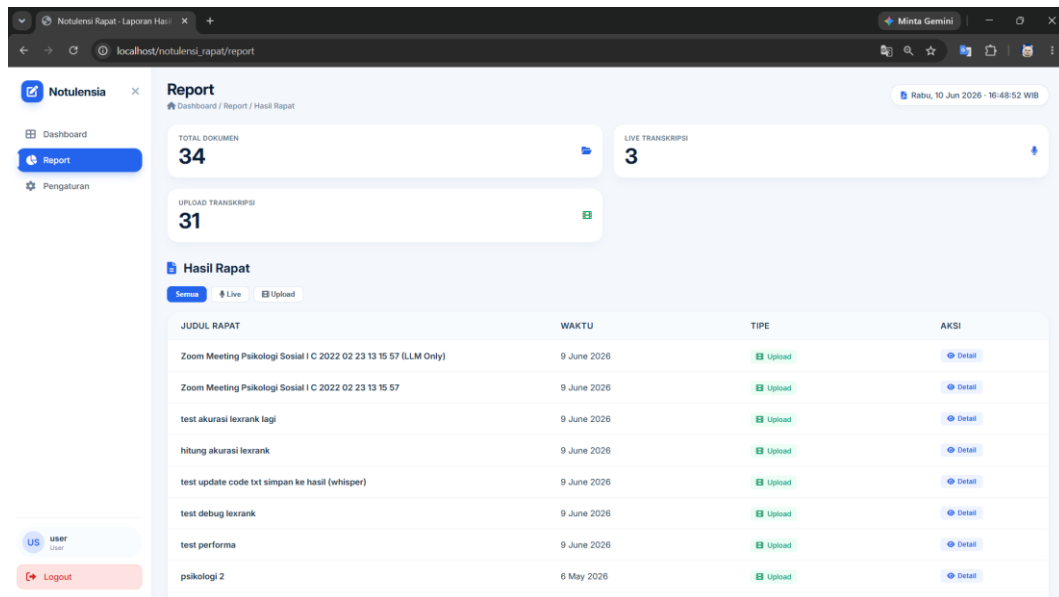


Gambar 4.8 Tampilan Halaman *Dashboard User*

Halaman *Dashboard User* berfungsi sebagai halaman utama yang ditampilkan setelah pengguna berhasil masuk ke dalam sistem. Halaman ini menyediakan informasi ringkas mengenai aktivitas notulensi rapat, seperti jumlah total rapat yang telah diproses, jumlah rapat pada periode tertentu, serta daftar rapat terbaru yang dapat diakses oleh pengguna. Selain itu, dashboard juga menyediakan menu navigasi yang memungkinkan pengguna untuk mengakses fitur *Report* guna melihat hasil notulensi yang telah tersimpan. Berbeda dengan dashboard administrator, dashboard pengguna hanya menampilkan fitur sesuai hak akses yang dimiliki, sehingga penggunaan sistem menjadi lebih sederhana dan terfokus pada kebutuhan pengguna.

9. Halaman Report User

Halaman report user merupakan halaman yang menyediakan informasi mengenai hasil rapat yang telah dilakukan atau di ringkas oleh sistem notulensi.

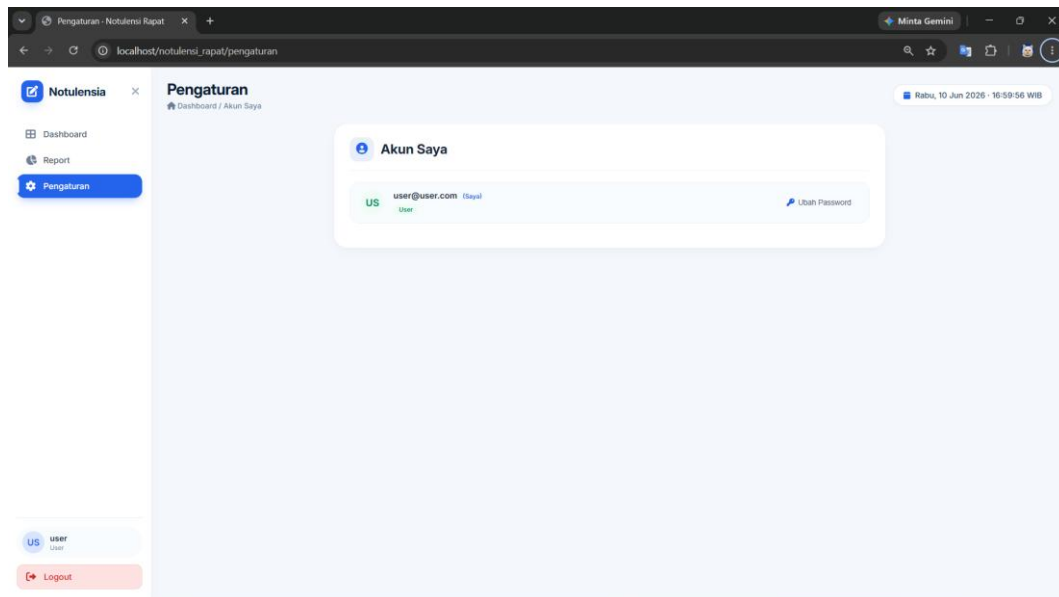


Gambar 4.9 Tampilan Halaman Report User

Halaman *Report User* berfungsi untuk menampilkan riwayat hasil notulensi rapat yang dapat diakses oleh pengguna sesuai dengan hak akses yang dimiliki. Pada halaman ini, pengguna dapat melihat informasi statistik mengenai jumlah dokumen hasil notulensi, jumlah hasil *live transcription*, serta jumlah hasil *upload transcription* yang tersimpan dalam sistem. Selain itu, halaman ini menyediakan daftar hasil rapat yang memuat informasi berupa judul rapat, waktu pelaksanaan, dan jenis transkripsi yang digunakan. Fitur penyaringan berdasarkan kategori tertentu juga tersedia untuk memudahkan pengguna dalam mencari data notulensi. Pengguna dapat menggunakan tombol *Detail* untuk melihat isi notulen dan ringkasan hasil rapat secara lebih lengkap.

10. Halaman Pengaturan User

Halaman Pengaturan pada user merupakan halaman yang menyediakan fitur ubah password untuk akun user.



Gambar 4.10 Tampilan Halaman Pengaturan User

Halaman pengaturan user berfungsi sebagai sarana bagi pengguna untuk mengelola informasi akun yang dimiliki pada sistem notulensi rapat otomatis. Pada halaman ini, pengguna dapat melihat identitas akun yang sedang digunakan, seperti nama pengguna, alamat email, dan level akses yang dimiliki. Selain itu, tersedia fitur *ubah password* yang memungkinkan pengguna memperbarui kata sandi akun guna menjaga keamanan akses terhadap sistem. Dengan adanya halaman ini, pengelolaan akun pengguna dapat dilakukan secara mandiri dengan lebih mudah dan aman.

4.3 Implentasi Whisper Transcription

Sistem pemrosesan audio pada aplikasi Notulensi Rapat menggunakan arsitektur yang terdiri atas frontend berbasis web (*notulensi.php*) dan backend Python Flask (*server.py*). Frontend mengirimkan *chunk* audio melalui *Socket.IO* pada mode *live transcription* atau berkas media melalui *HTTP POST REST API* pada mode *upload*.

1. Perekaman dan Pengiriman Audio Live dari Klien (Frontend)

Pada sisi klien, perekaman dan pengiriman audio secara langsung dikelola oleh berkas *JavaScript live-transcription.js*. Proses pengambilan suara mikrofon ini memanfaatkan antarmuka *AudioContext* dan *ScriptProcessorNode* dengan frekuensi sampel 16 kHz—standar yang dibutuhkan oleh model Whisper. Audio tersebut ditangkap dalam bentuk potongan data (audio chunks) dengan ukuran *buffer 4096 byte*, yang kemudian disimpan sementara di dalam larik *audioChunks*. Setiap 2 detik sekali, fungsi *setInterval* akan memicu penggabungan potongan-potongan tersebut menjadi satu kesatuan *Float32Array*. Data ini lalu dikonversi ke format biner dan dikirimkan ke backend Flask menggunakan pustaka *Socket.IO* melalui *event audio_data*. Implementasi proses perekaman dan pengiriman ini ditunjukkan pada Listing 4.1.

Listing 4.1 Implementasi Pengambilan dan Pengiriman Audio Live dilive-transcription.js

```
// Setup AudioContext untuk capture raw PCM (16kHz)
audioContext = new (window.AudioContext ||
window.webkitAudioContext)({
  sampleRate: AUDIO_SAMPLE_RATE
});

sourceNode =
audioContext.createMediaStreamSource(mediaStream);

// ScriptProcessorNode untuk mengakses raw audio buffer
scriptProcessor =
audioContext.createScriptProcessor(BUFFER_SIZE, 1, 1);
scriptProcessor.onaudioprocess = (event) => {
  if (!isListening) return;
  const inputData = event.inputBuffer.getChannelData(0);
  audioChunks.push(new Float32Array(inputData));
};

sourceNode.connect(scriptProcessor);
scriptProcessor.connect(audioContext.destination);

// Pengiriman berkala setiap 2 detik (2000 ms)
function sendAudioChunks() {
  if (!socket || !socket.connected || audioChunks.length
=== 0) return;

  // Gabungkan semua chunks menjadi satu Float32Array
```

```

    const totalLength = audioChunks.reduce((sum, chunk) =>
sum + chunk.length, 0);
    const combined = new Float32Array(totalLength);
    let offset = 0;
    for (const chunk of audioChunks) {
        combined.set(chunk, offset);
        offset += chunk.length;
    }

    // Kirim sebagai data biner mentah via WebSocket
    socket.emit('audio_data', combined.buffer);

    // Reset buffer chunks
    audioChunks = [];
}

```

2. Tahap Inisialisasi Audio (Menyesuaikan Standar Whisper)

Pada tahap ini, sistem menyiapkan lingkungan perekaman suara menggunakan Web Audio API dengan menginisialisasi *AudioContext* sebagai mesin utama pemroses audio di peramban (*browser*). Pengaturan frekuensi sampel (*sampleRate*) secara spesifik dikonfigurasi pada 16 kHz. Hal ini sangat krusial mengingat model Whisper secara khusus dilatih menggunakan data suara pada frekuensi 16.000 Hz. Dengan menetapkan frekuensi ini sejak awal di sisi klien, peladen (*backend*) tidak perlu lagi melakukan proses penyesuaian frekuensi (*resampling*) yang rentan mengonsumsi memori dan waktu komputasi ekstra. Selanjutnya, sumber suara dari mikrofon pengguna yang tersimpan dalam variabel *mediaStream* dihubungkan ke dalam sistem pemrosesan menggunakan metode *createMediaStreamSource*.

3. Tahap Pemotongan dan Penampungan Suara (*Buffering*)

Setelah mikrofon berhasil terhubung, aliran suara tidak langsung direkam secara utuh, melainkan dipecah menjadi potongan-potongan kecil. Proses ini memanfaatkan *ScriptProcessorNode* yang bertugas mengambil aliran audio mentah (Raw PCM) sesuai dengan ukuran penampung sementara (*BUFFER_SIZE*). Konfigurasi parameter 1, 1 pada *node* ini menandakan penggunaan mode *mono* (satu saluran masukan dan satu saluran keluaran), yang dinilai lebih ringan secara komputasi dan sudah sangat memadai untuk kebutuhan deteksi suara pada Whisper. Selama aliran audio masuk, pendengar kejadian (*event listener*) melalui fungsi

onaudioprocess akan terus menyala. Apabila variabel status *isListening* bernilai *true*, sistem secara otomatis akan mengekstrak data suara tersebut ke dalam wujud angka desimal berpresisi tinggi (*Float32Array*) dan menyimpannya sementara ke dalam larik penampung yang diberi nama *audioChunks*.

4. Tahap Penggabungan dan Pengiriman Data (Transmission)

Tahap terakhir dikendalikan oleh fungsi *sendAudioChunks()* yang dipanggil secara berkala setiap 2 detik (2000 milidetik) untuk mentransmisikan data audio dari *frontend* ke *backend*. Sebelum pengiriman dilakukan, sistem terlebih dahulu memvalidasi status koneksi *socket* dan memastikan larik *audioChunks* tidak dalam kondisi kosong. Hal ini bertujuan untuk mencegah transmisi data nihil yang berisiko menyebabkan galat (*error*) di peladen. Setelah validasi terpenuhi, seluruh potongan kecil di dalam *audioChunks* digabungkan menjadi satu larik data utuh (*combined*) berdasarkan perhitungan total panjang dan penyusunan ulang menggunakan variabel *offset*. Penggabungan ini diperlukan karena model *Whisper* membutuhkan potongan audio dengan konteks yang memadai untuk mengenali kata secara akurat. Data yang telah disatukan tersebut lalu ditransmisikan ke peladen melalui perintah *socket.emit('audio_data', combined.buffer)*. Penggunaan atribut *.buffer* memastikan bahwa data dikirim dalam bentuk biner mentah (*ArrayBuffer*) melalui *WebSocket*, sehingga prosesnya berjalan jauh lebih cepat, hemat *bandwidth*, dan efisien dibandingkan pengiriman berbasis teks atau JSON. Siklus ini diakhiri dengan pembersihan data melalui instruksi *audioChunks = []*, yang berfungsi mengosongkan penampungan agar siap menerima rekaman dua detik berikutnya sekaligus mencegah penumpukan memori serta duplikasi transmisi data.

5. Digital Signal Processing (DSP) & Audio Pre-processing (Backend)

Sebelum sinyal audio mentah dikirimkan ke model neural network *Whisper* untuk inferensi, backend *Flask* pada berkas *audio_processor.py* melakukan serangkaian tahap pemrosesan sinyal digital (Digital Signal Processing atau DSP) guna menjamin kualitas audio masukan berada dalam kondisi optimal. Pipeline DSP ini terbagi menjadi tiga tahapan kritis:

- a. Polyphase Resampling: Menggunakan fungsi `scipy.signal.resample_poly` untuk menyamakan laju sampel sinyal. Pemilihan penyaring polifase (polyphase filter) secara matematis jauh lebih akurat dibandingkan metode interpolasi linier sederhana karena mampu mencegah efek aliasing (distorsi frekuensi) dan menjaga karakteristik struktur formant pada frekuensi suara manusia.
- b. Spectral Subtraction Noise Reduction: Melalui pustaka `noisereduce`, sistem melakukan teknik pengurangan derau berbasis pengurangan spektral (spectral subtraction) terhadap tipe kebisingan stasioner (seperti dengung AC atau kipas komputer). Pengurangan intensitas derau dikendalikan oleh parameter `nr_prop_decrease = 0.80` (mengurangi 80% komponen derau stasioner) untuk meningkatkan rasio isyarat terhadap derau (Signal-to-Noise Ratio atau SNR) tanpa merusak kejelasan fonem ucapan asli.
- c. Amplitude Normalization: Menormalisasi amplitudo puncak sinyal audio ke rentang dinamis $[-1.0, 1.0]$. Hal ini bertujuan agar volume audio selalu konsisten, mencegah kliping sinyal, dan memenuhi spesifikasi distribusi data masukan yang diharapkan oleh tensor internal Whisper.

Implementasi lengkap pipeline DSP ini ditunjukkan pada Listing 4.2.

Listing 4.2 Implementasi Pre-processing Audio di `Audio_Processor.py`

```
class AudioProcessor:
    """
    Utilitas pemrosesan audio.
    Pipeline utama (method prepare):
        audio mentah → resample → noise reduction →
        normalisasi → siap untuk Whisper
    """

    def __init__(self, cfg: "Config"):
        self.noise_filter = NoiseFilter(
            stationary=cfg.nr_stationary,
            prop_decrease=cfg.nr_prop_decrease
        )

    def prepare(self, audio: np.ndarray, from_sr: int,
to_sr: int) -> np.ndarray:
        """Jalankan pipeline lengkap."""
```

```

        audio = self.resample(audio, from_sr, to_sr)
        audio = self.noise_filter.apply(audio, to_sr)
        audio = self.normalize(audio)
        return audio

    def resample(self, audio: np.ndarray, from_sr: int,
to_sr: int) -> np.ndarray:
        """Resample audio menggunakan
        scipy.signal.resample_poly."""
        if from_sr == to_sr:
            return audio
        common = gcd(from_sr, to_sr)
        up = to_sr // common
        down = from_sr // common
        return scipy.signal.resample_poly(audio, up,
down).astype(np.float32)

    def normalize(self, audio: np.ndarray) -> np.ndarray:
        """Normalisasi amplitudo audio ke rentang [-1,
1]."""
        peak = np.max(np.abs(audio))
        if peak > 0:
            return (audio / peak).astype(np.float32)
        return audio

```

Berdasarkan Listing 4.2, fungsi *prepare()* menyatukan ketiga tahapan pembersihan sinyal audio secara sekuensial. Pendekatan ini memastikan sinyal suara dari berbagai perangkat mikrofon klien yang memiliki karakteristik laju sampel bawaan berbeda (seperti 44.1 kHz atau 48 kHz) berhasil diselaraskan ke dalam format monokanal 16 kHz dengan tingkat bising yang rendah sebelum diserahkan ke mesin Whisper.

6. Konfigurasi dan Inferensi OpenAI Whisper (Backend)

Setelah sinyal audio dibersihkan melalui pipeline DSP, data suara diumpankan ke kelas *Transcriber.py* untuk dilakukan pengenalan ucapan otomatis (Automatic Speech Recognition atau ASR). Sistem Notulensi Rapat ini menggunakan model OpenAI Whisper (Small) yang ditranskrip secara lokal pada server menggunakan akselerasi CUDA GPU (bila terdeteksi) atau CPU sebagai cadangan otomatis. Untuk meminimalkan kesalahan transliterasi dan ketidakakuratan kalimat, pemanggilan inferensi diatur dengan parameter konfigurasi sebagai berikut:

- a. `language='id'`: Mengunci target bahasa ke Bahasa Indonesia untuk menghindari kesalahan klasifikasi bahasa (misalnya mendeteksi ucapan lokal sebagai bahasa lain).
- b. `initial_prompt`: Menyediakan kalimat panduan yang memuat daftar glosarium istilah teknis campuran bahasa Inggris (seperti machine learning, framework, meeting, budget, target, dll.). Teknik rekayasa prompt (prompt engineering) tingkat lanjut ini melatih model untuk melestarikan penulisan kata-kata teknis bahasa Inggris alih-alih mencoba menerjemahkannya ke bahasa Indonesia kasar.
- c. `beam_size=3`: Melakukan pencarian balok (beam search) dengan ukuran lebar 3 jalur paralel untuk menentukan kombinasi kata dengan probabilitas tertinggi, bukan sekadar menggunakan algoritma rakus (greedy decoding) satu jalur.
- d. `temperature_fallback=(0.0, 0.2, 0.4)`: Jika probabilitas transkripsi awal bernilai buruk (di bawah ambang batas `logprob_threshold = -1.0`), Whisper akan menaikkan suhu pencarian secara bertahap untuk mengeksplorasi susunan kata alternatif.
- e. `no_speech_threshold=0.45`: Jika probabilitas keheningan (no speech probability) pada jendela analisis melampaui 45%, Whisper akan menandai klip audio tersebut sebagai sunyi dan mengabaikan pembuatan teks baru.

Implementasi inferensi Whisper ini ditunjukkan pada Listing 4.3.

Listing 4.3 Implementasi Inferensi OpenAI Whisper di `transcriber.py`

```
# Pemanggilan inferensi model OpenAI Whisper secara lokal
dengan parameter khusus

res = self._model.transcribe(
    prepared,
    language=cfg.language,
```

```
task="transcribe",

initial_prompt=cfg.initial_prompt,

fp16=self._use_fp16,

beam_size=cfg.beam_size,

temperature=cfg.temperature_fallback,

condition_on_previous_text=cfg.condition_on_previous_text,

compression_ratio_threshold=cfg.compression_ratio_threshold,

    no_speech_threshold=cfg.no_speech_threshold,

    logprob_threshold=cfg.logprob_threshold,

    word_timestamps=False,

)

text = res["text"].strip()
```

Berdasarkan Listing 4.3, keluaran mentah teks dari Whisper (res["text"]) diekstraksi ke dalam variabel lokal text. Konfigurasi pencarian balok (beam_size=3) dan pembatasan presisi floating-point (fp16) diatur secara hati-hati agar inferensi tidak memakan sumber daya GPU berlebih namun tetap mempertahankan nilai akurasi karakter (CER) dan akurasi kata (WER) yang tinggi.

7. Modul Pengoreksi Tata Bahasa (LanguageTool Auto-Correction)

Karena data masukan berupa percakapan langsung yang sering kali memuat kesalahan pengucapan, ketidakbakuan bahasa, atau kekeliruan kecil pada tanda baca, sistem mengintegrasikan pustaka LanguageTool khusus Bahasa Indonesia (id) untuk melakukan koreksi tata bahasa otomatis (grammatical auto-correct) setelah hasil transkripsi dari Whisper didapatkan.

Proses koreksi dijalankan tepat setelah inferensi Whisper selesai dan hanya dieksekusi apabila bendera konfigurasi `use_autocorrect` aktif. Modul koreksi ini membandingkan kalimat dengan kamus tata bahasa baku bahasa Indonesia untuk membetulkan ejaan kata typo (misalnya mengubah "sekola" menjadi "sekolah") dan memperbaiki penempatan huruf kapital pada awal kalimat secara otomatis sebelum teks disajikan kepada pengguna. Implementasi modul koreksi ini ditunjukkan pada Listing 4.4.

Listing 4.4 Integrasi LanguageTool di `transcriber.py`

```
# Inisialisasi LanguageTool saat inisialisasi awal server
def load_autocorrect(self) -> None:
    """Inisialisasi LanguageTool untuk auto-correct Bahasa
    Indonesia."""
    if not self._cfg.use_autocorrect:
        return
    try:
        import language_tool_python
        self._lt_tool =
language_tool_python.LanguageTool("id")
        self._console.print("[bold green]✓ LanguageTool
siap.[/bold green]")
    except Exception as e:
        self._console.print(f"[yellow]⚠ Auto-correct
tidak tersedia: {e}[/yellow]")
        self._lt_tool = None

# ... di dalam loop _worker():
# Auto-correct dengan LanguageTool (jika aktif) setelah
inferensi selesai
if self._lt_tool and text:
    try:
        text = self._lt_tool.correct(text).strip()
    except Exception:
        pass # Fallback ke teks asli jika modul
LanguageTool mengalami error
```

Berdasarkan Listing 4.4, modul pengoreksi kesalahan ejaan tata bahasa (LanguageTool) dimasukkan ke dalam blok pengecekan try-except tambahan. Hal ini dimaksudkan agar jika pustaka mengalami kegagalan proses atau pemutusan koneksi lokal, sistem akan melakukan graceful fallback dengan menggunakan teks

transkripsi asli dari Whisper tanpa menghentikan jalannya aplikasi secara keseluruhan.

8. Penyaringan Halusinasi Pasca-transkripsi (*Hallucination Filter*)

Whisper sering kali memicu kesalahan halusinasi pengulangan kata yang tidak relevan akibat sensitivitas model ketika menangani audio sunyi. Sebagai lapisan pertahanan pasca-inferensi, teks transkripsi dievaluasi oleh kelas *HallucinationFilter*. Langkah verifikasi penyaringan ini meliputi:

- a. Pengecekan Teks Kosong & Batas Panjang Minimum: Memeriksa apakah teks hasil inferensi memiliki panjang di bawah 4 karakter.
- b. Pencocokan Frasa Halusinasi (Daftar Hitam): Mengevaluasi teks terhadap daftar hitam kalimat halusinasi umum (seperti "subscribe", "like dan subscribe", "thank you for watching"). Sistem menggunakan ekspresi reguler `re.sub(r"^[^w\s]", "", lower)` untuk membuang tanda baca sebelum pencocokan sehingga variasi penulisan frasa halusinasi tetap berhasil disaring secara bersih.
- c. Deteksi Redundansi Simetris: Menggunakan pemisahan string untuk mendeteksi pengulangan berlebih di mana separuh kalimat awal sama persis dengan separuh kalimat akhir (misalnya, "selamat pagi rekan-rekan selamat pagi rekan-rekan").

Implementasi lengkap logika deteksi halusinasi ini disajikan pada Listing 4.5.

Listing 4.5 Implementasi Deteksi Halusinasi di *hallucination_filter.py*

```
def is_hallucination(self, text: str) -> bool:
    """Kembalikan True jika teks dianggap halusinasi dan
    harus dibuang."""
    if not text or len(text.strip()) < self._min_len:
        return True

    lower = text.lower().strip()
    lower_clean = re.sub(r"^[^w\s]", "", lower).strip()
```

```

    # Kriteria 1 & 2: Pencocokan daftar hitam frasa
    halusinasi umum
    for phrase in self._phrases:
        phrase_clean = re.sub(r"^[^\\w\\s]", "",
        phrase).strip()

        # Exact match (dengan atau tanpa tanda baca)
        if lower == phrase or lower_clean == phrase_clean:
            return True

        # Teks sangat pendek yang memuat frasa halusinasi
        if lower.count(phrase) >= 1 and len(lower) <
        len(phrase) + 20:
            return True

        # Frasa halusinasi berulang lebih dari 2 kali
        if lower.count(phrase) > 2:
            return True

    # Kriteria 3: Deteksi pengulangan simetris (Separuh
    Awal == Separuh Akhir)
    words = text.strip().split()
    if len(words) >= 6:
        mid = len(words) // 2
        half = " ".join(words[:mid])
        rest = " ".join(words[mid:])
        if half.strip() == rest.strip():
            return True

    return False

```

Berdasarkan Listing 4.5, fungsi *is_hallucination()* mengidentifikasi segala bentuk anomali teks. Apabila data teks terbukti memuat salah satu bentuk halusinasi, fungsi segera mengembalikan nilai True agar teks tersebut dapat dibuang secara langsung oleh server Flask dan tidak dimasukkan ke dalam log akhir transkripsi rapat.

9. Penerimaan dan Transkripsi File Rekaman Asinkron (Upload)

Sistem Notulensi Rapat juga memfasilitasi kebutuhan transkripsi dari unggahan berkas rapat berdurasi panjang melalui berkas *notulensi_upload.php*. Permintaan ini diterima oleh *endpoint* REST API */transcribe* pada berkas *server.py*. Untuk memelihara stabilitas pemakaian RAM dan GPU pada server lokal, alur kerja diimplementasikan sebagai berikut:

- a. *Penyimpanan Berkas Sementara & Pembuatan Record Database*: Berkas yang diunggah divalidasi format ekstensinya (`ALLOWED_EXT`), disimpan ke folder temporer `uploads/`, dan record baris awal langsung didaftarkan ke MySQL database dengan status awal [`PROSES_TRANSKRIPSI`].
- b. *Asynchronous Threading*: Backend merespons permintaan klien dengan cepat melalui pengembalian status antrean (`queued`), sementara proses transkripsi yang berat didelegasikan ke thread latar belakang terpisah (`threading.Thread`) agar tidak mengunci (`freeze`) server Flask.
- c. *Queue Lock Mutex*: Sistem membatasi proses transkripsi berkas media agar hanya berjalan secara sekuensial (satu berkas pada satu waktu) menggunakan kunci pengecualian bersama (`mutual exclusion lock`) yaitu `_upload_lock`. Hal ini bertujuan mencegah `out-of-memory (OOM)` pada GPU server apabila banyak pengguna mengunggah video rapat secara bersamaan.
- d. *Pipeline Transkripsi, Penyimpanan, & Perangkuman*: Setelah thread memperoleh kunci lock, berkas media dibaca oleh `FileTranscriber` untuk menghasilkan teks transkripsi penuh, merekam durasi asli media ke database, memicu perangkuman `LexRank (LexRankSummarizer)`, dan menuliskan teks ringkasan akhir ke MySQL database.

Implementasi alur backend asinkron ini ditunjukkan pada Listing 4.6.

Listing 4.6 Implementasi REST API Transkripsi Berkas Asinkron di `server.py`

```
@app.route('/transcribe', methods=['POST'])

def api_transcribe():

    """Upload file rekaman → Whisper FileTranscriber →
    Pemrosesan Asinkron."""

    if 'video_file' not in request.files:
```



```

        return jsonify({"status": "error", "message":
"File tidak ditemukan"}), 400

    file = request.files['video_file']

    ts =
datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

    temp_name = f"upload_{ts}.{file.filename.rsplit('.',
1)[-1].lower()}"

    temp_path = os.path.join(UPLOAD_DIR, temp_name)

    file.save(temp_path)

    # Simpan record awal ke Database dengan status penanda
proses transkripsi

    db_id = save_to_db(

        judul=request.form.get('judul_rapat', '').strip(),

        tipe='Upload',

        txt_path='',

        ringkasan='[PROSES_TRANSKRIPSI]'

    )

    # Jalankan alur penuh pemrosesan secara asinkron di
latar belakang

    def _run_full_bg_pipeline(t_path, original_n,
db_record_id):

        # Kunci mutex antrean eksklusif

        with _upload_lock:

            try:

                file_transcriber =
FileTranscriber(file_cfg)

```

```

        file_transcriber.load_model()

        final_text =
file_transcriber.transcribe_file(t_path)

        if final_text:

            out_path =
file_transcriber.save_result(final_text, original_n)

            txt_filename =
os.path.basename(out_path)

            # Simpan data durasi & hasil transkrip
ke DB

            update_durasi_in_db(db_record_id,
getattr(file_transcriber, 'last_duration_sec', 0))

            update_transkrip_in_db(db_record_id,
txt_filename)

            update_summary_in_db(db_record_id,
'[PROSES_SUMMARIZE]')

            # — Ringkasan Otomatis LexRank —

            result_bg =
summarizer.summarize_with_debug(final_text)

            update_summary_in_db(db_record_id,
result_bg['ringkasan'])

            finally:

                if os.path.exists(t_path):
os.unlink(t_path)

            bg_thread =
threading.Thread(target=_run_full_bg_pipeline,
args=(temp_path, file.filename, db_id))

            bg_thread.start()

```

```
        return jsonify({"status": "queued", "db_id": db_id,
                        "message": "Proses transkripsi berjalan di latar belakang"})
```

Berdasarkan Listing 4.6, urutan operasi pada fungsi `_run_full_bg_pipeline()` bersifat kritis. Modul `FileTranscriber` dipanggil di dalam sekup blok `try-finally` untuk memastikan bahwa berkas temporer rekaman suara (`temp_path`) yang diunggah akan selalu dihapus secara permanen dari disk keras server setelah selesai diproses, terlepas dari apakah proses transkripsi tersebut berakhir sukses ataupun mengalami kegagalan.

4.4 Implementasi Word filler dan Noise Filter

Implementasi *noise filter* dan *word filler cleaning* dilakukan sebagai bagian dari tahapan pra-pemrosesan pada sistem Notulensi Rapat. Proses ini bertujuan untuk meminimalkan gangguan akibat kebisingan sinyal audio serta menghilangkan kata-kata pengisi yang dapat memengaruhi kualitas transkripsi dan hasil peringkasan.

1. Reduksi Kebisingan Sinyal Audio (Noise Filter)

Modul ini memanfaatkan pustaka `noisereduce` yang mengimplementasikan algoritma pengurangan derau berbasis pengurangan spektral (*spectral subtraction*). Algoritma ini membagi sinyal audio ke dalam domain frekuensi menggunakan transformasi Fourier, mengestimasi tanda kebisingan stasioner (*stationary noise*), kemudian mengurangi magnitudo kebisingan tersebut dari sinyal audio asli. Tingkat pengurangan derau diatur secara presisi melalui parameter `prop_decrease = 0.80` untuk mereduksi 80% kebisingan stasioner. Nilai ini dipilih karena efektif menurunkan tingkat bising tanpa merusak struktur kejelasan ucapan (*speech distortion*) yang dapat membingungkan encoder Whisper. Implementasi kelas `NoiseFilter` ditunjukkan pada Listing 4.7.

Listing 4.7 Implementasi Modul Reduksi Kebisingan di `noise_filter.py`

```

class NoiseFilter:
    """Kelas pemrosesan untuk mengurangi noise."""

    def __init__(self, stationary: bool = True,
prop_decrease: float = 0.80):
        self._nr_stationary = stationary
        self._nr_prop_decrease = prop_decrease

    def apply(self, audio: np.ndarray, sr: int) ->
np.ndarray:
        """
        Terapkan noise reduction pada audio.
        """
        try:
            import noisereduce as nr
            reduced = nr.reduce_noise(
                y=audio,
                sr=sr,
                stationary=self._nr_stationary,
                prop_decrease=self._nr_prop_decrease,
            )
            return reduced.astype(np.float32)
        except ImportError:
            # Fallback jika pustaka noisereduce tidak
            terpasang
            return audio

```

Berdasarkan Listing 4.7, fungsi `apply()` menerima larik data audio masukan berformat NumPy array beserta laju sampelnya (`sr`). Dengan menggunakan blok penanganan pengecualian `try-except`, sistem memastikan modul tetap berjalan secara robust. Jika pustaka `noisereduce` tidak dimuat di lingkungan server, audio mentah akan dikembalikan tanpa penyaringan untuk menjaga sistem tetap beroperasi. Proses reduksi derau ini dieksekusi secara otomatis pada `audio_processor.py:L44` sebelum data dikirim ke model Whisper.

2. Pembersihan Kata Pengisi Lisan (Word Filler / Filler Words Cleaning)

Hasil transkripsi langsung dari model Whisper umumnya masih mempertahankan karakteristik percakapan lisan yang tidak baku, seperti munculnya kata pengisi (`filler words` / interjeksi lisan seperti: "eh", "ah", "hmm", "oke", "ya kan", atau "eee"). Keberadaan `filler words` ini mengurangi formalitas dokumen notulensi dan mengacaukan pembobotan TF-IDF saat proses perangkuman. Untuk

menghasilkan ringkasan rapat yang bersih dan formal, sistem menerapkan mekanisme pembersihan kata pengisi pada modul lextank.py secara bertahap pada tingkat global (seluruh teks) dan tingkat lokal (kalimat tunggal):

- a. Global Text Cleaning: Sistem memindai teks hasil transkripsi secara keseluruhan untuk menghapus jeda bunyi vokal panjang inline (seperti "eee", "aaa", "uum") menggunakan ekspresi reguler `_FILLER_INLINE_RE`. Sistem juga menghapus frasa pengisi panjang yang cocok dengan kamus `_FILLER_WORDS_EXACT`.
- b. Sentence-Level Cleaning: Pada saat teks dipecah menjadi kalimat-kalimat individu, fungsi `_clean_single_sentence` memangkas filler words yang berada di batas luar kalimat. Kata pengisi di awal kalimat (seperti "nah", "jadi", "baik ya", "oke ini") dibersihkan menggunakan regex `_FILLER_START_RE`, dan kata pengisi di akhir kalimat (seperti "ya", "kan", "sih", "dong") dibersihkan menggunakan regex `_FILLER_END_RE`. Kalimat yang telah dibersihkan kemudian dirapikan tanda bacanya, diawali huruf kapital, dan dipastikan diakhiri tanda titik .

Listing 4.8 Konfigurasi Kamus dan Pola Regex Kata Pengisi di lextank.py

```
# Kamus kata pengisi lisan Indonesia & artefak halusinasi
Whisper
_FILLER_WORDS_EXACT = {
    "eh", "ah", "oh", "uh", "um", "em", "eem", "eee", "hmm",
    "hm",
    "nah", "ya", "yaa", "yaaa", "iya", "iye", "iyaa",
    "gitu", "gitu ya", "gitu lho", "gitu kan",
    "kan", "lho", "lah", "nih", "sih", "deh", "kok", "dong",
    "tuh", "tuh kan",
    "jadi", "jadi ya", "jadi gini",
    "intinya", "pokoknya", "kira-kira",
    "terus", "trus", "kemudian", "lalu",
    "udah", "udah ya", "oke", "oke ya", "oke oke",
    "baik", "baik ya", "baik baik",
    "ayo", "yuk", "yuk yuk",
    "terima kasih", "subscribe", "like", "komen", "music",
    "musik"
}
```

```

# Regex untuk mencocokkan kata pengisi di awal kalimat
(Case-Insensitive)
_FILLER_START_RE = re.compile(
    r'^(?:(:nah|ya|jadi|oke|baik|iya|eh|ah|oh|terus|trus|'
    r'gitu|kan|lah|lho|nih|sih|deh|kok|dong|tuh|'
    r'intinya|pokoknya|kemudian|lalu|udah|jadi\s+ya|'
    r'oke\s+ya|baik\s+ya|nah\s+ini|nah\s+itu),?\s*)+',
    re.IGNORECASE,
)

# Regex untuk mencocokkan kata pengisi di akhir kalimat
_FILLER_END_RE = re.compile(
    r'(?:\s*,?\s*(?:ya|kan|lho|nih|sih|deh|gitu|dong|tuh|'
    r'gitu\s+ya|gitu\s+kan|ya\s+kan))+[.,]?*\s*$',
    re.IGNORECASE,
)

# Regex untuk mendeteksi bunyi dengung/vokal panjang (eee,
aaa, hhhmm, dll)
_FILLER_INLINE_RE = re.compile(
    r'\b(e{2,}|a{2,}|u{2,}|h{2,}m+|[eEaAuU]{3,})\b',
    re.IGNORECASE,
)

```

Listing 4.9 Implementasi Fungsi Pembersihan Kalimat di lexranks.py

```

def _clean_single_sentence(self, sentence: str) -> str:
    """Bersihkan satu kalimat: hapus filler, rapikan tanda
    baca."""

    # 1. Hapus kata pengisi (filler) di awal kalimat
    sentence = _FILLER_START_RE.sub('', sentence).strip()

    # 2. Hapus sisa tanda koma atau spasi berlebih di awal
    sentence = re.sub(r'^[, \s]+', '', sentence)

    # 3. Hapus kata pengisi (filler) di akhir kalimat
    sentence = _FILLER_END_RE.sub('', sentence).strip()

    # 4. Rapikan jarak spasi di sekitar tanda baca utama
    sentence = re.sub(r'\s+([.,!])', r'\1', sentence)
    sentence = re.sub(r'([.,!]){2,}', r'\1', sentence)

    # 5. Hilangkan spasi ganda ganda
    sentence = re.sub(r'\s+', ' ', sentence).strip()

    # 6. Pastikan kalimat diakhiri dengan tanda baca akhir
    yang valid
    if sentence and sentence[-1] not in '!.?':

```

```
        sentence += ' '

    # 7. Format huruf pertama pada kalimat menjadi huruf kapital
    if sentence:
        sentence = sentence[0].upper() + sentence[1:]

    return sentence.strip()
```

Berdasarkan Listing 4.8 dan 4.9, fungsi `_clean_single_sentence()` melakukan restrukturisasi kalimat pasca-transkripsi. Penggunaan regex `_FILLER_START_RE` dan `_FILLER_END_RE` dengan flag `re.IGNORECASE` menjamin kata pengisi yang ditulis dengan huruf besar/kecil di awal atau akhir kalimat (misalnya "Nah, hari ini..." atau "...begitu ya.") terhapus secara sempurna tanpa merusak kalimat inti rapat. Langkah penjaminan kapitalisasi huruf pertama dan tanda baca titik memastikan teks hasil transkripsi mematuhi aturan tata tulis EYD Bahasa Indonesia, yang sangat menopang kualitas ringkasan saat diproses lebih lanjut oleh modul `rewriter LLM`.

4.5 Implementasi Hallucination Filter

Model neural network OpenAI Whisper memiliki karakteristik kelemahan bawaan di mana model cenderung memprediksi kata-kata fiktif ketika memproses segmen audio yang sunyi atau hanya berisi kebisingan latar belakang. Fenomena ini dikenal sebagai halusinasi Whisper. Halusinasi tersebut umumnya berbentuk frasa-frasa templat yang sering muncul pada data latih model (seperti "thank you", "terima kasih sudah menonton", atau "don't forget to subscribe"), atau berupa pengulangan kalimat yang sama secara terus-menerus (looping hallucination). Untuk mengatasi kendala ini, sistem menerapkan penyaringan berlapis menggunakan kelas `HallucinationFilter` pada modul `hallucination_filter.py` sebelum hasil transkripsi dipancarkan ke klien. Metode penyaringan ini mendeteksi anomali teks transkripsi melalui tiga tingkat pemeriksaan logika secara berurutan:

- a. Pemeriksaan Batas Panjang Minimum: Memastikan teks hasil inferensi tidak kosong dan memiliki panjang karakter minimal setara dengan ambang

batas konfigurasi (`min_text_length = 4`). Jika teks terlalu pendek, sistem langsung mengklasifikasikannya sebagai halusinasi/noise.

- b. Pencocokan Pola Daftar Hitam (Blacklist Phrase Matching): Teks mentah dikonversi ke huruf kecil dan seluruh tanda baca dihapus menggunakan ekspresi reguler `re.sub(r"^[^\w\s]", "", lower)`. Sistem kemudian membandingkan teks dengan kamus frasa halusinasi baku (`self._phrases`) melalui tiga skenario: pencocokan eksak (`exact match`), pencocokan substring pada teks pendek (untuk mendeteksi frasa tersemat noise seperti "thank you ", dan deteksi pengulangan frasa hitam yang muncul lebih dari dua kali dalam satu segmen.
- c. Deteksi Pengulangan Simetris (Symmetric Loop Detection): Whisper sering kali terjebak dalam loop inferensi tak terbatas yang mengulang-ulang satu kalimat panjang. Untuk menangkalnya, filter memecah teks menjadi daftar kata. Jika jumlah kata mencapai atau melebihi 6 kata (`len(words) >= 6`), sistem membagi kalimat tersebut menjadi dua bagian sama besar (separuh awal dan separuh akhir). Jika separuh awal identik dengan separuh akhir, kalimat tersebut diklasifikasikan sebagai halusinasi loop.

Implementasi logika deteksi halusinasi pada kelas `HallucinationFilter` ditunjukkan pada Listing 4.10.

Listing 4.10 Implementasi Kelas Deteksi Halusinasi di `hallucination_filter.py`

```
import re

class HallucinationFilter:
    """
    Mendeteksi dan memfilter output hallusinasi dari
    Whisper.
    """

    def __init__(self, cfg: "Config"):
        self._phrases = cfg.hallucination_phrases
        self._min_len = cfg.min_text_length

    def is_hallucination(self, text: str) -> bool:
        """
```



```

        Kembalikan True jika teks dianggap hallusinasi dan
        harus dibuang.
        """
        # Kriteria 1: Teks kosong atau terlalu pendek
        if not text or len(text.strip()) < self._min_len:
            return True

        lower = text.lower().strip()
        lower_clean = re.sub(r"^\w\s]", "",
lower).strip()

        # Kriteria 2: Cocok dengan frasa hallusinasi umum
        for phrase in self._phrases:
            phrase_clean = re.sub(r"^\w\s]", "",
phrase).strip()

            # Exact match (dengan atau tanpa tanda baca)
            if lower == phrase or lower_clean ==
phrase_clean:
                return True

            # Teks sangat pendek yang memuat frasa
            tersebut
            if lower.count(phrase) >= 1 and len(lower) <
len(phrase) + 20:
                return True

            # Frasa muncul lebih dari 2 kali (repetisi)
            if lower.count(phrase) > 2:
                return True

        # Kriteria 3: Deteksi pengulangan (separuh pertama
        == separuh kedua)
        words = text.strip().split()
        if len(words) >= 6:
            mid = len(words) // 2
            half = " ".join(words[:mid])
            rest = " ".join(words[mid:])
            if half.strip() == rest.strip():
                return True

        return False

```

Berdasarkan Listing 4.10, fungsi `is_hallucination()` mengembalikan nilai boolean (True atau False). Melalui normalisasi teks menggunakan `re.sub`, pencocokan frasa tidak terhambat oleh perbedaan tanda baca atau spasi yang dihasilkan Whisper secara acak. Logika pembelahan kalimat pada variabel `half` dan `rest` sangat krusial karena berhasil mengidentifikasi duplikasi kalimat penuh secara

dinamis tanpa harus mendaftarkan kalimat tersebut ke dalam kamus frasa daftar hitam. Segmen audio yang terdeteksi mengembalikan nilai True akan langsung diabaikan oleh server Flask pada alur pemrosesan Socket.IO live maupun pemrosesan file berkas rekaman.

4.6 Implementasi Algoritma Lexrank

Setelah teks transkripsi hasil model Whisper dibersihkan dari kata pengisi (filler words) dan halusinasi suara, sistem menyusun notulen ringkas menggunakan algoritma LexRank. LexRank merupakan metode perangkuman ekstraktif (extractive summarization) berbasis graf (graph-based) yang bekerja dengan prinsip mirip dengan algoritma PageRank Google. Pada metode ini, setiap kalimat dalam transkrip direpresentasikan sebagai sebuah simpul (node) pada graf, dan bobot hubungan (edge) antarsimpul didefinisikan sebagai tingkat kemiripan semantik antar-dua kalimat. Kalimat yang memiliki kemiripan tinggi dengan banyak kalimat lainnya di dalam dokumen akan dianggap memiliki derajat kepentingan (centrality) yang tinggi, sehingga diprioritaskan untuk masuk ke dalam hasil ringkasan.

Alur pemrosesan inti ekstraksi ringkasan LexRank pada berkas lexrank.py dibagi menjadi tiga langkah utama:

1. Pembentukan Vektor TF-IDF dan Matriks Kemiripan (Similarity Matrix)

Kalimat-kalimat masukan pertama-tama diubah ke dalam bentuk matematis berupa vektor pembobotan TF-IDF (Term Frequency–Inverse Document Frequency) melalui kelas TfidfVectorizer dari pustaka scikit-learn. Pembentukan TF-IDF dikonfigurasi menggunakan kombinasi kata unigram dan bigram (ngram_range = (1, 2)) serta menyaring daftar kata henti (stop words) khusus Bahasa Indonesia untuk membuang kata hubung non-informatif. Setelah matriks TF-IDF terbentuk, derajat kemiripan kosinus (cosine similarity) antarkalimat dihitung. Kemiripan di bawah ambang batas (similarity_threshold = 0.1) dibuang (diberi nilai 0.0) untuk mengeliminasi relasi lemah, dan matriks tersebut dinormalisasi secara baris (row-stochastic) agar dapat bertindak sebagai matriks

transisi probabilitas. Implementasi pembentukan matriks ini ditunjukkan pada Listing 4.11.

Listing 4.11 Implementasi Pembentukan Matriks TF-IDF dan Kemiripan di lexrang.py

```
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def _build_tfidf_matrix(self, sentences: list[str]):
    """Bangun TF-IDF matrix. Return None jika gagal."""
    try:
        # Daftar stop words dasar Bahasa Indonesia
        indo_stop = {
            'yang', 'di', 'ke', 'dari', 'ini', 'itu',
            'dan', 'atau',
            'dengan', 'untuk', 'pada', 'ada', 'akan',
            'juga', 'sudah',
            'bisa', 'tidak', 'kalau', 'kita', 'saya',
            'mereka', 'dia',
            'apa', 'itu', 'ini', 'lagi', 'lebih', 'harus',
            'bisa',
            'mau', 'pakai', 'pake', 'kalian', 'nya',
            'tadi', 'nanti',
            'misalnya', 'tentunya', 'memang', 'masih',
            'udah', 'pun',
        }
        vectorizer = TfidfVectorizer(
            min_df=1,
            max_df=0.90,
            token_pattern=r"(?u)\b\w{2,}\b",
            sublinear_tf=True,
            ngram_range=(1, 2),
            stop_words=list(indo_stop),
        )
        return vectorizer.fit_transform(sentences)
    except Exception:
        return None

def _build_similarity_matrix(self, tfidf_matrix) ->
np.ndarray:
    """Cosine similarity → adjacency matrix dengan
    threshold."""
    sim = cosine_similarity(tfidf_matrix)
    # Hapus kemiripan kalimat dengan dirinya sendiri
    np.fill_diagonal(sim, 0.0)
```

```

# Terapkan threshold untuk mengabaikan hubungan lemah
sim[sim < self.cfg.similarity_threshold] = 0.0

# Normalisasi baris matriks agar menjadi row-
stochastic
row_sums = sim.sum(axis=1, keepdims=True)
row_sums[row_sums == 0] = 1.0
return sim / row_sums

```

Berdasarkan Listing 4.11, fungsi `_build_similarity_matrix()` melakukan pengisian nilai diagonal matriks dengan 0.0 menggunakan perintah `np.fill_diagonal(sim, 0.0)` untuk mencegah perhitungan kemiripan kalimat dengan dirinya sendiri (self-loop). Nilai pembagi normalisasi baris disesuaikan dengan kondisi `row_sums == 0` menjadi 1.0 untuk menghindari error pembagian dengan nol pada kalimat terisolasi yang tidak memiliki kemiripan sama sekali dengan kalimat lain.

2. Perhitungan Sentralitas Kalimat Melalui Power Iteration

Setelah matriks kemiripan transisi dibentuk, algoritma mencari distribusi stasioner (stationary distribution) dari rantai Markov graf kalimat tersebut. Perhitungan nilai skor sentralitas untuk masing-masing kalimat ditentukan menggunakan metode Power Iteration. Metode numerik ini mengalikan vektor skor sentralitas $\mathbf{p}$ dengan matriks transisi $\mathbf{M}$ secara berulang hingga mencapai kondisi konvergen, yang dirumuskan secara matematis sebagai berikut:

$$\mathbf{p}_{t+1} = \frac{1-d}{N} \mathbf{e} + d \cdot \mathbf{M}^T \mathbf{p}_t$$

Di mana d melambangkan faktor redaman (damping factor, diset pada nilai default 0.85), N melambangkan jumlah total kalimat, dan $\mathbf{e}$ melambangkan vektor satuan. Iterasi dihentikan apabila selisih norma vektor skor

baru dengan skor sebelumnya telah berada di bawah nilai toleransi (`convergence_threshold = 1e-5`) atau ketika iterasi telah mencapai batas maksimum (`max_iter = 1000`). Implementasi kalkulasi numerik ini ditunjukkan pada Listing 4.12.

Listing 4.12 Implementasi Perhitungan Power Iteration di `lexrank.py`

```
def _power_iteration(self, sim_matrix: np.ndarray) ->
    np.ndarray:
    """LexRank power iteration (seperti PageRank)."""
    n = sim_matrix.shape[0]
    d = self.cfg.damping
    # Inisialisasi awal vektor skor dengan nilai seragam
    (1/N)
    scores = np.ones(n) / n

    for _ in range(self.cfg.max_iter):
        # Persamaan rekursif Power Iteration dengan faktor
        redaman (damping)
        new_scores = (1 - d) / n + d *
        sim_matrix.T.dot(scores)

        # Cek kondisi konvergensi (selisih L2-Norm di
        bawah threshold)
        if np.linalg.norm(new_scores - scores) <
        self.cfg.convergence_threshold:
            break
        scores = new_scores

    return scores
```

Berdasarkan Listing 4.12, fungsi `_power_iteration()` menginisialisasi vektor `scores` dengan bobot yang seragam. Operasi perkalian titik matriks transisi transpos `sim_matrix.T.dot(scores)` dihitung pada setiap langkah iterasi. Kriteria konvergensi diuji menggunakan fungsi norm aljabar linier `np.linalg.norm()` dari pustaka NumPy untuk mengukur jarak Euclidean L_2 antarvektor skor, menjamin iterasi berhenti tepat waktu saat perubahan skor sudah sangat kecil (konvergen).

3. Ekstraksi Top-N dan Penataan Kalimat

Setelah nilai skor sentralitas dari seluruh kalimat terkalkulasi, sistem menentukan jumlah target kalimat yang akan diekstraksi berdasarkan rumus rasio kompresi secara dinamis:

$$\text{target_sentences} = \frac{\text{max_sentences} \times \text{len}(\text{sentences}) \times \text{compression_ratio}}{\text{min_sentences}}$$

Sistem menyaring kalimat-kalimat dengan nilai skor LexRank tertinggi sebanyak jumlah target tersebut. Agar alur notulensi rapat yang dihasilkan tetap terstruktur dan logis sesuai kronologi jalannya rapat, kalimat-kalimat terpilih tidak disajikan berdasarkan urutan skor tertinggi melainkan diurutkan kembali berdasarkan nomor indeks posisi kemunculan aslinya di dalam berkas transkrip asli melalui fungsi `sorted(np.argsort(scores)[-n:].tolist())`. Terakhir, kalimat terpilih digabungkan kembali menjadi paragraf utuh yang rapi.

4.7 Implementasi Llama 3.2 (Abstractive Summarization)

Meskipun pemrosesan LexRank berhasil mengekstraksi kalimat-kalimat kunci yang merepresentasikan topik rapat, ringkasan ekstraktif tersebut sering kali terasa terputus-putus (disjointed), kurang koheren, dan tidak memiliki alur transisi bahasa yang alami. Untuk menjembatani kekurangan ini, sistem Notulensi Rapat mengintegrasikan modul Abstractive Summarization menggunakan model bahasa besar (Large Language Model atau LLM) Llama 3.2 (3B) yang dijalankan secara lokal di server menggunakan mesin Ollama via berkas `ollama_rewriter.py`.

Modul OllamaRewriter bertugas membaca poin-poin ekstraktif dari LexRank serta seluruh konteks transkrip asli, kemudian menulis ulang (rewrite) teks tersebut menjadi dokumen notulensi rapat formal yang koheren, berstruktur, dan profesional tanpa menimbulkan biaya API eksternal. Alur implementasinya dijabarkan melalui tahapan teknis sebagai berikut:

1. Rekayasa Prompt dan Strategi Konteks Meluncur (Sliding Context Window)

Sebelum memanggil API Llama 3.2, sistem menyusun instruksi prompt yang sangat ketat pada fungsi `_build_prompt`. Prompt ini dirancang dengan rekayasa prompt berbasis peran (role-based prompting) di mana model diposisikan sebagai sekretaris akademik profesional. Untuk mengatasi keterbatasan memori konteks pada transkrip rapat yang sangat panjang, sistem menerapkan strategi Sliding Context Window: jika ukuran transkrip asli melampaui 12.000 karakter (~3.000 token), sistem secara otomatis memotong dan menyatukan tiga bagian transkrip (awal, tengah, dan akhir) untuk disematkan sebagai konteks pendukung, menghindari kegagalan pemrosesan out-of-token. Batas konteks ditetapkan pada 12.000 karakter (bukan 20.000) secara sengaja agar ruang inference yang tersisa pada jendela konteks model (yang dikonfigurasi sebesar 16.384 token melalui parameter `num_ctx`) cukup lebar untuk menghasilkan output notulensi yang panjang dan tidak terpotong di tengah. Implementasi pembentukan prompt ini ditunjukkan pada Listing 4.13.

Listing 4.13 Implementasi Pembentukan Prompt Notulensi di `ollama_rewriter.py`

```
def _build_prompt(self, sentences: list[str], context: str = "") -> str:

    """Bangun prompt yang menghasilkan notulensi rapat
    berstruktur per bagian."""

    joined = "\n".join(f"- {s}" for s in sentences)
```

```

context_section = ""

if context:

    max_ctx = 12000 # ~3000 token – cukup untuk
konteks, hemat ruang agar output tidak terpotong

    if len(context) <= max_ctx:

        context_section = f"\n\nTRANSKRIP LENGKAP
RAPAT (gunakan sebagai sumber utama):\n{context}\n"

    else:

        # Strategi Konteks Meluncur: Ambil
potongan awal, tengah, dan akhir

        chunk = max_ctx // 3

        ctx_start = context[:chunk].strip()

        mid_point = len(context) // 2

        ctx_mid = context[mid_point - chunk//2 :
mid_point + chunk//2].strip()

        ctx_end = context[-chunk:].strip()

        context_section = f"""

TRANSKRIP RAPAT (potongan awal, tengah, dan akhir):

Bagian awal:

{ctx_start}

Bagian tengah:

{ctx_mid}

Bagian akhir:

{ctx_end}

```


"""

prompt = f"""Anda adalah sekretaris akademik profesional yang bertugas membuat notulensi rapat atau catatan perkuliahan yang lengkap, terstruktur, dan informatif.

TUGAS ANDA:

Baca transkrip rapat berikut dengan cermat, lalu susun NOTULENSI RAPAT yang komprehensif. Notulensi harus mampu dipahami oleh orang yang tidak hadir dalam rapat tersebut.

=== ATURAN KONTEN (WAJIB) ===

1. HANYA gunakan informasi dari transkrip yang diberikan di bawah. JANGAN mengarang atau menggunakan contoh.
2. JELASKAN SETIAP KONSEP SECARA LENGKAP dari isi transkrip. Jangan hanya menyebut nama konsep.
3. TANGKAP SEMUA POIN PENTING dari transkrip. Jangan melewatkan informasi substansial.
4. Setiap poin harus SPESIFIK dan INFORMATIF sesuai isi transkrip.
5. SINTESIS pertanyaan dan diskusi. Tulis pertanyaan apa yang diajukan dan jawaban apa yang diberikan.
6. Gunakan Bahasa Indonesia baku. Hapus kata filler (ya, gitu, nah, kan, oke, eee).

=== ATURAN FORMAT OUTPUT (WAJIB DIKUTI PERSIS) ===

1. Mulai langsung dengan bagian pertama. JANGAN tulis pengantar seperti "Berikut notulensi..." atau "Di bawah ini adalah...".
2. Setiap JUDUL BAGIAN ditulis dengan format: ****Judul Bagian:**** (diapit dua bintang, diakhiri titik dua).

3. Di bawah setiap judul, tulis poin-poin menggunakan tanda strip: - poin

4. Pisahkan setiap bagian dengan SATU baris kosong.

5. HANYA tanda ** boleh digunakan – hanya untuk judul bagian. DILARANG menggunakan ##, *, _, atau ` dalam output.

6. DILARANG menggunakan angka di depan poin (1. 2. 3.) – gunakan hanya tanda strip (-).

=== STRUKTUR BAGIAN (sesuaikan dengan isi transkrip yang diberikan) ===

****Pembukaan:****

- [Isi berdasarkan transkrip: siapa yang membuka, apa agenda, siapa yang hadir]

****[Topik Utama 1 dari transkrip]:****

- [Poin penting dari transkrip terkait topik ini]

- [Poin lainnya dari transkrip]

****[Topik Utama 2 dari transkrip – jika ada]:****

- [Isi dari transkrip]

****Pertanyaan dan Diskusi:****

- Pertanyaan: [isi dari transkrip] – Jawaban: [isi dari transkrip]

****Kesimpulan:****

- [Simpulan dari transkrip]

```

- [Tindak lanjut dari transkrip jika disebutkan]

**Penutup:**

- [Penutupan sesuai transkrip]

=== DATA TRANSKRIP YANG HARUS DIRINGKAS (SUMBER UTAMA –
WAJIB DIGUNAKAN) ===

POIN-POIN KUNCI DARI TRANSKRIP:

{joined}

{context_section}

NOTULENSI RAPAT (berdasarkan data di atas):"""

        return prompt

```

Berdasarkan Listing 4.13, fungsi `_build_prompt()` secara ketat memformat struktur prompt. Pembagian transkrip panjang menjadi tiga bagian (`ctx_start`, `ctx_mid`, dan `ctx_end`) bertujuan untuk menjaga agar informasi penting di awal rapat (seperti pembukaan agenda), tengah rapat (pembahasan detail), dan akhir rapat (kesimpulan/keputusan) tetap masuk ke dalam memori inferensi Llama 3.2.

2. Pemanggilan REST API Ollama dengan Parameter Konsistensi Rendah

Pemanggilan model dilakukan melalui REST API lokal Ollama di URL `/api/generate`. Untuk menjaga keandalan teks notulensi agar bersifat informatif dan tidak berhalusinasi di luar data transkrip (factually grounded), sistem mengonfigurasi parameter inferensi tingkat rendah secara hati-hati:

- a. `temperature = 0.15`: Nilai suhu diatur sangat rendah (hampir mendekati pencarian serakah/deterministik) agar Llama 3.2 menghasilkan output yang

konsisten mengikuti aturan prompt, format judul, serta menghindari pembuatan teks kreatif/fiktif.

- b. `num_ctx = 8192`: Mengalokasikan jendela konteks sebesar 8192 token untuk menampung volume data transkrip rapat berdurasi panjang.
- c. `num_predict = 4096`: Menetapkan batas pembuatan token keluaran hingga 4096 token untuk mencegah notulensi terpotong di tengah jalan.
- d. `repeat_penalty = 1.15`: Mencegah model menghasilkan kalimat yang redundan atau mengulang pola kata yang sama.

Implementasi pemanggilan API Ollama ini ditunjukkan pada Listing 4.14.

Listing 4.14 Implementasi Pemanggilan REST API Ollama di `ollama_rewriter.py`

```
import json
import urllib.request

def _call_ollama(self, prompt: str) -> str:
    """Panggil Ollama API untuk generate teks."""
    url = f"{self.cfg.ollama_base_url}/api/generate"

    # Menyusun payload JSON konfigurasi Llama 3.2
    payload = json.dumps({
        "model": self.cfg.ollama_model, # default:
        llama3.2:3b
        "prompt": prompt,
        "stream": False,                # Ambil respons
        secara utuh, bukan streaming
        "options": {
            "temperature": 0.15,        # Suhu rendah untuk
            konsistensi & kepatuhan prompt
        }
    })

    request = urllib.request.Request(url, data=payload.encode('utf-8'))
    response = urllib.request.urlopen(request)
    return response.read().decode('utf-8')
```

```

        "top_p": 0.90,
        "num_predict": 4096,          # Batas token
keluaran
        "num_ctx": 8192,              # Alokasi window
konteks
        "repeat_penalty": 1.15,      # Penalti
pengulangan kata
        "top_k": 40,
    },
    }).encode("utf-8")

req = urllib.request.Request(
    url,
    data=payload,
    headers={"Content-Type": "application/json"},
    method="POST",
)

print(f"[OllamaRewriter] Memanggil Ollama
({self.cfg.ollama_model})...", flush=True)

# Kirim request HTTP POST menggunakan urllib bawaan
Python (non-blocking)

with urllib.request.urlopen(req,
timeout=self.cfg.ollama_timeout) as resp:

    body = json.loads(resp.read().decode("utf-8"))

    return body.get("response", "")

```

Berdasarkan Listing 4.14, fungsi `_call_ollama()` menggunakan pustaka standar bawaan Python `urllib.request` alih-alih pustaka eksternal seperti `requests`. Hal ini

dipilih untuk menjaga keringan ketergantungan modul backend dan memastikan kompatibilitas pemanggilan API yang cepat ke service Ollama yang berjalan pada port lokal localhost.

3. Pasca-pemrosesan Pembersihan Hasil Output LLM (Post-Processing)

Model LLM sering kali menyertakan kalimat pengantar bawaan (seperti "Tentu, ini adalah notulensi rapat..."), menggunakan sintaksis heading markdown standar (**## Judul**), atau menerapkan penebalan kata acak di dalam teks paragraf. Untuk memastikan format keluaran bersih dan kompatibel dengan parser HTML frontend aplikasi PHP, modul melakukan pasca-pemrosesan pembersihan menggunakan ekspresi reguler pada fungsi `_clean_response`. Langkah pembersihan meliputi:

- a. Pembersihan Pengantar: Menghapus kalimat basa-basi di awal teks menggunakan pola regex `re.sub(r'^(ringkasan|summary|hasil|berikut|...)'...)`
- b. Pembersihan Judul: Mengubah semua tanda heading markdown **## Judul** menjadi format judul tebal khusus ****Judul Bagian:**** dengan titik dua di akhir kata.
- c. Pembersihan Bold Internal: Menghapus tanda bintang ganda ****** di luar baris judul agar teks di dalam poin-poin penjelasan bersih dari format cetak tebal acak yang mengganggu keterbacaan.

Implementasi lengkap logika pembersihan pasca-inferensi ini ditunjukkan pada Listing 4.15.

Listing 4.15 Implementasi Fungsi Pembersihan Respon LLM di
`ollama_rewriter.py`

```
def _clean_response(self, text: str) -> str:
    """Bersihkan output LLM agar sesuai dengan format
    render HTML PHP."""
```

```

# 1. Hapus tag HTML yang tidak sengaja ter-generate
text = re.sub(r'<[^>]+>', '', text)

# 2. Hapus pengantar generik/basa-basi di awal teks
text = re.sub(
    r'^(ringkasan|summary|hasil|berikut|di bawah
ini|adalah notulensi[^:]*|'

    r'notulensi rapat[^:]*|berdasarkan
transkrip[^:]*|'

r'tentu[, \s]+berikut[^:]*|baik[, \s]+berikut[^:]*)[:\s\n]*'
,

    '', text, flags=re.IGNORECASE
)

# 3. Hapus simbol heading markdown (## Header)
text = re.sub(r'^#{1,6}\s*', '', text,
flags=re.MULTILINE)

# 4. Hapus backtick format kode (`)
text = re.sub(r'`([^\`]+)`', r'\1', text)

# 5. Konversi standard bullet points (* -> -)
text = re.sub(r'^\s*\*\s+', '- ', text,
flags=re.MULTILINE)

# 6. Bersihkan formatting baris demi baris
cleaned_lines = []
for line in text.split('\n'):
    stripped = line.strip()
    # Deteksi baris yang berperan sebagai Judul Bagian
    is_heading = bool(re.match(r'^\*\*.*?:\s*\$',
stripped) or
re.match(r'^\*\*.*?*\s*?:\s*\$', stripped))
    if is_heading:
        # Normalisasi penulisan agar konsisten
        berformat: **Judul:**
        line = re.sub(r'^\*\*(.+?)\*\*?:\s*\$',
                        lambda m:
f'**{m.group(1).rstrip(":")}:**',
                        stripped)
    else:

```

```

        # Jika baris biasa, hapus format bold/italic
        internal untuk estetika notulen
        line = re.sub(r'\*\*(.+?)\*\*', r'\1', line)
        line = re.sub(r'__(.+?)__', r'\1', line)
        line =
re.sub(r'(?<\*)\*(?!\\*) (.+?) (?<\*)\*(?!\\*)', r'\1',
line)
        line =
re.sub(r'(?<_)\_(?!_)(.+?) (?<_)\_(?!_)', r'\1', line)
        line = re.sub(r'\*', '', line)
        cleaned_lines.append(line)

    text = '\n'.join(cleaned_lines)

    # 7. Normalisasi spasi dan baris baru yang berlebih
    text = re.sub(r'\n{3,}', '\n\n', text)
    text = re.sub(r' {2,}', ' ', text)

    return text.strip()

```

Berdasarkan Listing 4.15, fungsi `_clean_response()` menjamin kualitas teks notulensi yang sangat rapi. Proses iterasi per baris membagi teks secara ketat, mendeteksi header secara dinamis menggunakan regular expression, membuang format tebal (bold) yang berlebih pada teks deskriptif, namun mempertahankan tanda Judul Bagian agar dapat diubah menjadi tag `<strong>` tebal pada antarmuka web Notulensi Rapat.

4.8 Hasil Pengujian Sistem

Pengujian sistem dilakukan menggunakan dua metode utama, yaitu pengujian Black-Box dan pengujian White-Box. Pengujian Black-Box berfokus pada validasi fungsionalitas sistem dari sudut pandang pengguna tanpa memperhatikan kode internal, sedangkan pengujian White-Box menguji jalur logika internal kode program secara struktural.

4.2.1 Pengujian Akurasi Whisper Trancription

4.2.2 Pengujian Hybrid Extractive Lexrank

4.2.3 Pengujian Ollama Abstractive Summarization

4.2.4 Pengujian Black-Box (Fungsional)

Pengujian Black-Box menguji fungsionalitas sistem dari perspektif pengguna tanpa melihat kode internal. Fokus utama pengujian ini adalah memastikan bahwa setiap masukan (input) menghasilkan keluaran (output) yang sesuai dengan spesifikasi yang telah ditentukan. Berikut adalah hasil pengujian Black-Box pada masing-masing modul sistem.

1. Modul Login

Pengujian modul login dilakukan untuk memverifikasi bahwa sistem dapat memvalidasi kredensial pengguna dengan benar dan memberikan respons yang sesuai pada setiap kondisi masukan.

Tabel 4.1 Hasil Pengujian Black-Box Modul Login

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Login dengan kredensial valid	Buka halaman login, masukkan email dan password yang terdaftar, klik tombol Login	Email: Admin@admin.com Password: admin1234	Sistem berhasil mengautentikasi pengguna dan mengarahkan ke halaman Dashboard dengan pesan sambutan	Berhasil
2	Login dengan password salah	Buka halaman login, masukkan email valid namun password salah, klik tombol Login	Email: Admin@admin.com Password: salah123	Sistem menolak login dan menampilkan pesan kesalahan "Email atau	Berhasil

				Password salah!"	
3	Login dengan email tidak terdaftar	Buka halaman login, masukkan email yang tidak ada di database, klik tombol Login	Email: tidak@ada.com Password: test123	Sistem menampilkan pesan validasi "Email dan Password harus diisi!" dan tidak memproses login	Berhasil
4	Akses halaman terproteksi tanpa login	Buka halaman login, biarkan kedua field kosong, klik tombol Login	Email: (kosong) Password: (kosong)	Sistem mendeteksi sesi tidak valid dan mengarahkan pengguna ke halaman login	Berhasil
5	Login dengan field kosong	Buka URL dashboard.php secara langsung melalui browser tanpa sesi login aktif	URL: dashboard.php (tanpa login)	Sistem menampilkan pesan validasi "Email dan Password harus diisi!" dan tidak memproses login	Berhasil

2. Modul dashboard

Pengujian modul dashboard dilakukan untuk memastikan tampilan statistik dan data rapat ditampilkan sesuai dengan hak akses masing-masing pengguna.

Tabel 4.2 Hasil Pengujian Black-Box Modul Dashboard

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Menampilkan statistik dashboard	Login sebagai Admin → buka Dashboard	(akun Admin)	Menampilkan statistik jumlah rapat, chart mingguan, dan daftar rapat terbaru	Berhasil
2	Akses dashboard sebagai User biasa	Login sebagai User (level 2) → buka Dashboard	(akun User)	Dashboard tampil dengan data sesuai hak akses User	Berhasil

3. Modul Live Transcription

Pengujian modul live transcription bertujuan untuk memverifikasi bahwa sistem mampu mentranskripsi suara secara real-time dengan benar serta menangani berbagai kondisi operasional.

Tabel 4.3 Hasil Pengujian Black-Box Modul Live Transcription

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Memulai live transcription	Isi nama rapat → klik "Start Listening" →	Nama rapat: "Rapat Sprint Planning", Input suara:	Status berubah ke "Listening", teks transkripsi muncul secara	

		izinkan akses mikrofon → bicara	kalimat Bahasa Indonesia	real-time di layar	Berhasil
2	Menghentikan live transcription	Setelah live berjalan → klik "End Listening"	(live sedang berjalan)	Status berubah ke "Stopped", teks final ditampilkan, file .txt tersimpan	Berhasil
3	Menyimpan notulensi live	Setelah stop listening → klik "Simpan Notulen"	Teks transkripsi hasil live	Data tersimpan ke database, ringkasan otomatis diproses, panel ringkasan muncul	Berhasil
4	Live transcription tanpa server Python	Klik "Start Listening" saat server Python (port 5000) tidak berjalan	(server offline)	Muncul pesan error bahwa server tidak tersedia	Berhasil
5	Reset transcription	Setelah transkripsi selesai → klik "Reset"	(ada teks transkripsi)	Area transkripsi dikosongkan, siap untuk sesi baru	Berhasil
6	Akses halaman notulensi sebagai User biasa	Login sebagai User (level 2) → buka halaman Notulensi	(akun User)	Di-redirect ke Dashboard (hanya Admin yang boleh akses)	Berhasil

4. Modul Upload Video

Pengujian modul upload video dilakukan untuk memastikan sistem dapat memproses berbagai kondisi unggahan file video secara tepat.

Tabel 4.4 Hasil Pengujian Black-Box Modul Upload Video

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Upload file video valid	Drag & drop atau browse file MP4 → isi judul rapat → klik "Upload & Transkripsi"	File: rapat.mp4 (50MB+), Judul: "Rapat Divisi IT"	Progress bar muncul, file dikirim ke server, status "Proses Berjalan" tampil	Berhasil
2	Upload file format tidak didukung	Pilih file dengan format tidak valid → klik Upload	File: dokumen.pdf	Muncul pesan error "Format tidak didukung"	Berhasil
3	Upload tanpa memilih file	Klik "Upload & Transkripsi" tanpa memilih file	(tidak ada file)	Tombol Upload tetap disabled, tidak bisa diklik	Berhasil
4	Upload tanpa judul rapat	Pilih file video → biarkan judul kosong → klik Upload	File: rapat.mp4, Judul: (kosong)	Upload berhasil, judul otomatis diambil dari nama file video	Berhasil
5	Melihat progress proses background	Setelah upload → lihat card "Proses Berjalan"	(proses sedang berjalan)	Card menampilkan status fase (Mentranskripsi/Meringkas) dengan	

				persentase progress	Berhasil
--	--	--	--	---------------------	----------

5. Modul Report

Pengujian modul report bertujuan untuk memverifikasi bahwa sistem mampu menampilkan, memfilter, dan mengunduh hasil notulensi rapat dengan benar.

Tabel 4.5 Hasil Pengujian Black-Box Modul Report

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Melihat daftar hasil rapat	Buka halaman Report	(data tersedia di database)	Tabel menampilkan semua hasil rapat dengan judul, waktu, tipe (Live/Upload), dan tombol Detail	Berhasil
2	Filter report berdasarkan tipe	Klik filter "Live" atau "Upload"	(data tersedia)	Tabel hanya menampilkan hasil sesuai filter yang dipilih	Berhasil
3	Melihat detail	Klik tombol "Detail" pada	(ringkasan tersedia)	Modal muncul menampilkan ringkasan	

	ringkasan rapat	salah satu hasil rapat		terstruktur (Pembukaan, Topik, Kesimpulan, Penutup)	Berhasil
4	Download notulensi sebagai PDF	Klik "Unduh Notulensi (.pdf)" di modal detail	(ringkasan tersedia)	File PDF ter-download dengan format notulensi yang rapi	Berhasil

6. Modul Pengaturan

Pengujian modul pengaturan dilakukan untuk memvalidasi fitur manajemen pengguna yang hanya dapat diakses oleh Admin.

Tabel 4.6 Hasil Pengujian Black-Box Modul Pengaturan

No	Skenario Pengujian	Prosedur pengujian	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Tambah user baru (Admin)	Login Admin → Pengaturan → isi form Tambah User → klik Tambah	Nama: "User Baru", Email: user@test.com, Password: password123, Level: User	User baru berhasil ditambahkan, muncul di daftar pengguna	Berhasil
2	Ubah password sendiri	Klik "Ubah Password" → isi password baru → klik Simpan	Password baru: newpass123, Konfirmasi: newpass123	Muncul popup "Password berhasil diubah!"	Berhasil
3	Hapus user (Admin)	Login Admin → Pengaturan → klik icon hapus pada user lain → konfirmasi	(user target yang bukan diri sendiri)	User terhapus dari daftar, tidak bisa login lagi	Berhasil

4.2.5 Pengujian White-Box (Struktural)

Pengujian White-Box menguji jalur logika internal kode program. Pendekatan ini memfokuskan pengujian pada percabangan (branch), kondisi (condition), dan alur eksekusi kode guna memastikan setiap jalur logika berjalan dengan benar.

1. Proses Login (proses_login.php)

Pengujian white-box pada modul proses login bertujuan untuk memastikan setiap percabangan logika validasi dan autentikasi pengguna ditangani dengan benar.

Tabel 4.7 Hasil Pengujian White-Box Proses Login

No	Jalur yang Diuji	Kondisi	Test Data	Hasil yang Diharapkan	Status
1	Jalur: sudah login → redirect	isset(\$_SESSION['id']) == true	Session sudah aktif	Langsung redirect ke dashboard.php, proses login tidak dijalankan	Berhasil
2	Jalur: method bukan POST → redirect	\$_SERVER['REQUEST_METHOD'] != 'POST'	Akses langsung via GET	Redirect ke login.php	Berhasil
3	Jalur: input kosong → error	empty(\$email) empty(\$password)	Email: "", Password: ""	Set session error, redirect ke login.php	Berhasil

4	Jalur: email ditemukan + password bcrypt valid	password_verify() == true	Email ada, password cocok dengan hash bcrypt	Set session (id, nama, email, level), redirect ke dashboard.php	Berhasil
5	Jalur: email ditemukan + password plain text → auto- upgrade	\$data['password'] === \$password (plain text match)	Email ada, password masih plain text di DB	Login berhasil + password di- upgrade ke bcrypt hash secara otomatis	Berhasil
6	Jalur: email ditemukan + password salah	password_verify() == false && plain text != match	Email ada, password tidak cocok	Set session error "Email atau Password salah!", redirect ke login.php	Berhasil
7	Jalur: email tidak ditemukan	mysqli_num_rows(\$result) == 0	Email tidak ada di database	Set session error "Email atau Password salah!", redirect ke login.php	Berhasil

2. Validasi File Upload (server.py — api_transcribe)

Pengujian white-box pada modul validasi file upload menguji setiap jalur kondisi yang mungkin terjadi saat pengguna mengunggah file video.

Tabel 4.8 Hasil Pengujian White-Box Validasi File Upload

No	Jalur yang Diuji	Kondisi	Test Data	Hasil yang Diharapkan	Status
----	---------------------	---------	-----------	--------------------------	--------

1	Jalur: field video_file tidak ada	'video_file' not in request.files	POST tanpa field video_file	Return JSON error 400: "Field 'video_file' tidak ditemukan"	Berhasil
2	Jalur: filename kosong	not file.filename	Field ada tapi tidak ada file terpilih	Return JSON error 400: "Tidak ada file yang dipilih"	Berhasil
3	Jalur: ekstensi tidak diizinkan	allowed_file() == False	File: dokumen.exe	Return JSON error 400: "Format 'exe' tidak didukung"	Berhasil
4	Jalur: file valid → proses background	allowed_file() == True	File: rapat.mp4	File disimpan, record DB dibuat, background thread dimulai, return JSON success	Berhasil

3. Voice Activity Detection/VAD (server.py — process_live_audio)

Pengujian jalur logika pada modul Voice Activity Detection bertujuan untuk memverifikasi bahwa sistem dapat mendeteksi dan memproses aktivitas suara secara akurat sesuai parameter yang ditetapkan.

Tabel 4.9 Hasil Pengujian White-Box Voice Activity Detection

No	Jalur yang Diuji	Kondisi	<i>Test Data</i>	Hasil yang Diharapkan	Status
----	------------------	---------	------------------	-----------------------	--------

1	Jalur: buffer terlalu pendek → skip	$\text{buf_dur} < \text{min_process_sec} (2.0\text{s})$	Buffer audio 0.5 detik	Fungsi return tanpa proses, menunggu buffer lebih panjang	Berhasil
2	Jalur: RMS di bawah threshold → flush silence	$\text{rms} < \text{vad_threshold} (0.01)$	Audio hening (RMS = 0.002)	<code>current_text</code> dipindahkan ke <code>history_text</code> , buffer di-trim	Berhasil
3	Jalur: RMS di atas threshold → proses Whisper	$\text{rms} \geq \text{vad_threshold} (0.01)$	Audio ada suara (RMS = 0.05)	Buffer dikirim ke Whisper <code>submit_and_wait()</code> , hasil diproses	Berhasil
4	Jalur: buffer melebihi max → force flush	$\text{buf_dur} > \text{buffer_max_sec} (20.0\text{s})$	Buffer audio 25 detik	<code>current_text</code> → <code>history_text</code> , buffer direset ke kosong	Berhasil

4. Hallucination Filter (`hallucination_filter.py`)

Pengujian pada modul hallucination filter menguji setiap kondisi yang digunakan untuk mendeteksi dan membuang hasil transkripsi yang dianggap merupakan halusinasi model.

Tabel 4.10 Hasil Pengujian White-Box Hallucination Filter

No	Jalur yang Diuji	Kondisi	<i>Test Data</i>	Hasil yang Diharapkan	Status
----	------------------	---------	------------------	-----------------------	--------

1	Jalur: teks kosong → halusinasi	<code>len(text.strip()) < min_len (4)</code>	Teks: "" atau "ok"	Return True (dianggap halusinasi, teks dibuang)	Berhasil
2	Jalur: exact match frasa halusinasi	<code>lower == phrase</code>	Teks: "terima kasih sudah menonton"	Return True (frasa ada di daftar halusinasi)	Berhasil
3	Jalur: frasa berulang >2 kali → halusinasi	<code>lower.count(phrase) > 2</code>	Teks: "subscribe subscribe subscribe subscribe"	Return True (frasa muncul >2 kali = repetisi)	Berhasil
4	Jalur: separuh pertama == separuh kedua → halusinasi	<code>half.strip() == rest.strip() (>=6 kata)</code>	Teks: "jadi kita harus jadi kita harus"	Return True (pengulangan terdeteksi)	Berhasil
5	Jalur: teks normal → bukan halusinasi	Semua kondisi di atas False	Teks: "Kita perlu menyelesaikan fitur dashboard minggu ini"	Return False (teks valid, tidak dibuang)	Berhasil

5. File Transcription — Chunking & Loop (transcriber.py)

Pengujian jalur logika pada modul transkripsi file menguji proses pembagian audio menjadi potongan-potongan (chunk) dan validasi hasil transkripsi dari setiap potongan tersebut.

Tabel 4.11 Hasil Pengujian White-Box File Transcription Chunking

No	Jalur yang Diuji	Kondisi	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Jalur: teks kosong → halusinasi	<code>len(text.strip()) < min_len (4)</code>	Teks: "" atau "ok"	Return True (dianggap halusinasi, teks dibuang)	Berhasil
2	Jalur: exact match frasa halusinasi	<code>lower == phrase</code>	Teks: "terima kasih sudah menonton"	Return True (frasa ada di daftar halusinasi)	Berhasil
3	Jalur: frasa berulang >2 kali → halusinasi	<code>lower.count(phrase) > 2</code>	Teks: "subscribe subscribe subscribe subscribe"	Return True (frasa muncul >2 kali = repetisi)	Berhasil
4	Jalur: separuh pertama == separuh kedua → halusinasi	<code>half.strip() == rest.strip() (>=6 kata)</code>	Teks: "jadi kita harus jadi kita harus"	Return True (pengulangan terdeteksi)	Berhasil

6. LexRank Pipeline (summarize/lexrank.py)

Pengujian pada modul LexRank pipeline menguji setiap jalur kondisi dalam proses peringkasan teks, termasuk penanganan teks pendek dan ketersediaan server Ollama untuk abstractive rewriting.

Tabel 4.12 Hasil Pengujian White-Box LexRank Pipeline

--	--	--	--	--	--

No	Jalur yang Diuji	Kondisi	<i>Test Data</i>	Hasil yang Diharapkan	Status
1	Jalur: teks terlalu pendek → return langsung	<code>len(text.strip()) < 50</code>	Teks: "Rapat selesai." (15 karakter)	Return teks apa adanya tanpa proses LexRank	Berhasil
2	Jalur: kalimat ≤ target → skip LexRank	<code>len(filtered) ≤ target_sentences</code>	Teks pendek menghasilkan 5 kalimat, target 8	Semua kalimat digunakan langsung tanpa scoring LexRank	Berhasil
3	Jalur: Ollama tersedia → abstractive rewriting	<code>self.ollama.is_available() == True</code>	Ollama server berjalan di localhost:11434	Kalimat extractive dikirim ke LLM untuk ditulis ulang menjadi notulensi berstruktur	Berhasil
4	Jalur: Ollama tidak tersedia → fallback extractive	<code>self.ollama.is_available() == False</code>	Ollama server tidak berjalan	Output format extractive: kalimat di-polish dan dikelompokkan per 3 kalimat per paragraf	Berhasil

7. Database Operations (server.py)

Pengujian jalur logika pada modul operasi database memastikan sistem dapat menangani keberhasilan maupun kegagalan operasi penyimpanan data dengan baik.

Tabel 4.13 Hasil Pengujian White-Box Database Operations

No		Kondisi	<i>Test Data</i>		Status
----	--	---------	------------------	--	--------

	Jalur yang Diuji			Hasil yang Diharapkan	
1	Jalur: insert DB berhasil	conn.commit()) sukses	Data valid: judul, tipe, txt_path	Record baru tersimpan, return lastrowid > 0	Berhasil
2	Jalur: insert DB gagal → return 0	Exception saat pymysql.connect() atau cur.execute()	Database offline atau tabel tidak ada	Print error log, return 0 (tidak crash)	Berhasil

4.9 Pembahasan

Berdasarkan seluruh hasil pengujian yang telah dilakukan, berikut pembahasan terhadap masing-masing aspek pengujian pada sistem notulensi rapat otomatis yang dikembangkan.

4.3.1 Pembahasan Pengujian Akurasi Whisper Trancription

1. Fitur live rapat
2. Fitur upload rapat

Tahap akhir pada fitur Upload File dalam sistem notulensi rapat otomatis adalah pembentukan notulensi rapat terstruktur menggunakan model LLM lokal Llama 3.2 3B. Pada tahap ini, hasil transkripsi yang telah diringkas menggunakan metode LexRank diproses kembali secara abstraktif untuk menghasilkan notulensi yang lebih koheren dan mudah dipahami. Evaluasi kualitas ringkasan dilakukan menggunakan metrik ROUGE-L yang mengukur tingkat kesamaan urutan kata terpanjang (Longest Common Subsequence) antara notulensi yang dihasilkan

sistem dan notulensi referensi yang disusun secara manual. Berdasarkan hasil pengujian terhadap data uji pada fitur Upload File, diperoleh nilai rata-rata ROUGE-L F1-Score sebesar 16,20%, Precision sebesar 24,10%, dan Recall sebesar 12,46%.

Secara umum, nilai ROUGE-L yang berada pada kisaran belasan hingga dua puluh persen tidak dapat disamakan dengan metrik akurasi pada tugas klasifikasi. Dalam konteks Natural Language Processing (NLP), khususnya pada tugas abstractive summarization, nilai tersebut masih tergolong wajar karena model tidak hanya memilih kalimat dari sumber asli, tetapi juga menyusun kembali informasi menggunakan struktur dan kosakata yang berbeda. Akibatnya, meskipun makna yang disampaikan serupa, perbedaan pemilihan kata dan susunan kalimat akan mengurangi tingkat kecocokan kata demi kata yang diukur oleh ROUGE-L.

Selain itu, terdapat perbedaan gaya penulisan antara notulensi referensi dan notulensi yang dihasilkan oleh Llama 3.2. Notulensi referensi yang dibuat secara manual cenderung menggunakan format yang lebih ringkas, singkatan tertentu, serta pencatatan poin-poin penting secara langsung. Sebaliknya, model Llama 3.2 menghasilkan notulensi dalam bentuk bahasa Indonesia yang lebih formal, lengkap, dan terstruktur sesuai dengan prompt yang diberikan. Perbedaan gaya penulisan tersebut menyebabkan tingkat overlap kata menjadi lebih rendah meskipun informasi utama yang disampaikan tetap serupa.

Perbandingan nilai Precision dan Recall juga memberikan gambaran mengenai karakteristik ringkasan yang dihasilkan. Nilai Precision sebesar 24,10% yang lebih tinggi dibandingkan Recall sebesar 12,46% menunjukkan bahwa informasi yang dituliskan oleh model umumnya relevan dengan isi rapat dan memiliki tingkat penyimpangan informasi yang rendah. Di sisi lain, nilai Recall yang lebih rendah mengindikasikan bahwa ringkasan yang dihasilkan cenderung lebih padat dan hanya memuat poin-poin utama dibandingkan notulensi referensi yang mencatat informasi secara lebih rinci. Dengan demikian, hasil pengujian menunjukkan bahwa

fitur Upload File mampu menghasilkan notulensi yang ringkas serta tetap mempertahankan informasi penting dari hasil transkripsi rapat.

4.3.2 Pembahasan Pengujian Hybrid Extractive Lexrank

1. Fitur Live Rapat
2. Fitur Upload Rapat

Pada fitur Upload File, tahap peringkasan ekstraktif dilakukan menggunakan metode Hybrid Extractive LexRank untuk menyeleksi kalimat-kalimat yang dianggap paling representatif dari hasil transkripsi rapat. Evaluasi terhadap tahap ini dilakukan menggunakan tiga metrik utama, yaitu Cosine Similarity, Sentence Coverage, dan Compression Ratio. Berdasarkan hasil pengujian terhadap sembilan dataset skenario rapat, diperoleh nilai rata-rata Cosine Similarity sebesar 62,52%, Sentence Coverage sebesar 100,00%, dan Compression Ratio sebesar 7,25%.

Nilai rata-rata Cosine Similarity sebesar 62,52% menunjukkan bahwa ringkasan yang dihasilkan mampu mempertahankan sebagian besar informasi dan konteks penting yang terdapat pada transkripsi asli. Dalam konteks transkripsi rapat yang umumnya mengandung pengulangan informasi, percakapan di luar topik, maupun jeda komunikasi antar peserta, tingkat kemiripan semantik di atas 60% menunjukkan bahwa metode LexRank mampu mengidentifikasi dan mengekstraksi kalimat-kalimat yang paling relevan terhadap inti pembahasan rapat. Hal ini dimungkinkan karena LexRank menggunakan pendekatan berbasis graf yang menghitung tingkat keterhubungan antar kalimat untuk menentukan kalimat yang memiliki nilai informasi paling tinggi.

Hasil pengujian juga menunjukkan nilai Sentence Coverage sebesar 100,00%. Nilai tersebut mengindikasikan bahwa seluruh kalimat yang terdapat pada ringkasan merupakan kalimat yang diambil langsung dari hasil transkripsi tanpa adanya modifikasi, penambahan, maupun pengurangan isi informasi. Hasil ini

sesuai dengan karakteristik metode ekstraktif yang hanya memilih kalimat-kalimat penting dari dokumen sumber. Dalam arsitektur sistem yang dikembangkan, kondisi ini menjadi penting karena memastikan bahwa informasi yang diteruskan ke tahap peringkasan abstraktif tetap berasal dari fakta dan isi pembicaraan yang sebenarnya terjadi selama rapat.

Sementara itu, nilai Compression Ratio sebesar 7,25% menunjukkan bahwa metode LexRank mampu mereduksi panjang transkripsi secara signifikan dengan hanya mempertahankan sebagian kecil teks yang dianggap paling penting. Dengan kata lain, sebagian besar isi transkripsi yang bersifat repetitif atau kurang relevan berhasil disaring sebelum memasuki tahap pemrosesan berikutnya. Tingkat kompresi yang tinggi ini memberikan keuntungan dalam proses peringkasan abstraktif menggunakan LLM, karena jumlah teks yang harus diproses menjadi lebih kecil sehingga penggunaan sumber daya komputasi lebih efisien dan proses pembangkitan notulensi dapat dilakukan dengan lebih cepat.

Secara keseluruhan, hasil pengujian menunjukkan bahwa implementasi LexRank pada fitur Upload File mampu menjalankan fungsi ekstraksi informasi secara efektif dengan tetap mempertahankan konteks utama rapat, menjamin integritas informasi, serta mengurangi ukuran teks secara signifikan sebelum diproses lebih lanjut oleh model Llama 3.2 pada tahap abstractive summarization.

4.3.3 Pembahasan Pengujian Ollama Abstractive Summarization

Tahap akhir pada fitur Upload File dalam sistem notulensi rapat otomatis adalah proses abstractive summarization menggunakan model Large Language Model (LLM) Llama 3.2 3B yang dijalankan secara lokal melalui Ollama. Pada tahap ini, kalimat-kalimat penting hasil ekstraksi LexRank diolah kembali untuk menghasilkan notulensi rapat yang lebih terstruktur, memiliki tata bahasa yang baik, serta mudah dipahami oleh pengguna. Berbeda dengan metode ekstraktif yang hanya memilih kalimat dari sumber asli, pendekatan abstraktif memungkinkan

model menyusun ulang informasi menggunakan kalimat baru tanpa menghilangkan inti pembahasan rapat.

Evaluasi kualitas ringkasan pada fitur Upload File dilakukan menggunakan metrik ROUGE-L (Longest Common Subsequence), yang membandingkan tingkat kesamaan antara notulensi yang dihasilkan sistem dan notulensi referensi yang dibuat secara manual. Berdasarkan hasil pengujian terhadap sembilan skenario rapat, diperoleh nilai rata-rata ROUGE-L F1-Score sebesar 16,20%, ROUGE-L Precision sebesar 24,10%, dan ROUGE-L Recall sebesar 12,46%.

Nilai ROUGE-L F1-Score sebesar 16,20% menunjukkan tingkat keseimbangan antara ketepatan dan cakupan informasi yang berhasil dipertahankan oleh sistem. Meskipun nilai tersebut terlihat relatif rendah jika dibandingkan dengan metrik evaluasi pada tugas klasifikasi, hasil tersebut masih tergolong wajar dalam konteks abstractive summarization. Hal ini disebabkan karena model Llama 3.2 tidak sekadar menyalin kalimat dari sumber, melainkan menyusun ulang informasi menggunakan struktur kalimat dan pilihan kata yang berbeda. Akibatnya, tingkat kecocokan kata secara langsung dengan notulensi referensi menjadi lebih rendah meskipun makna yang disampaikan tetap serupa.

Nilai ROUGE-L Precision sebesar 24,10% yang merupakan nilai tertinggi di antara ketiga metrik menunjukkan bahwa informasi yang dihasilkan oleh model memiliki tingkat relevansi yang baik terhadap isi rapat. Hasil ini mengindikasikan bahwa sebagian besar informasi yang dituliskan dalam notulensi merupakan informasi yang benar dan sesuai dengan konteks pembahasan. Selain itu, penggunaan LexRank pada tahap sebelumnya turut membantu menjaga kualitas masukan yang diberikan kepada LLM karena kalimat yang diproses telah melalui tahap seleksi berdasarkan tingkat kepentingannya. Dengan demikian, kemungkinan munculnya informasi yang tidak relevan atau halusinasi dapat diminimalkan.

Sementara itu, nilai ROUGE-L Recall sebesar 12,46% menunjukkan bahwa tidak seluruh informasi yang terdapat pada notulensi referensi berhasil dimasukkan

ke dalam hasil ringkasan sistem. Kondisi ini menunjukkan bahwa notulensi yang dihasilkan cenderung lebih ringkas dan berfokus pada poin-poin utama dibandingkan notulensi yang disusun secara manual. Dalam praktiknya, notulensi yang dibuat oleh manusia sering kali memuat detail tambahan, penjelasan yang lebih panjang, maupun format pencatatan tertentu yang tidak selalu direproduksi oleh model. Oleh karena itu, nilai Recall yang lebih rendah tidak selalu menunjukkan kualitas ringkasan yang buruk, melainkan mencerminkan kemampuan sistem dalam memadatkan informasi dan menyajikannya secara lebih efisien.

Secara keseluruhan, hasil pengujian menunjukkan bahwa fitur Upload File mampu menghasilkan notulensi rapat yang terstruktur, relevan, dan ringkas. Kombinasi antara metode LexRank dan Llama 3.2 memungkinkan sistem mempertahankan informasi penting dari hasil transkripsi sekaligus menyajikannya dalam bentuk notulensi yang lebih mudah dipahami oleh pengguna.

4.3.4 Pembahasan Pengujian *Black Box*

Pengujian black box dilakukan secara fungsional dari sisi pengguna tanpa melihat kode internal sistem. Metode yang digunakan adalah Equivalence Partitioning dan Boundary Value Analysis untuk memastikan setiap fungsi utama sistem menghasilkan keluaran yang sesuai dengan yang diharapkan pada berbagai kondisi masukan. Pengujian mencakup serangkaian skenario yang meliputi fitur autentikasi pengguna, perekaman suara mikrofon secara langsung (live transcription), validasi pengunggahan berkas media (audio/video), pemrosesan transkripsi berbasis kecerdasan buatan, perangkuman otomatis melalui pipeline LexRank dan LLM, serta fitur manajemen dan ekspor hasil notulensi rapat.

4.3.5 Pembahasan Pengujian *White Box*

Pengujian white box dengan metode Basis Path Testing terhadap beberapa fungsi kritis sistem menunjukkan bahwa seluruh jalur logika, percabangan, dan kondisi batas berjalan sesuai dengan implementasi kode. Pada fungsi `process_live_audio()`, pengujian batas kritis (Voice Activity Detection) menunjukkan bahwa sistem dengan tepat melewati pemrosesan (skip) ketika durasi buffer kurang dari 2 detik atau ketika nilai amplitudo (RMS) di bawah threshold (hening) untuk menghindari noise, dan baru mengirimkan buffer ke model Whisper ketika ada deteksi suara yang jelas. Fungsi `is_hallucination()` berhasil menangkap seluruh variasi keluaran yang mengindikasikan halusinasi model AI, termasuk deteksi frasa yang diabaikan (exact match), anomali teks yang terlalu pendek, hingga pengulangan kata (repetisi) yang berlebihan, serta mengembalikan nilai aman False untuk teks transkripsi percakapan yang normal. Pada tahapan perangkuman di `lexrank.py`, pengujian membuktikan bahwa sistem secara efisien melewati scoring jika teks terlalu pendek. Mekanisme perangkuman juga berhasil menerapkan logika berlapis: memprioritaskan penulisan ulang secara abstractive menggunakan server LLM lokal (Ollama), dan secara mulus beralih (fallback) menggunakan metode ekstraktif LexRank murni apabila mendeteksi koneksi server LLM sedang offline atau tidak tersedia.

BAB V

SIMPULAN

5.1 Kesimpulan

5.2 Saran

JADWAL KEGIATAN PENELITIAN

RENCANA ANGGARAN PENELITIAN

DAFTAR PUSTAKA

- [1] F. Cahya Prista and S. Wulandari, "Implementasi Firebase Cloud Messaging (FCM) Pada Aplikasi Agenda Rapat," *JSAI: Journal Scientific and Applied Informatics*, vol. 7, no. 3, 2024, doi: 10.36085.
- [2] R. Campos *et al.*, "NLP for Local Governance Meeting Records: A Focus Article on Tasks, Datasets, Metrics and Benchmark," Feb. 2026, [Online]. Available: <http://arxiv.org/abs/2602.08162>
- [3] J. J. Koay, A. Roustai, X. Dai, and F. Liu, "A Sliding-Window Approach to Automatic Creation of Meeting Minutes," Apr. 2021, [Online]. Available: <http://arxiv.org/abs/2104.12324>
- [4] M. Arnold, M. Goldschmitt, and T. Rigotti, "Dealing with information overload: a comprehensive review," 2023, *Frontiers Media SA*. doi: 10.3389/fpsyg.2023.1122200.
- [5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," Dec. 2022, [Online]. Available: <http://arxiv.org/abs/2212.04356>
- [6] G. , & R. D. R. Erkan, "LexRank: Graph-based Lexical Centrality as Salience in Text Summarization," 2004.
- [7] Halimah, Surya Agustian, and Siti Ramadhani, "Peringkasan teks otomatis (automated text summarization) pada artikel berbahasa indonesia menggunakan algoritma lexrank," *Jurnal CoSciTech (Computer Science and Information Technology)*, vol. 3, no. 3, pp. 371–381, Dec. 2022, doi: 10.37859/coscitech.v3i3.4300.

- [8] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” Jul. 2023, [Online]. Available: <http://arxiv.org/abs/2307.09288>
- [9] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” Dec. 2022, [Online]. Available: <http://arxiv.org/abs/2212.04356>
- [10] M. Kaushik, M. Trinkle, and A. Hashemi-Sakhtsari, *Automatic Detection and Removal of Disfluencies from Spontaneous Speech*. 2010.
- [11] F. A. Silveira, “2-Terminal Coaxial Capacitance Measurements and Cable Corrections,” Jul. 2024, [Online]. Available: <http://arxiv.org/abs/2211.00757>
- [12] D. Daly, W. Brown, H. Ingo, J. O’Leary, and D. Bradford, “The use of change point detection to identify software performance regressions in a continuous integration system,” in *ICPE 2020 - Proceedings of the ACM/SPEC International Conference on Performance Engineering*, Association for Computing Machinery, Inc, Apr. 2020, pp. 67–75. doi: 10.1145/3358960.3375791.
- [13] A. Vaswani *et al.*, “Attention Is All You Need,” Aug. 2023, [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [14] K.-M. Kim, B. Hyeon, Y. Kim, J.-H. Park, and S. Lee, “Findings of the Association for Computational Linguistics Multi-pretraining for Large-scale Text Classification,” 2020.

- [15] O. Herman and M. Jakubíček, “ShadowSense: a Multi-annotated Dataset for Evaluating Word Sense Induction,” 1476. [Online]. Available: <https://github.com/ai-ku/>
- [16] X. Li, X. Shan, W. Yin, and H. Wang, “Heterogeneous Graph Pre-training Based Model for Secure and Efficient Prediction of Default Risk Propagation among Bond Issuers,” Jan. 2025, [Online]. Available: <http://arxiv.org/abs/2501.03268>
- [17] M. Barański, J. Jasiński, J. Bartolewska, S. Kacprzak, M. Witkowski, and K. Kowalczyk, “Investigation of Whisper ASR Hallucinations Induced by Non-Speech Audio,” Jan. 2025, doi: 10.1109/ICASSP49660.2025.10890105.
- [18] M. Ravaut, S. Joty, and N. F. Chen, “SummaReranker: A Multi-Task Mixture-of-Experts Re-ranking Framework for Abstractive Summarization,” Long Papers, 2022. [Online]. Available: <https://github.com/ntunlp/>
- [19] M. Lewis *et al.*, “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” Oct. 2019, [Online]. Available: <http://arxiv.org/abs/1910.13461>
- [20] A. Musyafa, Y. Gao, A. Solyman, C. Wu, and S. Khan, “Automatic Correction of Indonesian Grammatical Errors Based on Transformer,” *Applied Sciences (Switzerland)*, vol. 12, no. 20, Oct. 2022, doi: 10.3390/app122010380.
- [21] G. Erkan and D. R. Radev, “LexRank: Graph-based Lexical Centrality as Saliency in Text Summarization,” *Journal of Artificial Intelligence Research*, 2004.

- [22] S. A. and S. R. Halimah, “PERINGKASAN TEKS OTOMATIS (AUTOMATED TEXT SUMMARIZATION) PADA ARTIKEL BERBAHASA INDONESIA MENGGUNAKAN ALGORITMA LEXRANK,” 2022.
- [23] C.-Y. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” 2004.
- [24] S. Divya, N. Sripriya, J. Andrew, and M. Mazzara, “Unified extractive-abstractive summarization: a hybrid approach utilizing BERT and transformer models for enhanced document summarization,” *PeerJ Comput. Sci.*, vol. 10, pp. 1–26, 2024, doi: 10.7717/peerj-cs.2424.